Citation audit

Every cited work, the sentence citing it, and where the claim sits in the source

Ha Vien

Generated 2026-08-31

How to read this document

This document lists every work in `bibliography.bib`. It is ordered by the number the work carries in the full draft's reference list, so entry [**17**] here is reference [**17**] there and the two can be read side by side. Where a work is also cited by `Thesis_v2` or by the plan, the number it carries in those documents is given as well, because their lists are shorter and number the same work differently.

Each entry answers three questions in order. What is the source. Which sentences cite it, printed as they stand in the thesis with the citation shown as [**n**]. And what the source itself says, quoted with the page or clause it was read from. When the three agree, the citation is sound. When the quoted passage does not carry the sentence above it, that is the defect this document is built to expose.

Checking one entry. Read the citing sentence, then read the quotation under *Source says*. Ask only whether the second supports the first. The evidence level tells you how far to trust the quotation without opening the paper yourself.

Level A
: The source was opened and the supporting passage located. A page, clause or section is given. Nothing further is needed.

Level B
: A publisher-deposited record, such as a Crossref abstract or an official landing page, carries the claim. The full text was not opened.

Level C
: Bibliographic metadata is confirmed and the passage sits behind a paywall. Worth opening through the university proxy if the claim is load-bearing.

No level
: The record is a literature-sweep note rather than a located passage, so it gives no page or clause. Part 3 lists these together and they are the ones to close first.

Provenance. Nothing in this document is typed by hand. References are printed by biblatex from `bibliography.bib`. Printed citation numbers are read from each document's `.bbl`, which is what biber wrote, and the generator refuses to run when a `.bbl` disagrees with its source. Citing sentences are copied from the `.tex` sources with the generated measurement macros resolved, so a sentence quoting a measured value shows the value. Evidence comes from `docs/Sources_evidence.md`, with `docs/LITERATURE_NOTES.md` as the fallback, and the origin is named under each entry. Regenerate with:

```
.bibaudit/build_audit.py && latexmk -pdf source_audit.tex
```

One note on punctuation. The thesis style forbids em dashes and prose semicolons anywhere a reader sees them, and this document follows that rule in its own prose. Quotations from cited sources keep their original punctuation, because altering a quotation misquotes it. Any dash or semicolon below therefore sits inside quoted source material and is deliberate.

Coverage. 109 entries, all of them cited. 109 in the full draft, 52 in `Thesis_v2`, 31 in the plan. 71 at level A, 32 at level B, 6 at level C, and 0 with no located passage.

Part 1. Master checklist

One row per entry, ordered by the full draft's reference number. **v2** and **pl** give the number the same work carries in `Thesis_v2` and in the plan, and a blank means that document does not cite it. **cit** counts the places the work is cited across all three documents. **ev** is the evidence level, blank where no passage was located, and **status** reads *ok* where the verdict is clean, *note* where the record raises something, and *sweep* where the only record is a literature-sweep note. Tick the last column as you check.

#	v2	pl	key	source	kind	cit	ev	status	
1	1	1	`3gpp38401`	3GPP 2023	standard	9	A	ok	☐
2	2		`3gpp38300`	3GPP 2023	standard	4	A	ok	☐
3	3		`polese2023oran`	Polese et al. 2023	peer reviewed	6	A	ok	☐
4	4		`3gpp38331`	3GPP 2023	standard	7	C	note	☐
5	5		`3gpp38913`	3GPP 2024	standard	7	A	note	☐
6	6	8	`dean2013tail`	Dean 2013	peer reviewed	13	B	ok	☐
7	7		`barroso2017microseconds`	Barroso et al. 2017	peer reviewed	6	B	ok	☐
8	8	2	`msrc2019memory`	Miller 2019	grey	8	A	ok	☐
9	9		`google2022android`	Vander Stoep 2022	grey	7	A	ok	☐
10	10		`klabnik2023rust`	Klabnik 2023	book	4	C	ok	☐
11	11	4	`jung2018rustbelt`	Jung et al. 2018	peer reviewed	5	A	ok	☐
12	12		`balasubramanian2017rust`	Balasubramanian et al. 2017	peer reviewed	9	B	ok	☐
13	13	10	`itu2023imt2030`	ITU-R 2023	standard	5	A	ok	☐
14	14		`polese2026airan`	Polese et al. 2026	peer reviewed	2	B	ok	☐
15	15		`airan2026orchestrator`	AI-RAN Alliance 2026	grey	2	A	ok	☐
16	16		`vanderstoep2025android`	Vander Stoep 2025	grey	3	A	ok	☐
17	17		`google2024securebydesign`	Rebert 2024	grey	3	B	ok	☐
18	18		`watson2025memorysafety`	Watson et al. 2025	peer reviewed	2	A	note	☐
19	19	3	`cisa2023memorysafe`	Cybersecurity and Infrastructure Security Agency 2023	standard	6	A	note	☐
20	20		`oncd2024building`	Office of the National Cyber Director 2024	standard	5	A	ok	☐
21	21		`eu2024cra`	European Parliament and Council of the European Union 2024	regulation	2	A	note	☐
22	22		`agache2020firecracker`	Agache et al. 2020	peer reviewed	3	A	note	☐
23	23		`delloro2025ran3q`	Dell'Oro Group 2025	grey	2	A	ok	☐
24	24		`delloro2026ran`	Dell'Oro Group 2026	grey	2	A	ok	☐
25	25	9	`3gpp2024road6g`	Almodovar 2024	grey	3	A	ok	☐
26	26		`cisa2025badpractices`	Cybersecurity and Infrastructure Security Agency 2025	standard	2	A	ok	☐
27	27		`qin2020rust`	Qin et al. 2020	peer reviewed	5	A	ok	☐
28	34		`3gpp22261`	3GPP 2022	standard	3	C	note	☐
29			`3gpp38463`	3GPP 2022	standard	1	A	ok	☐
30			`li2014tales`	Li et al. 2014	peer reviewed	2	A	ok	☐
31	28	27	`schroeder2006openclosed`	Schroeder et al. 2006	peer reviewed	4	A	ok	☐
32	38	24	`georges2007rigorous`	Georges et al. 2007	peer reviewed	6	A	ok	☐

#	v2	pl	key	source	kind	cit	ev	status	
33	30		szekeres2013sok	Szekeres et al. 2013	peer reviewed	4	A	ok	☐
34			bensalem2005deadlock	Bensalem 2005	peer reviewed	1	A	ok	☐
35			tsandeadlock	The Sanitizers Project 2026	tool record	1	A	ok	☐
36			sutter2025profiles	Sutter 2025	committee draft	1	A	ok	☐
37			baxter2024safecpp	Baxter 2024	committee draft	1	A	ok	☐
38			vandevoorde2026profiles	Vandevoorde et al. 2026	committee draft	3	A	ok	☐
39	31		jung2021safe	Jung et al. 2021	peer reviewed	5	A	ok	☐
40			jung2020stacked	Jung et al. 2020	peer reviewed	2	A	ok	☐
41			astrauskas2020unsafe	Astrauskas et al. 2020	peer reviewed	2	B	ok	☐
42		21	xu2021rustcve	Xu et al. 2022	peer reviewed	4	A	note	☐
43	52		evans2020rust	Evans et al. 2020	peer reviewed	6	A	ok	☐
44			bugden2022rust	Bugden 2022	peer reviewed	2	A	ok	☐
45	35	23	enberg2019threadpercore	Enberg et al. 2019	peer reviewed	5	A	ok	☐
46		16	skoudlil2025asyncrust	Skoudlil et al. 2025	peer reviewed	3	A	ok	☐
47	50	30	rivera2021galeed	Rivera et al. 2021	peer reviewed	9	B	ok	☐
48	51		kirth2022pkrusafe	Kirth et al. 2022	peer reviewed	7	B	ok	☐
49	37		li2022crosslang	Li et al. 2022	peer reviewed	4	C	ok	☐
50			cai2026interop	Cai et al. 2026	peer reviewed	3	B	ok	☐
51			pearce2026rustmc	Pearce et al. 2026	peer reviewed	2	A	ok	☐
52	32	11	karlback2021ran	Karlback 2021	thesis	10	A	note	☐
53	46	12	olsson2023ranmgmt	Olsson 2023	thesis	8	A	ok	☐
54		13	olsson2023asyncpackets	Olsson 2023	thesis	5	A	ok	☐
55	44	14	arvidsson2025async	Arvidsson 2025	thesis	12	A	ok	☐
56	33	15	emberg2023asn1	Emberg 2023	thesis	8	A	ok	☐
57		20	gjomemo2016mesc	Gjomemo et al. 2016	peer reviewed	2	B	ok	☐
58			michaelides2026ranlatency	Michaelides et al. 2026	peer reviewed	1	A	ok	☐
59	36	29	natella2016fault	Natella et al. 2016	peer reviewed	4	B	ok	☐
60	39	25	efron1993bootstrap	Efron 1993	book	4	B	note	☐
61			rfc3393	Demichelis 2002	standard	2	A	ok	☐
62	40	26	holm1979simple	Holm 1979	peer reviewed	3	B	ok	☐
63	29	28	tene_hdrhistogram	Tene	tool record	6	A	ok	☐
64	41		lizard	Yin 2026	tool record	3	A	ok	☐
65	42		zhou2025oltp	Zhou et al. 2025	peer reviewed	3	A	ok	☐
66			clangtidyuseaftermove	The LLVM Project 2026	tool record	1	A	ok	☐
67			hughes2025gcrust	Hughes 2025	peer reviewed	1	B	ok	☐
68			cui2024rcanary	Cui et al. 2024	peer reviewed	1	A	ok	☐
69			dietz2015overflow	Dietz et al. 2015	peer reviewed	1	A	ok	☐
70	43		bornholt2021shardstore	Bornholt et al. 2021	peer reviewed	2	A	ok	☐
71		6	emmerich2019drivers	Emmerich et al. 2019	peer reviewed	7	A	ok	☐
72		7	zhang2022runtime	Zhang et al. 2022	peer reviewed	5	A	ok	☐
73			harris2023morello	Harris et al. 2023	peer reviewed	4	A	ok	☐
74	45	22	zhang2026deadlock	Zhang et al. 2026	peer reviewed	3	A	ok	☐
75			tang2025converos	Tang et al. 2025	peer reviewed	1	A	ok	☐
76			lu2008mistakes	Lu et al. 2008	peer reviewed	1	A	ok	☐
77	47		sundvisson2026drivers	Sundvisson 2026	thesis	4	A	ok	☐
78	48	31	fulton2021adopting	Fulton et al. 2021	peer reviewed	6	A	ok	☐
79	49		crichton2024learning	Crichton 2024	peer reviewed	3	A	ok	☐
80			li2026rustlinux	Li et al. 2026	peer reviewed	3	B	ok	☐
81			fan2026android	Fan et al. 2026	peer reviewed	2	B	ok	☐

#	v2	pl	key	source	kind	cit	ev	status	
82		17	li2021ipc	Li et al. 2021	peer reviewed	4	A	ok	☐
83			narayanan2020redleaf	Narayanan et al. 2020	peer reviewed	2	A	ok	☐
84		18	suchert2023stamp	Suchert 2023	peer reviewed	4	B	ok	☐
85		19	gonzalez2023rustudp	Gonzalez et al. 2023	peer reviewed	4	B	ok	☐
86			zhuang2026flowprioritize	Zhuang 2026	peer reviewed	1	B	ok	☐
87			andersson2026stdmemsafety	Andersson 2026	thesis	1	A	ok	☐
88			okawa2024ruststudy	Okawa 2024	peer reviewed	1	B	ok	☐
89			pirelli2023safelowlevel	Pirelli 2023	peer reviewed	2	B	ok	☐
90			qin2024safety	Qin et al. 2024	peer reviewed	2	B	ok	☐
91		5	pereira2017energy	Pereira et al. 2017	peer reviewed	2	A	ok	☐
92			levy2017kernel	Levy et al. 2017	peer reviewed	1	B	ok	☐
93			mo2023rustshyper	Mo et al. 2023	peer reviewed	1	C	ok	☐
94			xiao2024rpkt	Xiao et al. 2025	peer reviewed	1	C	ok	☐
95			cui2025perfbugs	Cui 2025	peer reviewed	1	B	ok	☐
96			naylor2026cherisimt	Naylor et al. 2026	peer reviewed	1	A	ok	☐
97			bae2021rudra	Bae et al. 2021	peer reviewed	1	B	ok	☐
98			zheng2024security	Zheng et al. 2024	peer reviewed	1	A	ok	☐
99			schuermann2023encapsulated	Schuermann et al. 2023	peer reviewed	1	B	ok	☐
100			emre2023aliasing	Emre et al. 2023	peer reviewed	1	B	ok	☐
101			sharma2024embedded	Sharma et al. 2024	peer reviewed	1	A	ok	☐
102			mccormack2026unsafe	McCormack et al. 2026	peer reviewed	1	A	ok	☐
103			hong2025ctorust	Hong 2025	peer reviewed	1	B	ok	☐
104			huang2017predictability	Huang et al. 2017	peer reviewed	1	B	ok	☐
105			liu2023l25gc	Liu et al. 2023	peer reviewed	1	B	ok	☐
106			park2025ctrlbench	Park 2025	peer reviewed	1	B	ok	☐
107			ardito2021verbosity	Ardito et al. 2021	peer reviewed	1	B	ok	☐
108			qazi2017pepc	Qazi et al. 2017	peer reviewed	1	A	ok	☐
109			alsoran	Larkin	tool record	1	A	ok	☐

Part 2. Entry records

Entry [**n**] is reference [**n**] of the full draft. Citing sentences are printed as they stand, with the citation shown as [**n**] and a cross-reference as *label*.

---

[1] `3gpp38401`

standard, level A · Thesis_v2 [1], Plan [1]

Reference. 3GPP. *NG-RAN; Architecture description.* Technical Specification (TS) 38.401. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: `https://www.3gpp.org/ftp/Specs/archive/38_series/38.401/38401-h50.zip`

Entry. source page. V17.5.0, Release 17.

Cited in. 9 places.

> **Full draft, Chapter 1.** The central unit is split again into control-plane and user-plane functions [**1, 2**].
>
> **Full draft, Section 1.1.** In the 3GPP disaggregated architecture it is the control-plane half of the central unit, hosting RRC and managing the UE context lifecycle [**1, 4**], under the millisecond-scale procedure budgets the 5G system places on control signalling [**5**] and at the event rates and connected-device counts a deployed network sustains.
>
> **Full draft, Section 2.1.** The NG-RAN architecture splits the gNB into a central unit (gNB-CU) and distributed units, with the control plane (gNB-CU-CP) handling RRC and connection management for each UE [**1, 2, 4**].
>
> **Full draft, Section 2.1.** These parts communicate over standardised interfaces: NG towards the 5G core, F1 between the CU and the DU, and E1 between the CU-CP and the CU-UP [**1, 2, 29**].
>
> **Full draft, Section 6.2.** It is the RRC connection lifecycle the 3GPP specifications define for a UE, from establishment through in-session signalling and bearer setup to reconfiguration and release [**1, 4**].
>
> **Thesis_v2, Chapter 1.** The central unit is split again into control-plane and user-plane functions [**1, 2**].
>
> **Thesis_v2, Section 1.1.** In the 3GPP disaggregated architecture it is the control-plane half of the central unit, hosting RRC and managing the UE context lifecycle [**1, 4**], under the millisecond-scale procedure budgets the 5G system places on control signalling [**5**] and at the event rates and connected-device counts a deployed network sustains.
>
> **Thesis_v2, Chapter 2.** UE Context Management sits in the control plane, the gNB-CU-CP, and talks to its neighbours over the standardised F1, E1, and NG interfaces [**1**].
>
> **Plan, Section 1.** Modern RAN control-plane software (the L3 of a gNB, e.g. the gNB-CU-CP [**1**]) is written mostly in C++, which delivers the throughput and latency such systems need but carries well-known memory-safety and concurrency risks (use-after-free, data races, double-free).

Claim it carries. the CU/DU split, the CU-CP/CU-UP separation, the F1/E1/NG interfaces, and the gNB-CU-CP as the node hosting RRC

Source says.

> *(`ts_138401v170500p.pdf`):*
>
> - *p. 11, clause 3.1: "gNB-CU-Control Plane (gNB-CU-CP): a logical node hosting the RRC and the control plane part of the PDCP protocol of the gNB-CU … terminates the E1 interface connected with the gNB-CU-UP and the F1-C interface connected with the gNB-DU."*
> - *p. 16, clause 6.1.2 "Overall architecture for separation of gNB-CU-CP and gNB-CU-UP", Figure 6.1.2-1.*
> - *Clause 10.4 "E1 interface".*

Metadata. [verified] `38401-h50.zip` in the 3GPP archive is dated **2023-06-29**, matching `jun 2023`. The ETSI mirror is *ETSI TS 138 401 V17.5.0 (2023-07)*. URL in the entry resolves.

Verdict. OK. This entry, not `3gpp38300`, is what carries the CP/UP separation claim.

Record. Sources_evidence.md, entry record.

---

[2] `3gpp38300`

standard, level A · Thesis_v2 [2]

Reference. 3GPP. *NR; NR and NG-RAN Overall description; Stage 2.* Technical Specification (TS) 38.300. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.300/38300-h50.zip

Entry. source page. V17.5.0, Release 17.

Cited in. 4 places.

> **Full draft, Chapter 1.** The central unit is split again into control-plane and user-plane functions **[1, 2]**.
>
> **Full draft, Section 2.1.** The NG-RAN architecture splits the gNB into a central unit (gNB-CU) and distributed units, with the control plane (gNB-CU-CP) handling RRC and connection management for each UE **[1, 2, 4]**.
>
> **Full draft, Section 2.1.** These parts communicate over standardised interfaces: NG towards the 5G core, F1 between the CU and the DU, and E1 between the CU-CP and the CU-UP **[1, 2, 29]**.
>
> **Thesis_v2, Chapter 1.** The central unit is split again into control-plane and user-plane functions **[1, 2]**.

Claim it carries. alongside 38.401 for architecture and functional split

Source says.

> *(`ts_138300v170500p.pdf`): p. 21, clause 4.1 "Overall Architecture" (an NG-RAN node is a gNB or an ng-eNB. Figure 4.1-1), p. 22, clause 4.2 "Functional Split", listing gNB functions including Radio Resource Management and "Connection setup and release".*

Metadata. [verified] `38300-h50.zip` dated **2023-06-30**. ETSI mirror *V17.5.0 (2023-07)*.

Note. 38.300 v17.5.0 does **not** describe the gNB-CU-CP / gNB-CU-UP separation, where it mentions CU functionality it defers to TS 38.401 (e.g. p. 19, p. 28). The pairing in the thesis is fine because 38.401 leads, but 38.300 should not be cited alone for the CP/UP split.

Verdict. OK with that note.

Record. Sources_evidence.md, entry record.

---

[3] `polese2023oran`

peer reviewed, level A · Thesis_v2 [3]

Reference. M. Polese et al. "Understanding O-RAN: Architecture, Interfaces, Algorithms, Security, and Research Challenges". In: *IEEE Communications Surveys & Tutorials* 25.2 (2023), pp. 1376–1411. DOI: 10.1109/COMST.2023.3239220. URL: https://doi.org/10.1109/COMST.2023.3239220

Entry. DOI 10.1109/COMST.2023.3239220.

Cited in. 6 places.

> **Full draft, Chapter 1.** Open-RAN pushes the same idea further: the RAN becomes virtualised, multi-vendor software linked by open interfaces **[3]**.
>
> **Full draft, Section 1.1.** This shift is accelerated by Open-RAN disaggregation, which turns the base station into virtualised, multi-vendor software connected by open interfaces **[3]**, so that the control plane a vendor ships increasingly competes on software-engineering quality (reliability, predictable timing, security, and the ease with which a large codebase can be evolved safely) and no longer on purpose-built hardware.

Full draft, Section 2.1. Open-RAN initiatives standardise these interfaces further so that each part can be supplied by a different vendor and run as software on commodity hardware **[3]**.

Full draft, Appendix B. The fourth strand is the domain and the method: the tail-latency and microsecond-scale literature **[6, 7, 30, 104]**, thread-per-core structuring moving the tail in its own right **[45]**, O-RAN disaggregation **[3]**, control-plane latency as an active target in the core network too **[105, 106]**, adoption evidence pointing at the learning curve rather than run-time cost **[78, 79, 107]**, and standard guidance on repeat-based evaluation and fault injection **[32, 59]**.

Thesis_v2, Chapter 1. Open-RAN pushes the same idea further: the RAN becomes virtualised, multi-vendor software linked by open interfaces **[3]**.

Thesis_v2, Section 1.1. This shift is accelerated by Open-RAN disaggregation, which turns the base station into virtualised, multi-vendor software connected by open interfaces **[3]**, so that the control plane a vendor ships increasingly competes on software-engineering quality (reliability, predictable timing, security, and the ease with which a large codebase can be evolved safely) and no longer on purpose-built hardware.

Claim it carries. O-RAN turns the RAN into virtualised, multi-vendor software connected by open interfaces

Source says.

(arXiv 2202.01032, p. 1, abstract): "***O-RAN promotes virtualized RANs where disaggregated components are connected via open interfaces*** *and optimized by intelligent controllers . . .* ***O-RAN networks can be built with multi-vendor, interoperable components.***"

Metadata. [verified] Crossref matches every field. (The apostrophe in D'Oro differs typographically only.)

Verdict. OK, a close match to the thesis's phrasing.

Record. Sources_evidence.md, entry record.

---

[4] `3gpp38331`

standard, level C · Thesis_v2 [4]

Reference. 3GPP. *NR; Radio Resource Control (RRC); Protocol specification.* Technical Specification (TS) 38.331. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), July 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.331/38331-h50.zip

Entry. source page. V17.5.0, Release 17.

Cited in. 7 places.

Full draft, Chapter 1. It tracks every connected device, runs RRC signalling **[4]**, and coordinates mobility and handovers, all within strict latency limits.

Full draft, Section 1.1. In the 3GPP disaggregated architecture it is the control-plane half of the central unit, hosting RRC and managing the UE context lifecycle **[1, 4]**, under the millisecond-scale procedure budgets the 5G system places on control signalling **[5]** and at the event rates and connected-device counts a deployed network sustains.

Full draft, Section 2.1. The NG-RAN architecture splits the gNB into a central unit (gNB-CU) and distributed units, with the control plane (gNB-CU-CP) handling RRC and connection management for each UE **[1, 2, 4]**.

Full draft, Section 3.1. It is modelled on a real Nokia gNB-CU-CP service domain that owns the UE context lifecycle, RRC procedure execution **[4]**, and mobility/handover, which are the functions the 3GPP architecture assigns to the control-plane part of a disaggregated gNB (Section *sec:ranarch*).

Full draft, Section 6.2. It is the RRC connection lifecycle the 3GPP specifications define for a UE, from establishment through in-session signalling and bearer setup to reconfiguration and release **[1, 4]**.

Thesis_v2, Chapter 1. It tracks every connected device, runs RRC signalling **[4]**, and coordinates mobility and handovers, all within strict latency limits.

> **Thesis_v2, Section 1.1.** In the 3GPP disaggregated architecture it is the control-plane half of the central unit, hosting RRC and managing the UE context lifecycle **[1, 4]**, under the millisecond-scale procedure budgets the 5G system places on control signalling **[5]** and at the event rates and connected-device counts a deployed network sustains.

Claim it carries. RRC signalling procedures and the UE lifecycle from establishment to release

Source says.

> *The RRC connection control procedures are clause 5.3 (establishment, reconfiguration, re-establishment, release). The document itself was not opened for this audit, it is 1,000 pages and the claim is structural rather than numeric.*

Metadata. [nit] `38331-h50.zip` is dated **2023-07-01**, the entry says June 2023. ETSI mirror is *V17.5.0 (2023-07)*. One-month slip, cosmetic.

Verdict. Metadata: change month to `jul` if you want it exact.

Record. Sources_evidence.md, entry record.

---

[5] `3gpp38913`

standard, level A · Thesis_v2 [5]

Reference. 3GPP. *Study on scenarios and requirements for next generation access technologies.* Technical Report (TR) 38.913. V18.0.0, Release 18. Clause 7.4 sets the control plane latency target, for the transition from a battery-efficient state to the start of continuous data transfer, at 10 ms. 3rd Generation Partnership Project (3GPP), Apr. 2024. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.913/38913-i00.zip

Entry. source page. V18.0.0, Release 18. Clause 7.4 sets the control plane latency target, for the transition from a battery-efficient state to the start of continuous data transfer, at 10 ms.

Cited in. 7 places.

> **Full draft, Chapter 1.** A single site can run thousands of connection-management and mobility procedures per second, each of which has to complete inside the millisecond-scale budget the 5G system sets for control-plane signalling **[5]**.

> **Full draft, Section 1.1.** In the 3GPP disaggregated architecture it is the control-plane half of the central unit, hosting RRC and managing the UE context lifecycle **[1, 4]**, under the millisecond-scale procedure budgets the 5G system places on control signalling **[5]** and at the event rates and connected-device counts a deployed network sustains.

> **Full draft, Section 2.1.** Such control-plane systems must satisfy demanding non-functional requirements: very high event rates (UE mobility, signalling storms), strict and predictable latency bounds that the 5G service and RAN specifications quantify as KPIs **[5, 28]**, long-lived software with continuously evolving complexity, and heavily concurrent workloads.

> **Full draft, Section 3.1.** It is core RAN layer-3 functionality, and it carries the properties that make production control-plane code difficult: per-UE mutable state, a strict operation lifecycle, high concurrency across UEs, and the latency and reliability targets the 5G system sets for control-plane procedures **[5, 28]**.

> **Thesis_v2, Chapter 1.** A single site can run thousands of connection-management and mobility procedures per second, each of which has to complete inside the millisecond-scale budget the 5G system sets for control-plane signalling **[5]**.

> **Thesis_v2, Section 1.1.** In the 3GPP disaggregated architecture it is the control-plane half of the central unit, hosting RRC and managing the UE context lifecycle **[1, 4]**, under the millisecond-scale procedure budgets the 5G system places on control signalling **[5]** and at the event rates and connected-device counts a deployed network sustains.

> **Thesis_v2, Chapter 3.** It has mutable state per device, a strict lifecycle, heavy concurrency, and hard latency targets **[5, 34]**.

Claim it carries. the millisecond-scale control-plane budget, and specifically the 10 ms target

Source says.

> *(`tr_138913v180000p.pdf`, **p. 24**, clause 7.4 "Control plane latency"): "Control plane latency refers to the time to move from a battery efficient state (e.g., IDLE) to start of continuous data transfer (e.g., ACTIVE). **The target for control plane latency should be 10ms.**" This is exactly the claim at `Thesis_v2.tex:128` and the wording already in the entry's `note` field.*

Metadata. [nit] month. `38913-i00.zip` is dated **2024-04-04**. ETSI mirror is *ETSI TR 138 913 V18.0.0 (2024-05)*. The entry says March 2024. → `month = apr`.

Verdict. Claim verified, fix the month.

Record. Sources_evidence.md, entry record.

---

[6] `dean2013tail`

peer reviewed, level B · Thesis_v2 [6], Plan [8]

Reference. J. Dean and L. A. Barroso. "The Tail at Scale". In: *Communications of the ACM* 56.2 (2013), pp. 74–80. DOI: 10.1145/2408776.2408794. URL: https://doi.org/10.1145/2408776.2408794

Entry. DOI 10.1145/2408776.2408794.

Cited in. 13 places.

> **Full draft, Chapter 1.** Every device the site serves is affected [**6, 7**].
>
> **Full draft, Chapter 1.** emphTail latency, the p99 and p99.9 percentiles rather than the mean, is the property that, more than average speed, decides whether such a system meets its deadlines [**6, 7**].
>
> **Full draft, Section 1.2.** Second, what governs whether RAN deadlines are met is not average throughput but worst-case and tail behaviour under load and overload [**6**], a property that is easy to regress and hard to measure correctly.
>
> **Full draft, Section 2.2.** At the scale of a network, tail latency dominates user-visible behaviour: when a single user action fans out into many internal operations, the slowest of them sets the response time, so rare per-operation delays become common per-request delays [**6**].
>
> **Full draft, Section 2.4.** Java, C# and Go achieve memory safety through a runtime and garbage collection, which is an excellent trade for much software and a poor one for a tail-latency-critical control plane: collection pauses are a recognised source of tail latency in large systems [**6**], and a managed runtime gives up the control over memory layout and scheduling this domain relies on.
>
> **Full draft, Section 7.6.** RQ1 is answered with high percentiles and the maximum instead of the mean, which is the standard way of measuring predictability in this literature [**6**].
>
> **Full draft, Appendix B.** The fourth strand is the domain and the method: the tail-latency and microsecond-scale literature [**6, 7, 30, 104**], thread-per-core structuring moving the tail in its own right [**45**], O-RAN disaggregation [**3**], control-plane latency as an active target in the core network too [**105, 106**], adoption evidence pointing at the learning curve rather than run-time cost [**78, 79, 107**], and standard guidance on repeat-based evaluation and fault injection [**32, 59**].
>
> **Thesis_v2, Chapter 1.** Every device the site serves is affected [**6, 7**].
>
> **Thesis_v2, Chapter 1.** emphTail latency, the p99 and p99.9 percentiles rather than the mean, is the property that, more than average speed, decides whether such a system meets its deadlines [**6, 7**].
>
> **Thesis_v2, Section 1.2.** Second, what governs whether RAN deadlines are met is not average throughput but worst-case and tail behaviour under load and overload [**6**], a property that is easy to regress and hard to measure correctly.
>
> **Thesis_v2, Chapter 2.** The metrics that matter are the p99 and p99.9 latencies and the jitter around them [**6**].

Thesis_v2, Chapter 3. A p99.9 measured on a single operation becomes a steady rate of degraded procedures in the field [**6**].

Plan, Section 1. For a RAN control plane the operative metric is not the average but the tail [**8**].

Claim it carries. fan-out makes a rare per-operation latency a common per-request latency

Source says.

Crossref-deposited abstract: "Software techniques that tolerate latency variability are vital to building responsive large-scale Web services." The fan-out argument is the paper's central section ("The Tail at Scale", §"Why Variability Exists" and §"Reducing Component Variability").

Metadata. [verified] Crossref matches every field.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[7] `barroso2017microseconds`

peer reviewed, level B · Thesis_v2 [7]

Reference. L. Barroso et al. "Attack of the Killer Microseconds". In: *Communications of the ACM* 60.4 (2017), pp. 48–54. DOI: 10.1145/3015146. URL: https://doi.org/10.1145/3015146

Entry. DOI 10.1145/3015146.

Cited in. 6 places.

Full draft, Chapter 1. Every device the site serves is affected [**6, 7**].

Full draft, Chapter 1. emphTail latency, the p99 and p99.9 percentiles rather than the mean, is the property that, more than average speed, decides whether such a system meets its deadlines [**6, 7**].

Full draft, Section 2.2. Modern data-centre and networking software increasingly operates in a regime where the relevant latencies are measured in microseconds, a range in which conventional operating-system and software overheads are no longer negligible [**7**].

Full draft, Appendix B. The fourth strand is the domain and the method: the tail-latency and microsecond-scale literature [**6, 7, 30, 104**], thread-per-core structuring moving the tail in its own right [**45**], O-RAN disaggregation [**3**], control-plane latency as an active target in the core network too [**105, 106**], adoption evidence pointing at the learning curve rather than run-time cost [**78, 79, 107**], and standard guidance on repeat-based evaluation and fault injection [**32, 59**].

Thesis_v2, Chapter 1. Every device the site serves is affected [**6, 7**].

Thesis_v2, Chapter 1. emphTail latency, the p99 and p99.9 percentiles rather than the mean, is the property that, more than average speed, decides whether such a system meets its deadlines [**6, 7**].

Claim it carries. at microsecond scale, conventional OS and software overheads stop being negligible

Source says.

*Crossref-deposited abstract: "**Microsecond-scale I/O means tension between performance and productivity** that will need new latency-mitigating ideas, including in hardware."*

Metadata. [verified] Crossref matches.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[8] `msrc2019memory`

grey, level A · Thesis_v2 [8], Plan [2]

Reference. M. Miller. *Trends, challenge, and shifts in software vulnerability mitigation.* BlueHat IL, Microsoft Security Response Center (MSRC). Approximately 70% of Microsoft CVEs are memory-safety issues. 2019. URL: https://github.com/microsoft/MSRC-Security-Research/tree/master/presentations/2019_02_BlueHatIL

Entry. source page. Approximately 70% of Microsoft CVEs are memory-safety issues.

Cited in. 8 places.

> **Full draft, Chapter 1.** C++ places the burden of memory and concurrency management on the programmer, and decades of experience show that this burden is routinely mishandled, producing use-after-free errors, double-frees, data races, and related defects that are expensive to find, dangerous in production, and responsible for a large share of security vulnerabilities in industry practice [**8, 9**].
>
> **Full draft, Section 1.1.** Large software organisations report that the majority of their serious security vulnerabilities stem from memory-safety errors, and that adopting memory-safe languages measurably reduces them [**8, 9, 16, 17**].
>
> **Full draft, Section 2.3.** Memory-safety issues account for roughly 70 % of the security vulnerabilities reported at Microsoft [**8**], and Google reports a sharp drop in such vulnerabilities as memory-safe languages are adopted in Android, to below a fifth of that platform's total vulnerabilities by 2025 [**9, 16**].
>
> **Full draft, Appendix B.** The safety strand is settled the same partial way, with machine-checked foundations for the safe fragment and the aliasing discipline `unsafe` code has to respect [**11, 39, 40**], confirmation that `unsafe` is used sparingly and that memory-safety CVEs always involve it [**27, 41, 42**], auditing that finds defects at scale even so and unsafety spreading through dependency chains [**97, 98, 90, 43**], and the reports motivating the shift set against a long record of memory corruption resisting runtime mitigation [**8, 9, 17, 19, 20, 33**].
>
> **Full draft, Appendix G.** Adopting Rust removes use-after-free, double-free, iterator invalidation, and data races from the defect budget entirely, and those are the classes that dominate the security statistics [**8, 9**].
>
> **Thesis_v2, Chapter 1.** C++ places the burden of memory and concurrency management on the programmer, and decades of experience show that this burden is routinely mishandled, producing use-after-free errors, double-frees, data races, and related defects that are expensive to find, dangerous in production, and responsible for a large share of security vulnerabilities in industry practice [**8, 9**].
>
> **Thesis_v2, Section 1.1.** Large software organisations report that the majority of their serious security vulnerabilities stem from memory-safety errors, and that adopting memory-safe languages measurably reduces them [**8, 9, 16, 17**].
>
> **Plan, Section 1.** These classes dominate real defect populations, and migrating to memory-safe languages is now an explicit policy position [**2, 3**].

Claim it carries. most sharply at 751: "memory-safety issues account for roughly 70 % of the security vulnerabilities reported at Microsoft"

Source says.

> *(deck **p. 11**): " **70% of the vulnerabilities addressed through a security update each year continue to be memory safety issues**", above the chart "% of memory safety vs. non-memory safety CVEs by patch year", 2006–2018.*

Metadata. [verified] Matt Miller, MSRC, February 2019. The repository path in the URL contains the deck: `2019_02 - BlueHatIL - Trends, challenge, and shifts in software vulnerability mitigation.pdf`.

Verdict. OK. The source says "addressed through a security update" rather than "reported", the entry's `note` ("Approximately 70 % of Microsoft CVEs are memory-safety issues") matches the chart title.

Record. Sources_evidence.md, entry record.

[9] `google2022android`

grey, level A · Thesis_v2 [9]

Reference. J. Vander Stoep. *Memory Safe Languages in Android 13*. Google Security Blog. Dec. 2022. URL: https://security.googleblog.com/2022/12/memory-safe-languages-in-android-13.html

Entry. source page.

Cited in. 7 places.

> **Full draft, Chapter 1.** C++ places the burden of memory and concurrency management on the programmer, and decades of experience show that this burden is routinely mishandled, producing use-after-free errors, double-frees, data races, and related defects that are expensive to find, dangerous in production, and responsible for a large share of security vulnerabilities in industry practice **[8, 9]**.
>
> **Full draft, Section 1.1.** Large software organisations report that the majority of their serious security vulnerabilities stem from memory-safety errors, and that adopting memory-safe languages measurably reduces them **[8, 9, 16, 17]**.
>
> **Full draft, Section 2.3.** Memory-safety issues account for roughly 70 % of the security vulnerabilities reported at Microsoft **[8]**, and Google reports a sharp drop in such vulnerabilities as memory-safe languages are adopted in Android, to below a fifth of that platform's total vulnerabilities by 2025 **[9, 16]**.
>
> **Full draft, Appendix B.** The safety strand is settled the same partial way, with machine-checked foundations for the safe fragment and the aliasing discipline `unsafe` code has to respect **[11, 39, 40]**, confirmation that `unsafe` is used sparingly and that memory-safety CVEs always involve it **[27, 41, 42]**, auditing that finds defects at scale even so and unsafety spreading through dependency chains **[97, 98, 90, 43]**, and the reports motivating the shift set against a long record of memory corruption resisting runtime mitigation **[8, 9, 17, 19, 20, 33]**.
>
> **Full draft, Appendix G.** Adopting Rust removes use-after-free, double-free, iterator invalidation, and data races from the defect budget entirely, and those are the classes that dominate the security statistics **[8, 9]**.
>
> **Thesis_v2, Chapter 1.** C++ places the burden of memory and concurrency management on the programmer, and decades of experience show that this burden is routinely mishandled, producing use-after-free errors, double-frees, data races, and related defects that are expensive to find, dangerous in production, and responsible for a large share of security vulnerabilities in industry practice **[8, 9]**.
>
> **Thesis_v2, Section 1.1.** Large software organisations report that the majority of their serious security vulnerabilities stem from memory-safety errors, and that adopting memory-safe languages measurably reduces them **[8, 9, 16, 17]**.

Source says.

> *for identity and date, the specific "below a fifth by 2025" figure at `Ha_Vien_Masterthesis_2026.tex:754` is carried by `vanderstoep2025android` (below), which is cited alongside it.*

Metadata. [verified] Page title and URL confirm: *Memory Safe Languages in Android 13*, Google Online Security Blog, December 2022, Jeff Vander Stoep.

Verdict. OK.

Record. Sources_evidence.md, entry record.

[10] `klabnik2023rust`

book, level C · Thesis_v2 [10]

Reference. S. Klabnik and C. Nichols. *The Rust Programming Language*. 2nd. San Francisco, CA: No Starch Press, 2023. ISBN: 978-1-7185-0310-6

Cited in. 4 places.

> **Full draft, Chapter 1.** Rust has emerged as the first mainstream systems language to promise an escape from this trade-off: memory safety and data-race freedom enforced at compile time on machine-checked foundations [**10, 11**], with performance intended to be competitive with C++ [**12**].
>
> **Full draft, Section 2.4.** Rust enforces memory safety and data-race freedom at compile time through its ownership and borrowing model [**10**], and ships with modern tooling in Cargo and integrated testing.
>
> **Thesis_v2, Chapter 1.** Rust has emerged as the first mainstream systems language to promise an escape from this trade-off: memory safety and data-race freedom enforced at compile time on machine-checked foundations [**10, 11**], with performance intended to be competitive with C++ [**12**].
>
> **Thesis_v2, Chapter 2.** It checks memory safety and freedom from data races at compile time, using ownership, borrowing, and thread-safety traits such as `Send` [**10, 31**].

Claim it carries. ownership and borrowing, safety at compile time

Source says.

> *Book not opened. The relevant chapters are 4 (ownership) and 16 (fearless concurrency), add a chapter to the citation if you want it tighter.*

Metadata. [verified] OpenLibrary record for ISBN 9781718503106: *Rust Programming Language, 2nd Edition*, Steve Klabnik and Carol Nichols, No Starch Press, 2023.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[11] `jung2018rustbelt`

peer reviewed, level A · Thesis_v2 [11], Plan [4]

Reference. R. Jung et al. “RustBelt: Securing the Foundations of the Rust Programming Language”. In: *Proceedings of the ACM on Programming Languages* 2.POPL (2018), 66:1–66:34. DOI: 10.1145/3158154. URL: https://doi.org/10.1145/3158154

Entry. DOI 10.1145/3158154.

Cited in. 5 places.

> **Full draft, Chapter 1.** Rust has emerged as the first mainstream systems language to promise an escape from this trade-off: memory safety and data-race freedom enforced at compile time on machine-checked foundations [**10, 11**], with performance intended to be competitive with C++ [**12**].
>
> **Full draft, Section 2.4.** The soundness of the safe fragment of the language, including the role of `unsafe` as an explicitly marked escape hatch, has been established formally by the RustBelt project [**11**], and the aliasing discipline that even `unsafe` code must respect has been given a precise operational semantics [**40**].
>
> **Full draft, Appendix B.** The safety strand is settled the same partial way, with machine-checked foundations for the safe fragment and the aliasing discipline `unsafe` code has to respect [**11, 39, 40**], confirmation that `unsafe` is used sparingly and that memory-safety CVEs always involve it [**27, 41, 42**], auditing that finds defects at scale even so and unsafety spreading through dependency chains [**97, 98, 90, 43**], and the reports motivating the shift set against a long record of memory corruption resisting runtime mitigation [**8, 9, 17, 19, 20, 33**].
>
> **Thesis_v2, Chapter 1.** Rust has emerged as the first mainstream systems language to promise an escape from this trade-off: memory safety and data-race freedom enforced at compile time on machine-checked foundations [**10, 11**], with performance intended to be competitive with C++ [**12**].
>
> **Plan, Section 1.** Rust guarantees safety at compile time on machine-checked foundations [**4**].

Claim it carries. machine-checked foundations for safe Rust

Source says.

> *(p. 1, abstract, restated p. 2): "we give the **first formal (and machine-checked) safety proof** for a language representing a realistic subset of Rust."*

Metadata. [verified] The paper's own footer reads "Proceedings of the ACM on Programming Languages, Vol. 2, No. POPL, **Article 66. Publication date: January 2018**", so `year = 2018` and `pages = 66:1-66:34` are both right. (Crossref's "2017" is the online-first date, do not "correct" the entry to it.)

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[12] `balasubramanian2017rust`

peer reviewed, level B · Thesis_v2 [12]

Reference. A. Balasubramanian et al. "System Programming in Rust: Beyond Safety". In: *Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS)*. 2017, pp. 156–161. DOI: 10.1145/3102980.3103006. URL: https://doi.org/10.1145/3102980.3103006

Entry. DOI 10.1145/3102980.3103006.

Cited in. 9 places.

> **Full draft, Chapter 1.** Rust has emerged as the first mainstream systems language to promise an escape from this trade-off: memory safety and data-race freedom enforced at compile time on machine-checked foundations [**10, 11**], with performance intended to be competitive with C++ [**12**].
>
> **Full draft, Section 1.2.** The available evidence is largely general-purpose and average-case [**12, 27**], leaving these three questions (predictability, prevented-defect scope, and migration cost) open for a representative subsystem.
>
> **Full draft, Section 2.4.** Systems research argues Rust attains safety without the overhead of garbage collection or runtime checks, and even enables efficiency techniques that are impractical in C [**12**].
>
> **Full draft, Section 2.4.** Rust is the first widely supported language to combine compile-time memory and thread safety with the absence of a garbage collector and throughput and memory use alongside C and C++ [**12, 39, 44**].
>
> **Full draft, Section 7.2.** The median result is consistent with the wider evidence that Rust's safety checks are largely absorbed by modern out-of-order CPUs [**12, 71**].
>
> **Full draft, Section 7.5.** The "zero-cost" framing [**12**] is therefore best read as neither established nor refuted in general, but as a property of a particular program on a particular workload.
>
> **Full draft, Appendix B.** Compiled systems languages sit close on time and energy [**44, 91**], Rust's safety is cheap at run time [**12, 92**], and that holds where timing is tightest, in embedded hypervisors, production cloud infrastructure, and packet processing within a few percent of a C reference [**93, 22, 71, 94**].
>
> **Thesis_v2, Chapter 1.** Rust has emerged as the first mainstream systems language to promise an escape from this trade-off: memory safety and data-race freedom enforced at compile time on machine-checked foundations [**10, 11**], with performance intended to be competitive with C++ [**12**].
>
> **Thesis_v2, Section 1.2.** The available evidence is largely general-purpose and average-case [**12, 27**], leaving these three questions (predictability, prevented-defect scope, and migration cost) open for a representative subsystem.

Claim it carries. the zero-cost framing, and as of the 31 August citation-attachment pass, Chapter 1's clause on performance intended to be competitive with C++

Source says.

> *(Crossref-deposited abstract): "Rust is unique in that it **enforces safety without runtime overhead**, most importantly, without the overhead of garbage collection. While **zero-cost safety** is remarkable on its own, we argue that the superpowers of Rust go beyond safety. In particular, Rust's linear type system enables capabilities that cannot be implemented efficiently in traditional languages, both safe and unsafe."*

Verdict. OK, and this one now matters more than it did: the citation-attachment pass moved Chapter 1's performance clause onto this entry, and the deposited abstract carries that clause directly. §2.4's stronger reading, that Rust "even enables efficiency techniques that are impractical in C", is the abstract's own "capabilities that cannot be implemented efficiently in traditional languages".

Record. Sources_evidence.md, entry record.

---

[13] `itu2023imt2030`

standard, level A · Thesis_v2 [13], Plan [10]

Reference. ITU-R. *Framework and overall objectives of the future development of IMT for 2030 and beyond.* Recommendation ITU-R M.2160-0. Approved 13 November 2023. Defines the IMT-2030 ("6G") framework and its development timeline. International Telecommunication Union, Radiocommunication Sector, Nov. 2023. URL: https://www.itu.int/rec/R-REC-M.2160/en

Entry. source page. Approved 13 November 2023. Defines the IMT-2030 ("6G") framework and its development timeline.

Cited in. 5 places.

> **Full draft, Section 1.1.** IMT-2030, the ITU framework for 6G, lists artificial intelligence and communication among its six usage scenarios and ubiquitous intelligence among its four design principles **[13]**.

> **Full draft, Section 1.1.** IMT-2030 names security and resilience as one of its four design principles, expects the system to be secure by design, defines resilience as the ability to keep operating correctly during and after a disruptive event, and carries the ultra-reliable low-latency scenario forward in strengthened form **[13]**.

> **Thesis_v2, Section 1.1.** IMT-2030, the ITU framework for 6G, lists artificial intelligence and communication among its six usage scenarios and ubiquitous intelligence among its four design principles **[13]**.

> **Thesis_v2, Section 1.1.** IMT-2030 names security and resilience as one of its four design principles, expects the system to be secure by design, defines resilience as the ability to keep operating correctly during and after a disruptive event, and carries the ultra-reliable low-latency scenario forward in strengthened form **[13]**.

> **Plan, Section 1.** The decision has a long horizon. 3GPP has allocated Release 20 to the 6G studies and Release 21 to the normative work, with full specifications due to the ITU by mid-2030 and migration from 5G explicitly in scope **[9, 10]**, so a language chosen for the control plane of the current generation is inherited by the next.

Source says.

> *for the metadata. The ITU page gives M.2160-0 (11/2023), approved 2023-11, in force, title as in the entry*

Record. Sources_evidence.md, 24 August verification pass.

---

[14] `polese2026airan`

peer reviewed, level B · Thesis_v2 [14]

Reference. M. Polese et al. "Beyond Connectivity: An Open Architecture for AI-RAN Convergence in 6G". in: *IEEE Communications Magazine* 64.7 (2026), pp. 90–95. DOI: 10.1109/MCOM.001.2500438. URL: https://doi.org/10.1109/MCOM.001.2500438

Entry. DOI 10.1109/MCOM.001.2500438.

Cited in. 2 places.

> **Full draft, Section 1.1.** The architecture forming around that is AI-RAN, in which telecommunications and AI workloads are orchestrated together on shared, accelerated infrastructure, so the sites that run the radio also host distributed edge AI platforms **[14]**.

> **Thesis_v2, Section 1.1.** The architecture forming around that is AI-RAN, in which telecommunications and AI workloads are orchestrated together on shared, accelerated infrastructure, so the sites that run the radio also host distributed edge AI platforms [**14**].

Source says.

> *Crossref confirms IEEE Communications Magazine 64(7):90–95, 2026, five authors as listed*

Record. Sources_evidence.md, 24 August verification pass.

---

[15] `airan2026orchestrator`

grey, level A · Thesis_v2 [15]

Reference. AI-RAN Alliance. *AI-RAN Platform and Infrastructure Orchestrator*. AI-RAN Alliance, Working Group 2 (AI-and-RAN). Version 10-rc1, clause 2.2. Defines AI-and-RAN as one common IT infrastructure accommodating both AI and RAN workloads, which the infrastructure is required to run concurrently, and notes that the two kinds of workload may have different service-level agreements. Contributors from Ericsson, Nokia, Nvidia, Red Hat, Samsung, SoftBank, zTouch Networks and Northeastern University. Accessed August 2026. Feb. 2026. URL: https://ai-ran.org/documents/AI-RAN-Platform-Infrastructure-Orchestrator.pdf

Entry. source page. Version 10-rc1, clause 2.2. Defines AI-and-RAN as one common IT infrastructure accommodating both AI and RAN workloads, which the infrastructure is required to run concurrently, and notes that the two kinds of workload may have different service-level agreements. Contributors from Ericsson, Nokia, Nvidia, Red Hat, Samsung, SoftBank, zTouch Networks and Northeastern University. Accessed August 2026.

Cited in. 2 places.

> **Full draft, Section 1.1.** The AI-RAN Alliance states the operational requirement directly: one common infrastructure has to run both kinds of workload concurrently, and their service-level agreements may differ [**15**].

> **Thesis_v2, Section 1.1.** The AI-RAN Alliance states the operational requirement directly: one common infrastructure has to run both kinds of workload concurrently, and their service-level agreements may differ [**15**].

Source says.

> *clause 2.2: "The concept of 'AI-and-RAN' means using a common IT infrastructure to accommodate both AI and RAN workloads. Therefore, there is a fundamental requirement for the infrastructure to run AI and RAN workloads concurrently. It is important to note that AI and RAN workloads may have different SLAs." The contributor list in the note matches the eight organisations on page 2. Only the date was wrong, see above*

Record. Sources_evidence.md, 24 August verification pass.

---

[16] `vanderstoep2025android`

grey, level A · Thesis_v2 [16]

Reference. J. Vander Stoep. *Rust in Android: Move Fast and Fix Things*. Google Security Blog. Nov. 2025. URL: https://blog.google/security/rust-in-android-move-fast-fix-things/

Entry. source page.

Cited in. 3 places.

> **Full draft, Section 1.1.** Large software organisations report that the majority of their serious security vulnerabilities stem from memory-safety errors, and that adopting memory-safe languages measurably reduces them [**8, 9, 16, 17**].

> **Full draft, Section 2.3.** Memory-safety issues account for roughly 70 % of the security vulnerabilities reported at Microsoft [**8**], and Google reports a sharp drop in such vulnerabilities as memory-safe languages are adopted in Android, to below a fifth of that platform's total vulnerabilities by 2025 [**9, 16**].

> **Thesis_v2, Section 1.1.** Large software organisations report that the majority of their serious security vulnerabilities stem from memory-safety errors, and that adopting memory-safe languages measurably reduces them **[8, 9, 16, 17]**.

Claim it carries. memory-safety vulnerabilities falling below a fifth of Android's total by 2025

Source says.

> *(page metadata/description): "The 2025 data continues to validate the approach, with **memory safety vulnerabilities falling below 20% of total vulnerabilities for the first time**."*

Metadata. [verified] blog.google, author metadata "Jeff Vander Stoep", November 2025.

Verdict. OK, the thesis's "below a fifth ... by 2025" is the source's own claim.

Record. Sources_evidence.md, entry record.

---

[17] `google2024securebydesign`

grey, level B · Thesis_v2 [17]

Reference. A. Rebert and C. Kern. *Secure by Design: Google's Perspective on Memory Safety*. Google Security Engineering, Technical Report. 2024. URL: https://research.google/pubs/secure-by-design-googles-perspective-on-memory-safety/

Entry. source page.

Cited in. 3 places.

> **Full draft, Section 1.1.** Large software organisations report that the majority of their serious security vulnerabilities stem from memory-safety errors, and that adopting memory-safe languages measurably reduces them **[8, 9, 16, 17]**.

> **Full draft, Appendix B.** The safety strand is settled the same partial way, with machine-checked foundations for the safe fragment and the aliasing discipline `unsafe` code has to respect **[11, 39, 40]**, confirmation that `unsafe` is used sparingly and that memory-safety CVEs always involve it **[27, 41, 42]**, auditing that finds defects at scale even so and unsafety spreading through dependency chains **[97, 98, 90, 43]**, and the reports motivating the shift set against a long record of memory corruption resisting runtime mitigation **[8, 9, 17, 19, 20, 33]**.

> **Thesis_v2, Section 1.1.** Large software organisations report that the majority of their serious security vulnerabilities stem from memory-safety errors, and that adopting memory-safe languages measurably reduces them **[8, 9, 16, 17]**.

Source says.

> *Title and venue support the claim as used (vendor position on memory safety). No page-level quote taken.*

Metadata. [verified] research.google landing page: *Secure by Design: Google's Perspective on Memory Safety*, **Google Security Engineering (2024)**, authors Alex Rebert and Christoph Kern.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[18] `watson2025memorysafety`

peer reviewed, level A · Thesis_v2 [18]

Reference. R. N. M. Watson et al. "It Is Time to Standardize Principles and Practices for Software Memory Safety". In: *Communications of the ACM* 68.2 (2025), pp. 40–45. DOI: 10.1145/3708553. URL: https://doi.org/10.1145/3708553

Entry. DOI 10.1145/3708553.

Cited in. 2 places.

> **Full draft, Section 1.1.** The case no longer rests on vendor reporting alone: a group spanning academia and industry has argued that memory safety now needs standardised terminology and requirements if it is to be specified and procured at all **[18]**.

> **Thesis_v2, Section 1.1.** The case no longer rests on vendor reporting alone: a group spanning academia and industry has argued that memory safety now needs standardised terminology and requirements if it is to be specified and procured at all **[18]**.

Claim it carries. memory safety needs standardised terminology and requirements

Source says.

> *(UCAM-CL-TR-996, p. 1 and p. 3, the extended version of the same paper): "It is time to standardize principles and practices for software memory safety . . . we explore memory-safety standardization, which we argue is an essential step. . . ". Crossref's deposited CACM abstract: ". . . argue that standardization is an essential next step to achieving universal strong memory safety."*

Metadata. [wrong] author list. Crossref and OpenAlex both give **21** authors, the CACM abstract itself opens "**Twenty-one authors**, spanning academia and industry. . . ". The entry lists 22: the extra name is **Christoph Kern**, who appears on the cover of the *extended* version (Cambridge Computer Laboratory Technical Report UCAM-CL-TR-996) but not on the CACM article. CACM also gives "Murali Vijayaraghavan" (entry: "Muralidaran"), and orders Chisnall before Chen and Richardson before Rebert. Volume, issue, pages, year and DOI are all correct.

Verdict. Claim verified. **fix the author list** (fix #3).

Record. Sources_evidence.md, entry record.

---

[19] `cisa2023memorysafe`

standard, level A · Thesis_v2 [19], Plan [3]

Reference. Cybersecurity and Infrastructure Security Agency. *The Case for Memory Safe Roadmaps: Why Both C-Suite Executives and Technical Experts Need to Take Memory Safe Coding Seriously.* Cybersecurity Information Sheet. CISA, NSA, FBI, ASD's ACSC, CCCS, NCSC-UK, NCSC-NZ, CERT NZ, Dec. 2023. URL: https://www.cisa.gov/resources-tools/resources/case-memory-safe-roadmaps

Entry. source page.

Cited in. 6 places.

> **Full draft, Section 1.1.** This concern has risen to the level of national policy: cybersecurity agencies now urge software manufacturers to publish "memory-safe roadmaps" and to migrate away from memory-unsafe languages such as C and C++ **[19, 20]**.

> **Full draft, Section 1.4.** The intended readers are the vendor making the language decision, the standards discussion around memory-safe roadmaps **[19, 20]**, and anyone repeating a comparison of this shape, who can reuse the specification, the pair, the harness and the catalogue.

> **Full draft, Appendix B.** The safety strand is settled the same partial way, with machine-checked foundations for the safe fragment and the aliasing discipline `unsafe` code has to respect **[11, 39, 40]**, confirmation that `unsafe` is used sparingly and that memory-safety CVEs always involve it **[27, 41, 42]**, auditing that finds defects at scale even so and unsafety spreading through dependency chains **[97, 98, 90, 43]**, and the reports motivating the shift set against a long record of memory corruption resisting runtime mitigation **[8, 9, 17, 19, 20, 33]**.

> **Thesis_v2, Section 1.1.** This concern has risen to the level of national policy: cybersecurity agencies now urge software manufacturers to publish "memory-safe roadmaps" and to migrate away from memory-unsafe languages such as C and C++ **[19, 20]**.

> **Thesis_v2, Section 1.4.** The intended readers are the vendor making the language decision, the standards discussion around memory-safe roadmaps **[19, 20]**, and anyone repeating a comparison of this shape, who can reuse the specification, the pair, the harness and the catalogue.

> **Plan, Section 1.** These classes dominate real defect populations, and migrating to memory-safe languages is now an explicit policy position **[2, 3]**.

Claim it carries. the "publish memory-safe roadmaps" claim

Source says.

> *(**p. 2**): "the agencies urge software manufacturers to **create and publish memory safe roadmaps** that detail how they will eliminate memory safety vulnerabilities in their products."*

Metadata. [verified] for content, [nit] for the `author` field. Cover page (**p. 1**) gives "Publication: December 2023" and the eight authoring agencies in exactly the order the entry's `institution` field lists them: US CISA, US NSA, US FBI, Australian Signals Directorate's ACSC, Canadian Centre for Cyber Security, UK NCSC, New Zealand NCSC, CERT NZ. The `author` field, however, is a single braced string ending "and others" and will print as a run-on name.

Verdict. Claim verified, tidy the `author` field (fix #9).

Record. Sources_evidence.md, entry record.

---

[20] `oncd2024building`

standard, level A · Thesis_v2 [20]

Reference. Office of the National Cyber Director. *Back to the Building Blocks: A Path Toward Secure and Measurable Software*. Technical Report. Washington, DC: The White House, Office of the National Cyber Director (ONCD), Feb. 2024. URL: https://bidenwhitehouse.archives.gov/wp-content/uploads/2024/02/Final-ONCD-Technical-Report.pdf

Entry. source page.

Cited in. 5 places.

> **Full draft, Section 1.1.** This concern has risen to the level of national policy: cybersecurity agencies now urge software manufacturers to publish "memory-safe roadmaps" and to migrate away from memory-unsafe languages such as C and C++ **[19, 20]**.

> **Full draft, Section 1.4.** The intended readers are the vendor making the language decision, the standards discussion around memory-safe roadmaps **[19, 20]**, and anyone repeating a comparison of this shape, who can reuse the specification, the pair, the harness and the catalogue.

> **Full draft, Appendix B.** The safety strand is settled the same partial way, with machine-checked foundations for the safe fragment and the aliasing discipline `unsafe` code has to respect **[11, 39, 40]**, confirmation that `unsafe` is used sparingly and that memory-safety CVEs always involve it **[27, 41, 42]**, auditing that finds defects at scale even so and unsafety spreading through dependency chains **[97, 98, 90, 43]**, and the reports motivating the shift set against a long record of memory corruption resisting runtime mitigation **[8, 9, 17, 19, 20, 33]**.

> **Thesis_v2, Section 1.1.** This concern has risen to the level of national policy: cybersecurity agencies now urge software manufacturers to publish "memory-safe roadmaps" and to migrate away from memory-unsafe languages such as C and C++ **[19, 20]**.

> **Thesis_v2, Section 1.4.** The intended readers are the vendor making the language decision, the standards discussion around memory-safe roadmaps **[19, 20]**, and anyone repeating a comparison of this shape, who can reuse the specification, the pair, the harness and the catalogue.

Claim it carries. the policy push away from memory-unsafe languages

Source says.

> *(**p. 7**): "Experts have identified a few programming languages that both lack traits associated with memory safety and also have high proliferation across critical systems, such as C and C++", and "in most cases, **using a memory safe programming language is the most efficient way to substantially improve software security**".*

Metadata. [verified] Cover page: "BACK TO THE BUILDING BLOCKS: A PATH TOWARD SECURE AND MEASURABLE SOFTWARE, **FEBRUARY 2024**".

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[21] `eu2024cra`

regulation, level A · Thesis_v2 [21]

Reference. European Parliament and Council of the European Union. *Regulation (EU) 2024/2847 on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act)*. Official Journal of the European Union, OJ L, 2024/2847. Nov. 2024. URL: https://eur-lex.europa.eu/eli/reg/2024/2847/oj

Entry. source page.

Cited in. 2 places.

> **Full draft, Section 1.1.** The EU Cyber Resilience Act places secure-by-design and lifecycle vulnerability-handling obligations on manufacturers of products with digital elements, with the vulnerability-reporting obligations applying from September 2026 and most of the remainder from December 2027 **[21]**.
>
> **Thesis_v2, Section 1.1.** The EU Cyber Resilience Act places secure-by-design and lifecycle vulnerability-handling obligations on manufacturers of products with digital elements, with the vulnerability-reporting obligations applying from September 2026 and most of the remainder from December 2027 **[21]**.

Claim it carries. "with reporting obligations applying from September 2026 and the remainder from December 2027"

Source says.

> *(**Article 71(2)**, EUR-Lex consolidated text): "This Regulation shall apply from **11 December 2027**. However, **Article 14 shall apply from 11 September 2026** and **Chapter IV (Articles 35 to 51) shall apply from 11 June 2026**." Recital (126) says the same in prose.*

Metadata. [verified] OJ L, 2024/2847, published 20 November 2024. ELI URL resolves.

Verdict. Dates are right, but "the remainder from December 2027" omits the June 2026 tranche (fix #6).

Record. Sources_evidence.md, entry record.

---

[22] `agache2020firecracker`

peer reviewed, level A · Thesis_v2 [22]

Reference. A. Agache et al. "Firecracker: Lightweight Virtualization for Serverless Applications". In: *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2020, pp. 419–434. URL: https://www.usenix.org/conference/nsdi20/presentation/agache

Entry. source page.

Cited in. 3 places.

> **Full draft, Section 1.1.** Production hypervisors such as AWS Firecracker, which underpins its serverless platform, are already written in Rust **[22]**.
>
> **Full draft, Appendix B.** Compiled systems languages sit close on time and energy **[44, 91]**, Rust's safety is cheap at run time **[12, 92]**, and that holds where timing is tightest, in embedded hypervisors, production cloud infrastructure, and packet processing within a few percent of a C reference **[93, 22, 71, 94]**.
>
> **Thesis_v2, Section 1.1.** Production hypervisors such as AWS Firecracker, which underpins its serverless platform, are already written in Rust **[22]**.

Claim it carries. production hypervisors such as AWS Firecracker, which underpins its serverless platform, are already written in Rust

Source says.

> *(nsdi20-paper-agache.pdf): p. 5. "Firecracker contains **approximately 50k lines of Rust code** (96% fewer lines than QEMU)", p. 13. "The use of virtualization, along with side-channel hardening, **implementation in Rust**, extensive testing and validation…" and "We have been **running Lambda workloads in production on Firecracker since 2018**."*

Metadata. [wrong] **missing author**. The USENIX proceedings page and the paper's title page both read: "Alexandru Agache, Marc Brooker, **Andreea Florescu**, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa, Amazon Web Services." The entry has seven of those eight. Pages 419–434 confirmed by DBLP. (DBLP also omits Florescu, the proceedings page is the better authority.)

Verdict. Claim verified. **add Andreea Florescu** (fix #2).

Record. Sources_evidence.md, entry record.

---

[23] `delloro2025ran3q`

grey, level A · Thesis_v2 [23]

Reference. Dell'Oro Group. *RAN Mostly Stable in 3Q 2025.* Dell'Oro Group, RAN Quarterly Report summary. The five largest RAN suppliers by worldwide revenue (Huawei, Ericsson, Nokia, ZTE, Samsung) accounted for 96% of the 1Q25–3Q25 RAN market, up from 95% in 2024. Accessed August 2026. Nov. 2025. URL: https://www.delloro.com/news/ran-mostly-stable-in-3q-2025/

Entry. source page. The five largest RAN suppliers by worldwide revenue (Huawei, Ericsson, Nokia, ZTE, Samsung) accounted for 96% of the 1Q25–3Q25 RAN market, up from 95% in 2024. Accessed August 2026.

Cited in. 2 places.

> **Full draft, Section 1.1.** That market is concentrated in very few hands: the five largest suppliers took 96% of worldwide RAN revenue over the first three quarters of 2025, with concentration by Herfindahl–Hirschman Index at a ten-year high **[23, 24]**.

> **Thesis_v2, Section 1.1.** That market is concentrated in very few hands: the five largest suppliers took 96% of worldwide RAN revenue over the first three quarters of 2025, with concentration by Herfindahl–Hirschman Index at a ten-year high **[23, 24]**.

Source says.

> *Press release of 20 November 2025. "The top five suppliers accounted for 96 percent of the 1Q25-3Q25 RAN market, up from 95 percent in 2024", and the top five are Huawei, Ericsson, Nokia, ZTE and Samsung. Both numbers and the ordering match*

Record. Sources_evidence.md, 24 August verification pass.

---

[24] `delloro2026ran`

grey, level A · Thesis_v2 [24]

Reference. Dell'Oro Group. *RAN Market Stabilized in 2025.* Dell'Oro Group, RAN Quarterly Report summary. Full-year 2025 preliminary findings: supplier rankings unchanged, and overall market concentration measured by the Herfindahl–Hirschman Index reached a ten-year high. Accessed August 2026. Feb. 2026. URL: https://www.delloro.com/news/ran-market-stabilized-in-2025/

Entry. source page. Full-year 2025 preliminary findings: supplier rankings unchanged, and overall market concentration measured by the Herfindahl–Hirschman Index reached a ten-year high. Accessed August 2026.

Cited in. 2 places.

> **Full draft, Section 1.1.** That market is concentrated in very few hands: the five largest suppliers took 96% of worldwide RAN revenue over the first three quarters of 2025, with concentration by Herfindahl–Hirschman Index at a ten-year high **[23, 24]**.

> **Thesis_v2, Section 1.1.** That market is concentrated in very few hands: the five largest suppliers took 96% of worldwide RAN revenue over the first three quarters of 2025, with concentration by Herfindahl–Hirschman Index at a ten-year high **[23, 24]**.

Source says.

> *Press release of 17 February 2026. “Overall market concentration, as measured by the Herfindahl–Hirschman Index, reached a 10-year high in 2025”, and “Revenue rankings did not change in 2025”*

Record. Sources_evidence.md, 24 August verification pass.

---

[25] `3gpp2024road6g`

grey, level A · Thesis_v2 [25], Plan [9]

Reference. J. Almodovar and A. Sultan. *The SA1 road to 6G*. 3GPP Highlights, Issue 09. Written by the chair and the secretary of 3GPP WG SA1. Release 20 carries the 6G studies and Release 21 the normative work. ITU-R sets early 2029 for technology proposals and mid-2030 for the submitted specifications. Accessed August 2026. Dec. 2024. URL: https://www.3gpp.org/news-events/3gpp-news/sa1-6g-road

Entry. source page. Written by the chair and the secretary of 3GPP WG SA1. Release 20 carries the 6G studies and Release 21 the normative work. ITU-R sets early 2029 for technology proposals and mid-2030 for the submitted specifications. Accessed August 2026.

Cited in. 3 places.

> **Full draft, Section 1.1.** The next one is being specified now: 3GPP has allocated Release 20 to the 6G studies and Release 21 to the normative work, against an ITU calendar that asks for technology proposals in early 2029 and the submitted specifications by mid-2030 **[25]**.

> **Thesis_v2, Section 1.1.** The next one is being specified now: 3GPP has allocated Release 20 to the 6G studies and Release 21 to the normative work, against an ITU calendar that asks for technology proposals in early 2029 and the submitted specifications by mid-2030 **[25]**.

> **Plan, Section 1.** The decision has a long horizon. 3GPP has allocated Release 20 to the 6G studies and Release 21 to the normative work, with full specifications due to the ITU by mid-2030 and migration from 5G explicitly in scope **[9, 10]**, so a language chosen for the control plane of the current generation is inherited by the next.

Source says.

> *Jose Almodovar (SA1 Chair) and Alain Sultan (SA1 Secretary), “First published December 2024, in Highlights Issue 09”. Release 20 for studies and Release 21 for the normative work, against an ITU calendar of early 2029 for technology proposals and mid-2030 for the specifications. The migration and interworking claim sits in the TR 22.870 section the article names. The web page is stamped 2 January 2025, and the entry follows the Highlights issue instead, which the note states*

Record. Sources_evidence.md, 24 August verification pass.

---

[26] `cisa2025badpractices`

standard, level A · Thesis_v2 [26]

Reference. Cybersecurity and Infrastructure Security Agency. *Product Security Bad Practices*. Joint Guidance. Version 2.0. Bad practice 1 covers the development of new product lines in memory-unsafe languages and the memory-safety roadmap expectation, from which only products with an announced end-of-support date before 1 January 2030 are exempt. CISA and FBI, Jan. 2025. URL: https://www.cisa.gov/resources-tools/resources/product-security-bad-practices

Entry. source page. Version 2.0. Bad practice 1 covers the development of new product lines in memory-unsafe languages and the memory-safety roadmap expectation, from which only products with an announced end-of-support date before 1 January 2030 are exempt.

Cited in. 2 places.

> **Full draft, Section 1.1.** CISA and the FBI classify the development of new product lines in a memory-unsafe language as a bad practice where the product serves critical infrastructure, a scope mobile-network infrastructure sits squarely inside, and they exempt an existing product from the memory-safe roadmap expectation only when its announced support ends before January 2030 **[26]**.

Thesis_v2, Section 1.1. CISA and the FBI classify the development of new product lines in a memory-unsafe language as a bad practice where the product serves critical infrastructure, a scope mobile-network infrastructure sits squarely inside, and they exempt an existing product from the memory-safe roadmap expectation only when its announced support ends before January 2030 **[26]**.

Source says.

Version 2.0, January 2025, both confirmed on the revision table. Bad practice 1 covers new product lines for critical infrastructure in a memory-unsafe language. On the exemption the source says: "Publication of a memory safety roadmap does not apply to products that have an announced end-of-support date that is prior to Jan. 1, 2030." That is the claim at `Ha_Vien_Mastersthesis_2026.tex:437`, at the strength the thesis states it

Record. Sources_evidence.md, 24 August verification pass.

---

[27] `qin2020rust`

peer reviewed, level A · Thesis_v2 [27]

Reference. B. Qin et al. "Understanding Memory and Thread Safety Practices and Issues in Real-World Rust Programs". In: *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2020, pp. 763–779. DOI: 10.1145/3385412.3386036. URL: https://doi.org/10.1145/3385412.3386036

Entry. DOI 10.1145/3385412.3386036.

Cited in. 5 places.

Full draft, Section 1.2. The available evidence is largely general-purpose and average-case **[12, 27]**, leaving these three questions (predictability, prevented-defect scope, and migration cost) open for a representative subsystem.

Full draft, Section 2.4. Empirical evidence supports the model in practice: in real codebases `unsafe` is used sparingly and in localised ways **[27, 41]**, and a study of all Rust memory-safety CVEs found that every one of them but a single compiler bug involved `unsafe` code **[42]**.

Full draft, Section 7.5. Empirical studies of real-world Rust find that memory- and thread-safety bugs still occur in practice, with blocking bugs and misused synchronisation among them **[27, 90]**.

Full draft, Appendix B. The safety strand is settled the same partial way, with machine-checked foundations for the safe fragment and the aliasing discipline `unsafe` code has to respect **[11, 39, 40]**, confirmation that `unsafe` is used sparingly and that memory-safety CVEs always involve it **[27, 41, 42]**, auditing that finds defects at scale even so and unsafety spreading through dependency chains **[97, 98, 90, 43]**, and the reports motivating the shift set against a long record of memory corruption resisting runtime mitigation **[8, 9, 17, 19, 20, 33]**.

Thesis_v2, Section 1.2. The available evidence is largely general-purpose and average-case **[12, 27]**, leaving these three questions (predictability, prevented-defect scope, and migration cost) open for a representative subsystem.

Source says.

*(author copy, p. 1, abstract): "We performed the **first empirical study of Rust by close, manual inspection of 850 unsafe code usages and 170 bugs** in five open-source Rust projects, five widely-used Rust libraries, two online security databases, and the Rust standard library."*

Metadata. [verified] Crossref matches.

Verdict. OK, supports both the "used sparingly" claim and the "blocking bugs still occur" claim at 3744.

Record. Sources_evidence.md, entry record.

---

[28] 3gpp22261

standard, level C · Thesis_v2 [34]

Reference. 3GPP. *Service requirements for the 5G system; Stage 1.* Technical Specification (TS) 22.261. V18.6.0, Release 18. 3rd Generation Partnership Project (3GPP), Mar. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/22_series/22.261/22261-i60.zip

Entry. source page. V18.6.0, Release 18.

Cited in. 3 places.

> **Full draft, Section 2.1.** Such control-plane systems must satisfy demanding non-functional requirements: very high event rates (UE mobility, signalling storms), strict and predictable latency bounds that the 5G service and RAN specifications quantify as KPIs **[5, 28]**, long-lived software with continuously evolving complexity, and heavily concurrent workloads.

> **Full draft, Section 3.1.** It is core RAN layer-3 functionality, and it carries the properties that make production control-plane code difficult: per-UE mutable state, a strict operation lifecycle, high concurrency across UEs, and the latency and reliability targets the 5G system sets for control-plane procedures **[5, 28]**.

> **Thesis_v2, Chapter 3.** It has mutable state per device, a strict lifecycle, heavy concurrency, and hard latency targets **[5, 34]**.

Claim it carries. the 5G system's latency and reliability KPIs

Source says.

> *ETSI no longer mirrors v18.6.0 (only 18.13.0 and later), so the KPI clause was not opened. The performance requirements live in clause 7. If you want a page-level anchor here, the cleanest option is to lean on 3gpp38913 clause 7.4, which* is *verified below.*

Metadata. [verified] 22261-i60.zip (i = Release 18, 6.0) is dated **2022-04-01** in the 3GPP archive, consistent with mar 2022 (SA plenary approval, posted 1 April). Re-checked 24 August 2026, date unchanged. Note the convention clash flagged in that pass: this entry follows the approval month while 3gpp38913 was moved to the posting month by this same audit, and the two situations are identical.

Verdict. Metadata OK, claim evidence unverified.

Record. Sources_evidence.md, entry record.

---

[29] 3gpp38463

standard, level A

Reference. 3GPP. *NG-RAN; E1 Application Protocol (E1AP).* Technical Specification (TS) 38.463. V17.0.0, Release 17. 3rd Generation Partnership Project (3GPP), Apr. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.463/38463-h00.zip

Entry. source page. V17.0.0, Release 17.

Cited in. 1 place.

> **Full draft, Section 2.1.** These parts communicate over standardised interfaces: NG towards the 5G core, F1 between the CU and the DU, and E1 between the CU-CP and the CU-UP **[1, 2, 29]**.

Claim it carries. the E1 interface between CU-CP and CU-UP

Source says.

> *Title page of ts_138463v170000p.pdf: "NG-RAN; E1 Application Protocol (E1AP) (3GPP TS 38.463 version 17.0.0 Release 17)".*

Metadata. [verified] 38463-h00.zip dated **2022-04-06**, matching apr 2022. ETSI mirror *ETSI TS 138 463 V17.0.0 (2022-05).*

Verdict. OK.

Record. Sources_evidence.md, entry record.

[30] `li2014tales` peer reviewed, level A

Reference. J. Li et al. “Tales of the Tail: Hardware, OS, and Application-level Sources of Tail Latency”. In: *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*. 2014, 9:1–9:14. DOI: 10.1145/2670979.2670988. URL: https://doi.org/10.1145/2670979.2670988

Entry. DOI 10.1145/2670979.2670988.

Cited in. 2 places.

> **Full draft, Section 2.2.** Fine-grained measurement studies show that, even at moderate utilisation, hardware, operating-system, and application-level effects such as background interference, scheduling, interrupt routing, and power-saving states inflate the tail far beyond what queueing models predict **[30]**.

> **Full draft, Appendix B.** The fourth strand is the domain and the method: the tail-latency and microsecond-scale literature **[6, 7, 30, 104]**, thread-per-core structuring moving the tail in its own right **[45]**, O-RAN disaggregation **[3]**, control-plane latency as an active target in the core network too **[105, 106]**, adoption evidence pointing at the learning curve rather than run-time cost **[78, 79, 107]**, and standard guidance on repeat-based evaluation and fault injection **[32, 59]**.

Claim it carries. hardware, OS and application-level effects inflate the tail beyond what queueing models predict

Source says.

> *(author copy, p. 1, title and abstract): “Tales of the Tail: **Hardware, OS, and Application-level Sources of Tail Latency**”, University of Washington.*

Metadata. [verified] Crossref matches, `pages = 9:1-9:14` is the article-number convention.

Verdict. OK.

Record. Sources_evidence.md, entry record.

[31] `schroeder2006openclosed` peer reviewed, level A · Thesis_v2 [28], Plan [27]

Reference. B. Schroeder, A. Wierman, and M. Harchol-Balter. “Open Versus Closed: A Cautionary Tale”. In: *Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI ’06)*. 2006. URL: https://www.usenix.org/conference/nsdi-06/open-versus-closed-cautionary-tale

Entry. source page.

Cited in. 4 places.

> **Full draft, Section 2.2.** In a closed system a new request is triggered only by the completion of a previous one, whereas in an open system arrivals are independent of completions, and the two produce systematically different response-time behaviour on the same service process, with the closed model reporting the lower figures **[31]**.

> **Full draft, Section 6.2.** The send schedule is fixed and the histogram is corrected, so a stall introduces no coordinated-omission bias **[31, 63]**.

> **Thesis_v2, Chapter 2.** They must also be recorded without coordinated omission, that is, without letting a load generator that has fallen behind hide its own slowest requests **[28, 29]**.

> **Plan, Section 4.** Latency is captured in high-dynamic-range histograms against an intended send schedule, so coordinated omission does not hide the tail **[27, 28]**.

Claim it carries. open and closed generators produce systematically different response times, the closed model reporting lower figures

Source says.

> *(schroeder.pdf, p. 1, abstract): "Workload generators may be classified as based on a **closed system model, where new job arrivals are only triggered by job completions** ... or an **open system model, where new jobs arrive independently of job completions** ... we illustrate that there is a **vast difference in behavior between open and closed models** in real-world settings."*

Metadata. [verified] DBLP confirms NSDI 2006 and assigns **no page numbers**, the entry correctly omits them. USENIX page live.

Verdict. OK. This is the right primary citation for the coordinated-omission argument.

Record. Sources_evidence.md, entry record.

---

[32] `georges2007rigorous`

peer reviewed, level A · Thesis_v2 [38], Plan [24]

Reference. A. Georges, D. Buytaert, and L. Eeckhout. "Statistically Rigorous Java Performance Evaluation". In: *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*. 2007, pp. 57–76. DOI: 10.1145/1297027.1297033. URL: https://doi.org/10.1145/1297027.1297033

Entry. DOI 10.1145/1297027.1297033.

Cited in. 6 places.

> **Full draft, Section 2.2.** Sound measurement therefore requires an open-loop model with an intended schedule, correction for coordinated omission, high-dynamic-range latency histograms that retain resolution across many orders of magnitude, and statistically rigorous reporting with repeated runs and confidence intervals rather than single point estimates [**32**].

> **Full draft, Section 4.4.** This follows established guidance on statistically rigorous performance evaluation, which warns that reporting a single number invites conclusions that run-to-run variation does not support [**32**].

> **Full draft, Appendix B.** The fourth strand is the domain and the method: the tail-latency and microsecond-scale literature [**6, 7, 30, 104**], thread-per-core structuring moving the tail in its own right [**45**], O-RAN disaggregation [**3**], control-plane latency as an active target in the core network too [**105, 106**], adoption evidence pointing at the learning curve rather than run-time cost [**78, 79, 107**], and standard guidance on repeat-based evaluation and fault injection [**32, 59**].

> **Thesis_v2, Chapter 4.** Reporting follows the guidance that a single figure invites conclusions run-to-run variation does not support [**38**].

> **Thesis_v2, Chapter 6.** Every point is a median with a bootstrap 95% confidence interval [**38, 39**].

> **Plan, Section 4.** The workload is swept from nominal load to overload on pinned cores, with interleaved repeats and interval estimates rather than single figures [**24**]: medians with nonparametric bootstrap confidence intervals [**25**], and exact paired permutation tests corrected across the family by the Holm procedure [**26**].

Claim it carries. reporting a single number invites conclusions run-to-run variation does not support

Source says.

> *(Ghent University copy, p. 1, abstract): "This paper shows that **prevalent methodologies can be misleading, and can even lead to incorrect conclusions**. The reason is that the data analysis is not statistically rigorous."*

Metadata. [verified] Crossref matches, booktitle wording differs cosmetically.

Verdict. OK, near-verbatim support for the sentence at 1572.

Record. Sources_evidence.md, entry record.

---

[33] `szekeres2013sok`

peer reviewed, level A · Thesis_v2 [30]

Reference. L. Szekeres et al. “SoK: Eternal War in Memory”. In: *Proceedings of the 2013 IEEE Symposium on Security and Privacy (S&P)*. 2013, pp. 48–62. DOI: 10.1109/SP.2013.13. URL: https://doi.org/10.1109/SP.2013.13

Entry. DOI 10.1109/SP.2013.13.

Cited in. 4 places.

> **Full draft, Section 2.3.** Memory-corruption errors of this kind have been a persistent security problem for decades, and deployed mitigations remain only partial defences **[33]**.

> **Full draft, Section 3.2.** These are the same memory-corruption classes that decades of security research have struggled to contain in C and C++ **[33]**.

> **Full draft, Appendix B.** The safety strand is settled the same partial way, with machine-checked foundations for the safe fragment and the aliasing discipline `unsafe` code has to respect **[11, 39, 40]**, confirmation that `unsafe` is used sparingly and that memory-safety CVEs always involve it **[27, 41, 42]**, auditing that finds defects at scale even so and unsafety spreading through dependency chains **[97, 98, 90, 43]**, and the reports motivating the shift set against a long record of memory corruption resisting runtime mitigation **[8, 9, 17, 19, 20, 33]**.

> **Thesis_v2, Chapter 2.** Safety remains the programmer’s responsibility **[30]**.

Claim it carries. memory corruption as a decades-old problem whose deployed mitigations are partial

Source says.

> *(p. 1, abstract): “This problem has existed for **more than 30 years** and a vast number of potential solutions have been proposed, yet memory corruption attacks continue to pose a serious threat. Real world exploits show that **all currently deployed protections can be defeated**.”*

Metadata. [verified] IEEE S&P 2013, pp. 48–62. IEEE’s record parses “Tao Wei” as a surname, the entry is correct.

Verdict. OK, this is a precise match for “deployed mitigations remain only partial defences”.

Record. Sources_evidence.md, entry record.

---

[34] `bensalem2005deadlock`

peer reviewed, level A

Reference. S. Bensalem and K. Havelund. “Dynamic Deadlock Analysis of Multi-threaded Programs”. In: *Hardware and Software, Verification and Testing (Haifa Verification Conference)*. Vol. 3875. Lecture Notes in Computer Science. Springer, 2005, pp. 208–223. DOI: 10.1007/11678779_15. URL: https://doi.org/10.1007/11678779_15

Entry. DOI 10.1007/11678779_15.

Cited in. 1 place.

> **Full draft, Section 2.3.** Its deadlock detector inherits that limitation from the technique it implements, which infers deadlock potentials from the lock-acquisition graph of a single trace **[34]**, and it ships as an experimental component covering only the POSIX mutex families **[35]**.

Source says.

> *“Dynamic Deadlock Analysis of Multi-threaded Programs”. Saddek Bensalem, Klaus Havelund. Haifa Verification Conference 2005, LNCS 3875, pages 208–223. DOI 10.1007/11678779_15.* Note the date: *the conference is 2005, Springer’s deposit records the LNCS volume as 2006. Cite the conference year and the volume, as DBLP does.*

> ***This is the peer-reviewed foundation for the mechanism `Ha_Vien_Mastersthesis_2026.tex:3009–3012` explains in its own voice.*** *The draft argues that TSan builds its lock-order graph from acquisitions that succeed, so a run in which the first inversion blocks yields one edge and never the second, and the detector stays silent at the moment the bug manifests. Bensalem and Havelund set out the technique that reasoning is about: lock-graph analysis over a single execution trace, in which a cycle signifies a deadlock* potential*, and whose stated purpose is to "identify deadlock potentials even though no deadlocks occur in the examined execution trace".*
>
> *Read together, the draft's finding stops being an oddity and becomes the expected behaviour of the technique. The analysis is designed for traces that* survive*, so the trace that deadlocks is the one it has least to say about. That is a much stronger way to write the same paragraph, and it generalises the way the draft already claims it does.*
>
> *The paper is also about reducing false positives specifically, via thread labels, gate locks and segmentation, which is the complementary error mode and worth one clause for balance.*

Record. LITERATURE_NOTES.md, section B.5 Bensalem and Havelund.

---

[35] `tsandeadlock` tool record, level A

Reference. The Sanitizers Project. *ThreadSanitizer deadlock detector*. Project documentation. Accessed August 2026. 2026. URL: https://github.com/google/sanitizers/wiki/ThreadSanitizerDeadlockDetector

Entry. source page. Project documentation. Accessed August 2026.

Cited in. 1 place.

> **Full draft, Section 2.3.** Its deadlock detector inherits that limitation from the technique it implements, which infers deadlock potentials from the lock-acquisition graph of a single trace **[34]**, and it ships as an experimental component covering only the POSIX mutex families **[35]**.

Source says.

> *Neither is peer-reviewed. Both are primary sources about the software, which is what is wanted here, and both must be labelled as such if cited.*

- ***ThreadSanitizerDeadlockDetector****, google/sanitizers wiki.* *https://github.com/google/sanitizers/wiki/ThreadSanitizerDeadlockDetector*. *States the mechanism in one sentence ("The deadlock detector maintains a directed graph of lock acquisitions. If a lock B is acquired while a lock A is being held by the same thread, a directed edge `A=>B` is added to the lock acquisition graph. A potential deadlock is reported when there is a cycle in the graph"), and describes the detector as* ***experimental****, supporting only `pthread_mutex_*` / `pthread_rwlock_*` / `pthread_spin_*`, and "tested only on tiny test cases". It also confirms that detection needs `TSAN_OPTIONS=detect_deadlocks=1`, which is worth checking against what `faults/` actually sets.*
- ***google/sanitizers issue #1258.*** *Upstream record that there are two detector implementations and that* ***neither is actively maintained****, plus a contributor's diagnosis that version 1, the default, checks for the cycle in `DD::MutexBeforeLock` while the edge is only added in `DD::MutexAfterLock`. That is the draft's mechanism, described by the tool's own issue tracker, and it independently corroborates a paragraph the draft currently supports with its own measurements alone.*

Record. LITERATURE_NOTES.md, section B.6 The two tool records, as tool records.

---

[36] `sutter2025profiles` committee draft, level A

Reference. H. Sutter. *Core Safety Profiles for C++26*. ISO/IEC JTC1/SC22/WG21 document P3081R2. Feb. 2025. URL: https://isocpp.org/files/papers/P3081R2.pdf

Entry. source page.

Cited in. 1 place.

> **Full draft, Section 2.3.** Since 2023 the standards committee has pursued *profiles*: named sets of rules covering type, bounds, lifetime, and arithmetic safety that a conforming implementation must enforce wherever they are enabled [**36**].

Claim it carries. profiles a conforming implementation must enforce wherever enabled

Source says.

> *(p. 4, §2.1 Strategy): "**Define standard enforced 'profiles' that a conforming C++ implementation must enforce when enabled, notably bounds, type, and lifetime.** … Each rule must be **deterministically decidable at compile time (even if it results in injecting a check enforced at run time)** … Rules are portable and enforced in the C++ implementation, not in a separate tool such as a static analyzer." The four profiles appear as §§4–7 (type, bounds, lifetime, arithmetic).*

Metadata. [verified] Document header: "Core safety profiles for C++26, Document Number: **P3081 R2**, Date: **2025-02-04**, Reply-to: **Herb Sutter**, Audience: EWG, LEWG."

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[37] `baxter2024safecpp` committee draft, level A

Reference. S. Baxter and C. Mazakas. *Safe C++*. ISO/IEC JTC1/SC22/WG21 document P3390R0. Sept. 2024. URL: https://safecpp.org/P3390R0.html

Entry. source page.

Cited in. 1 place.

> **Full draft, Section 2.3.** That direction was chosen over *Safe C++*, a proposal for a safe subset with borrow checking and its own standard library, which is Rust's model expressed in C++ [**37**].

Claim it carries. a proposal for a rigorously safe subset with borrow checking and an accompanying standard library

Source says.

> *(document body): the proposal's own compiler output shows "**borrow checking**: iterator.cxx:10:11" under "safety: during safety checking of int main() safe", and the comparison section states "**C++ lacks traits, relocation and borrow checking**".*

Metadata. [verified] Published at safecpp.org as WG21 P3390R0, authors Sean Baxter and Christian Mazakas. September 2024.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[38] `vandevoorde2026profiles` committee draft, level A

Reference. D. Vandevoorde et al. *Profiles and Safety: A Call to Action*. ISO/IEC JTC1/SC22/WG21 document P3970R0. Jan. 2026. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2026/p3970r0.pdf

Entry. source page.

Cited in. 3 places.

> **Full draft, Section 2.3.** Profiles are not yet standardised, and in January 2026 their own authors were still calling for implementations and treating the design of the initial profiles as work still to be done [**38**].

> **Full draft, Section 2.3.** The concurrency profile, the one that would bear on the defect classes catalogued in Chapter *chapter:context*, is placed after that initial set and not within it: "more profiles should be defined and implemented, notably a concurrency profile" [**38**].

> **Full draft, Section 7.6.** That qualifies nothing measured here, because none of that work is standardised or generally implemented and the concurrency profile that would reach the boundary classes of RQ2 comes after an initial set still to be specified **[38]**.

Claim it carries. the thesis quotes this document twice

Source says.

> *:*
>
> - *p. 1: "Currently, **there is – to the best of our knowledge – a specification and an experimental implementation is being conducted in a major C++ compiler**. Also, an implementation guide is being written." Then: "Beyond the framework implementation(s), **we need to design and precisely specify a few initial profiles**." → supports the quotation at 779.*
> - *p. 2: "**After that, more profiles should be defined and implemented, notably a concurrency profile.**" → supports the quotation at 782 word for word.*

Metadata. [verified] Document header: "Doc. No. **P3970R0**, Date: **2026-01-16**", authors David Vandevoorde, Jeff Garland, Paul E. McKenney, Roger Orr, Bjarne Stroustrup, Michael Wong, exactly the entry's list and order.

Verdict. OK. Both quoted fragments are verbatim.

Record. Sources_evidence.md, entry record.

---

[39] `jung2021safe`

peer reviewed, level A · Thesis_v2 [31]

Reference. R. Jung et al. "Safe Systems Programming in Rust". In: *Communications of the ACM* 64.4 (2021), pp. 144–152. DOI: 10.1145/3418295. URL: https://doi.org/10.1145/3418295

Entry. DOI 10.1145/3418295.

Cited in. 5 places.

> **Full draft, Section 2.4.** The type system statically forbids mutation of shared state and confines the unavoidable exceptions to `unsafe` code kept behind safe APIs **[39]**.

> **Full draft, Section 2.4.** Rust is the first widely supported language to combine compile-time memory and thread safety with the absence of a garbage collector and throughput and memory use alongside C and C++ **[12, 39, 44]**.

> **Full draft, Appendix B.** The safety strand is settled the same partial way, with machine-checked foundations for the safe fragment and the aliasing discipline `unsafe` code has to respect **[11, 39, 40]**, confirmation that `unsafe` is used sparingly and that memory-safety CVEs always involve it **[27, 41, 42]**, auditing that finds defects at scale even so and unsafety spreading through dependency chains **[97, 98, 90, 43]**, and the reports motivating the shift set against a long record of memory corruption resisting runtime mitigation **[8, 9, 17, 19, 20, 33]**.

> **Thesis_v2, Chapter 2.** It checks memory safety and freedom from data races at compile time, using ownership, borrowing, and thread-safety traits such as `Send` **[10, 31]**.

> **Thesis_v2, Chapter 3.** The guarantee rests on aliasing control and data-race freedom **[31]**.

Claim it carries. ownership and borrowing statically forbid mutation of shared state, exceptions confined to `unsafe` behind safe APIs

Source says.

> *(author copy, p. 1): "It tackles this challenge using a strong type system based on the ideas of **ownership and borrowing, which statically prohibits the mutation of shared state**", and the following bullet on data types that "fundamentally depend on shared mutable state" being encapsulated.*

Metadata. [verified] Crossref matches on every field.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[40] `jung2020stacked` peer reviewed, level A

Reference. R. Jung et al. “Stacked Borrows: An Aliasing Model for Rust”. In: *Proceedings of the ACM on Programming Languages* 4.POPL (2020), 41:1–41:32. DOI: 10.1145/3371109. URL: https://doi.org/10.1145/3371109

Entry. DOI 10.1145/3371109.

Cited in. 2 places.

> **Full draft, Section 2.4.** The soundness of the safe fragment of the language, including the role of `unsafe` as an explicitly marked escape hatch, has been established formally by the RustBelt project [**11**], and the aliasing discipline that even `unsafe` code must respect has been given a precise operational semantics [**40**].

> **Full draft, Appendix B.** The safety strand is settled the same partial way, with machine-checked foundations for the safe fragment and the aliasing discipline `unsafe` code has to respect [**11, 39, 40**], confirmation that `unsafe` is used sparingly and that memory-safety CVEs always involve it [**27, 41, 42**], auditing that finds defects at scale even so and unsafety spreading through dependency chains [**97, 98, 90, 43**], and the reports motivating the shift set against a long record of memory corruption resisting runtime mitigation [**8, 9, 17, 19, 20, 33**].

Claim it carries. the aliasing discipline `unsafe` code must respect, given a precise operational semantics

Source says.

> *(p. 1, abstract): “we propose **Stacked Borrows, an operational semantics for memory accesses in Rust**. Stacked Borrows defines an aliasing discipline and declares programs violating it to have undefined behavior.”*

Metadata. [verified] Footer: “Proc. ACM Program. Lang., Vol. 4, No. POPL, **Article 41. Publication date: January 2020**”. `pages = 41:1-41:32` correct.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[41] `astrauskas2020unsafe` peer reviewed, level B

Reference. V. Astrauskas et al. “How Do Programmers Use Unsafe Rust?” In: *Proceedings of the ACM on Programming Languages* 4.OOPSLA (2020), 136:1–136:27. DOI: 10.1145/3428204. URL: https://doi.org/10.1145/3428204

Entry. DOI 10.1145/3428204.

Cited in. 2 places.

> **Full draft, Section 2.4.** Empirical evidence supports the model in practice: in real codebases `unsafe` is used sparingly and in localised ways [**27, 41**], and a study of all Rust memory-safety CVEs found that every one of them but a single compiler bug involved `unsafe` code [**42**].

> **Full draft, Appendix B.** The safety strand is settled the same partial way, with machine-checked foundations for the safe fragment and the aliasing discipline `unsafe` code has to respect [**11, 39, 40**], confirmation that `unsafe` is used sparingly and that memory-safety CVEs always involve it [**27, 41, 42**], auditing that finds defects at scale even so and unsafety spreading through dependency chains [**97, 98, 90, 43**], and the reports motivating the shift set against a long record of memory corruption resisting runtime mitigation [**8, 9, 17, 19, 20, 33**].

Claim it carries. `unsafe` is used sparingly and in localised ways

Source says.

> *Crossref-deposited abstract: "an important assumption of the Rust language, which we dub the **Rust hypothesis**, is that programmers use Rust by following three main principles: **use unsafe code sparingly, make it easy to review, and hide it behind a safe abstraction**... This paper studies empirically how unsafe code is used in practice."*

Metadata. [verified] Crossref matches, `pages = 136:1-136:27` is the ACM article-number convention.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[42] `xu2021rustcve` peer reviewed, level A · Plan [21]

Reference. H. Xu et al. "Memory-Safety Challenge Considered Solved? An In-Depth Study with All Rust CVEs". In: *ACM Transactions on Software Engineering and Methodology* 31.1 (2022), 3:1–3:25. DOI: 10.1145/3466642. URL: https://doi.org/10.1145/3466642

Entry. DOI 10.1145/3466642.

Cited in. 4 places.

> **Full draft, Section 2.4.** Empirical evidence supports the model in practice: in real codebases `unsafe` is used sparingly and in localised ways **[27, 41]**, and a study of all Rust memory-safety CVEs found that every one of them but a single compiler bug involved `unsafe` code **[42]**.

> **Full draft, Section 7.2.** That fits the CVE evidence that Rust memory-safety bugs always involve `unsafe` **[42]**, and the caution that unsafety can still spread through dependencies **[43]**, which is why keeping the core `unsafe`-free matters.

> **Full draft, Appendix B.** The safety strand is settled the same partial way, with machine-checked foundations for the safe fragment and the aliasing discipline `unsafe` code has to respect **[11, 39, 40]**, confirmation that `unsafe` is used sparingly and that memory-safety CVEs always involve it **[27, 41, 42]**, auditing that finds defects at scale even so and unsafety spreading through dependency chains **[97, 98, 90, 43]**, and the reports motivating the shift set against a long record of memory corruption resisting runtime mitigation **[8, 9, 17, 19, 20, 33]**.

> **Plan, Section 3.** Memory-safety defects in real Rust code invariably trace back to `unsafe` **[21]**, which one boundary class now reproduces directly, while deadlock remains an open problem for the language **[22]**.

Source says.

> *(arXiv 2003.03296, the authors' copy of the same study): p. 1, abstract. "Our analysis result shows that Rust can keep its promise that **all memory-safety bugs require unsafe code**", and p. 2. "**all bugs in our dataset require unsafe code except one compiler bug.**"*

Metadata. [verified] TOSEM 31(1), article 3, issue dated January 2022 → `year = 2022` is right (Crossref's 2021 is online-first). Pages `3:1-3:25` correct.

Verdict. Entry OK. The thesis sentence at 805 ("every one of them involved `unsafe` code") should carry the exception (fix #5).

Record. Sources_evidence.md, entry record.

---

[43] `evans2020rust` peer reviewed, level A · Thesis_v2 [52]

Reference. A. N. Evans, B. Campbell, and M. L. Soffa. "Is Rust Used Safely by Software Developers?" In: *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*. 2020, pp. 246–257. DOI: 10.1145/3377811.3380413. URL: https://doi.org/10.1145/3377811.3380413

Entry. DOI 10.1145/3377811.3380413.

Cited in. 6 places.

Full draft, Section 2.4. Fewer than a third of libraries use `unsafe` directly, but a much larger fraction cannot be fully statically checked because it hides somewhere in their call chain [**43**], which is a further argument for keeping the safety-critical core of a subsystem `unsafe`-free, as this thesis does.

Full draft, Section 7.2. That fits the CVE evidence that Rust memory-safety bugs always involve `unsafe` [**42**], and the caution that unsafety can still spread through dependencies [**43**], which is why keeping the core `unsafe`-free matters.

Full draft, Section 7.5. They also find that a substantial fraction of the ecosystem cannot be fully statically checked, because `unsafe` propagates through dependency chains [**43**].

Full draft, Appendix B. The safety strand is settled the same partial way, with machine-checked foundations for the safe fragment and the aliasing discipline `unsafe` code has to respect [**11, 39, 40**], confirmation that `unsafe` is used sparingly and that memory-safety CVEs always involve it [**27, 41, 42**], auditing that finds defects at scale even so and unsafety spreading through dependency chains [**97, 98, 90, 43**], and the reports motivating the shift set against a long record of memory corruption resisting runtime mitigation [**8, 9, 17, 19, 20, 33**].

Full draft, Appendix G. It keeps the dependency-propagation problem Evans et al. describe [**43**] away from the safety-critical code, and it makes the reviewable surface of the migration a fixed, countable thing.

Thesis_v2, Chapter 7. That keeps unsafety out of the dependency chains it otherwise propagates through [**52**], and makes the reviewable surface a countable thing.

Claim it carries. a small fraction of crates use `unsafe` directly, but a much larger fraction cannot be fully statically checked

Source says.

> *(arXiv 2007.00752, p. 1, abstract): "software engineers use the keyword* ***unsafe in less than 30% of Rust libraries, but more than half cannot be entirely statically checked*** *by the Rust compiler because of Unsafe Rust* ***hidden somewhere in a library's call chain.****"*

Metadata. [verified] Pages and year match. Crossref's booktitle is "Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering", the entry's shorter form is fine.

Verdict. OK, this is close to a word-for-word match with the thesis sentence at 810.

Record. Sources_evidence.md, entry record.

---

[44] `bugden2022rust` peer reviewed, level A

Reference. W. Bugden and A. Alahmar. "Rust: The Programming Language for Safety and Performance". In: *arXiv preprint arXiv:2206.05503* (2022). DOI: 10.48550/arXiv.2206.05503. URL: https://doi.org/10.48550/arXiv.2206.05503

Entry. DOI 10.48550/arXiv.2206.05503.

Cited in. 2 places.

Full draft, Section 2.4. Rust is the first widely supported language to combine compile-time memory and thread safety with the absence of a garbage collector and throughput and memory use alongside C and C++ [**12, 39, 44**].

Full draft, Appendix B. Compiled systems languages sit close on time and energy [**44, 91**], Rust's safety is cheap at run time [**12, 92**], and that holds where timing is tightest, in embedded hypervisors, production cloud infrastructure, and packet processing within a few percent of a C reference [**93, 22, 71, 94**].

Claim it carries. cross-language benchmark surveys place Rust alongside C and C++

Source says.

> *(arXiv PDF, p. 1, abstract): "we **surveyed and analyzed recent research on Rust and research that benchmarks Rust with other available prominent programming languages** ... recent experimental results in benchmarking research show Rust's overall superiority..."*

Metadata. [verified] arXiv 2206.05503, DOI 10.48550/arXiv.2206.05503 (DataCite), 2022.

Verdict. OK, but note this is still the only load-bearing arXiv-only citation, `ASSESSMENT.md` already flags replacing it as the last optional lift on criterion 2. Its use at 3782 is honest (the table marks it "General benchmarks / no / no / no").

Record. Sources_evidence.md, entry record.

---

[45] `enberg2019threadpercore`

peer reviewed, level A · Thesis_v2 [35], Plan [23]

Reference. P. Enberg, A. Rao, and S. Tarkoma. "The Impact of Thread-per-Core Architecture on Application Tail Latency". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–8. DOI: 10.1109/ANCS.2019.8901874. URL: https://doi.org/10.1109/ANCS.2019.8901874

Entry. DOI 10.1109/ANCS.2019.8901874.

Cited in. 5 places.

> **Full draft, Section 2.5.** The choice also moves the quantity measured here: thread-per-core structuring changes application tail latency on its own **[45]**, so the model is held fixed across both implementations instead of left to each language's idiom.
>
> **Full draft, Appendix B.** The fourth strand is the domain and the method: the tail-latency and microsecond-scale literature **[6, 7, 30, 104]**, thread-per-core structuring moving the tail in its own right **[45]**, O-RAN disaggregation **[3]**, control-plane latency as an active target in the core network too **[105, 106]**, adoption evidence pointing at the learning curve rather than run-time cost **[78, 79, 107]**, and standard guidance on repeat-based evaluation and fault injection **[32, 59]**.
>
> **Thesis_v2, Chapter 4.** That structuring itself changes tail latency **[35]**.
>
> **Thesis_v2, Chapter 5.** That structuring is known to change tail latency by itself **[35]**.
>
> **Plan, Section 4.** Holding the concurrency model fixed matters because thread-per-core structuring itself changes tail latency **[23]**.

Claim it carries. the justification for holding the concurrency model fixed across both implementations

Source says.

> *(author copy, p. 1, abstract): "The **thread-per-core architecture has emerged to reduce these overheads, but it also has its challenges** from thread synchronization and OS interfaces. Applications can mitigate both issues with different techniques, but **their impact on application tail latency is an open question.**"*

Metadata. [verified] Crossref matches.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[46] `skoudlil2025asyncrust`

peer reviewed, level A · Plan [16]

Reference. M. Škoudlil, M. Sojka, and Z. Hanzálek. "A First Look at ROS 2 Applications Written in Asynchronous Rust". In: *37th Euromicro Conference on Real-Time Systems (ECRTS 2025)*. Vol. 335. Leibniz International Proceedings in Informatics (LIPIcs). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, 1:1–1:21. DOI: 10.4230/LIPIcs.ECRTS.2025.1. URL: https://doi.org/10.4230/LIPIcs.ECRTS.2025.1

Entry. DOI 10.4230/LIPIcs.ECRTS.2025.1.

Cited in. 3 places.

Full draft, Section 2.5. Its timing behaviour belongs to the runtime, not to the language hosting it: a real-time study of asynchronous Rust finds one application meeting or missing its deadlines depending on the runtime and thread-priority mapping it is given, and the configuration that meets them is not the idiomatic work-stealing one [**46**].

Full draft, Section 7.5. vSkoudlil et al. report that Tokio is "designed to maximize throughput rather than time determinism", and that restructuring onto prioritised threads yields response times that match classical analysis [**46**].

Plan, Section 2. Independent real-time analysis of those runtimes supports that reading [**16**].

Claim it carries. the configuration that meets deadlines is not the idiomatic work-stealing one, tail behaviour is a property of the runtime

Source says.

:

- *p. 5, §2.2.2: "**Tokio** is a popular multi-threaded runtime. It seems to be **designed to maximize throughput rather than time determinism**. Their scheduling policies are complex and difficult to understand due to the use of abstraction layers."*
- *p. 15: "**futures-rt and rclcpp-rt are the only variants where all callbacks manage to complete before the deadlines** given by the publication periods. All other implemented variants fail to meet shorter deadlines for topics 1–3. This is expected for the default configurations of Rust async runtimes."*
- *p. 1, abstract: "the experimental results indicate that our proposed structure **achieves bounded response times for time-critical tasks**."*

Metadata. [verified] The LIPIcs PDF's running heads confirm **1:1–1:21**, volume 335, ECRTS 2025, and the three authors as entered.

Verdict. OK, supports exactly the rebuttal the thesis builds on it.

Record. Sources_evidence.md, entry record.

---

[47] `rivera2021galeed`

peer reviewed, level B · Thesis_v2 [50], Plan [30]

Reference. E. Rivera et al. "Keeping Safe Rust Safe with Galeed". In: *Proceedings of the 37th Annual Computer Security Applications Conference (ACSAC)*. 2021, pp. 824–836. DOI: 10.1145/3485832.3485903. URL: https://doi.org/10.1145/3485832.3485903

Entry. DOI 10.1145/3485832.3485903.

Cited in. 9 places.

Full draft, Section 2.6. This has been shown experimentally, and the proposed remedies isolate the two heaps in hardware or in the runtime instead of trusting the discipline of the interface [**47, 48**].

Full draft, Section 7.2. That is why the remedies proposed in the literature isolate the two heaps with hardware support instead of relying on interface discipline [**47, 48**], and the defect classes that arise at such boundaries have been catalogued in their own right [**49**].

Full draft, Section 7.6. RQ4 measures the seam and not the safety of the composed system: host and component share an address space [**47, 48**].

Full draft, Chapter 8. It can also compare Rust async runtimes against the thread-per-shard model, which would separate the tail divergence Arvidsson and Kaller attribute to `async/await` [**55**], measure in-process isolation of the two heaps at the seam [**47, 48**], and track defect density and review effort as Rust is adopted in a production RAN codebase.

Full draft, Appendix B. The interoperability strand qualifies the adoption argument, and Section *sec:interop* states its central limit: confining `unsafe` to a seam does not protect the Rust side, because foreign code in the same address space can corrupt Rust-owned data anyway [**47, 48**].

Full draft, Appendix G. While the host remains C++ the two share a heap, so a team needing the stronger property has to look at in-process isolation [**47, 48**] and not at interface discipline alone.

Thesis_v2, Chapter 7. Host and component share one address space [**50, 51**].

Thesis_v2, Chapter 8. Measuring what in-process isolation of the two heaps costs would price what protecting an `unsafe`-free core is worth [**50**].

Plan, Section 4. On the FFI, the plan takes a narrow position: confining `unsafe` to the seam is a statement about what the compiler checks, not run-time protection, since host and component share an address space [**30**].

Claim it carries. proposed remedies isolate the two heaps rather than relying on interface discipline

Source says.

> *(deposited abstract): "these guarantees are violated when safe Rust interacts with unsafe code, most notably code written in other programming languages … **a dangling pointer in a linked C/C++ library can access and overwrite memory allocated to Rust even when the Rust code is fully safe.**"*

Verdict. OK, and it is the sharpest statement in the bibliography of the limit §7.6 draws around RQ4: confining `unsafe` bounds what must be reviewed, not what can be corrupted.

Record. Sources_evidence.md, entry record.

---

[48] `kirth2022pkrusafe`

peer reviewed, level B · Thesis_v2 [51]

Reference. P. Kirth et al. "PKRU-Safe: Automatically Locking Down the Heap Between Safe and Unsafe Languages". In: *Proceedings of the 17th European Conference on Computer Systems (EuroSys)*. 2022, pp. 132–148. DOI: 10.1145/3492321.3519582. URL: https://doi.org/10.1145/3492321.3519582

Entry. DOI 10.1145/3492321.3519582.

Cited in. 7 places.

Full draft, Section 2.6. This has been shown experimentally, and the proposed remedies isolate the two heaps in hardware or in the runtime instead of trusting the discipline of the interface [**47, 48**].

Full draft, Section 7.2. That is why the remedies proposed in the literature isolate the two heaps with hardware support instead of relying on interface discipline [**47, 48**], and the defect classes that arise at such boundaries have been catalogued in their own right [**49**].

Full draft, Section 7.6. RQ4 measures the seam and not the safety of the composed system: host and component share an address space [**47, 48**].

Full draft, Chapter 8. It can also compare Rust async runtimes against the thread-per-shard model, which would separate the tail divergence Arvidsson and Kaller attribute to `async`/`await` [**55**], measure in-process isolation of the two heaps at the seam [**47, 48**], and track defect density and review effort as Rust is adopted in a production RAN codebase.

Full draft, Appendix B. The interoperability strand qualifies the adoption argument, and Section *sec:interop* states its central limit: confining `unsafe` to a seam does not protect the Rust side, because foreign code in the same address space can corrupt Rust-owned data anyway [**47, 48**].

Full draft, Appendix G. While the host remains C++ the two share a heap, so a team needing the stronger property has to look at in-process isolation [**47, 48**] and not at interface discipline alone.

Thesis_v2, Chapter 7. Host and component share one address space [**50, 51**].

Claim it carries. proposed remedies isolate the two heaps rather than relying on interface discipline

Source says.

> *(deposited abstract): "many memory-safe languages still have to **interface with unsafe code** to some extent, which opens up the possibility for attackers to exploit memory-corruption vulnerabilities in the unsafe part of the system and **subvert the safety guarantees provided by the memory-safe language**." The mechanism is in the title: "Automatically Locking Down the Heap Between Safe and Unsafe Languages".*

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[49] `li2022crosslang`

peer reviewed, level C · Thesis_v2 [37]

Reference. Z. Li et al. "Detecting Cross-Language Memory Management Issues in Rust". In: *Computer Security – ESORICS 2022*. 2022, pp. 680–700. DOI: 10.1007/978-3-031-17143-7_33. URL: https://doi.org/10.1007/978-3-031-17143-7_33

Entry. DOI 10.1007/978-3-031-17143-7_33.

Cited in. 4 places.

> **Full draft, Section 2.6.** The defect classes specific to Rust/C boundaries have been catalogued separately from those of either language **[49]**, and the binding generators themselves carry bugs **[50]**.
>
> **Full draft, Section 7.2.** That is why the remedies proposed in the literature isolate the two heaps with hardware support instead of relying on interface discipline **[47, 48]**, and the defect classes that arise at such boundaries have been catalogued in their own right **[49]**.
>
> **Full draft, Appendix B.** The seam carries its own defect classes and its own tooling bugs, lighter crossing models exist for embedded use, aliasing patterns expressible in C but not in safe Rust bound translation in principle, interoperation is the central open problem in embedded Rust, `unsafe` concentrates at exactly these boundaries, and the two largest documented adoptions into existing C and C++ report the resulting friction empirically **[49, 50, 99, 100, 101, 102, 80, 81]**.
>
> **Thesis_v2, Chapter 4.** Cross-language memory-management defects concentrate at that seam **[37]**.

Claim it carries. defect classes arising specifically at Rust/C boundaries have been catalogued in their own right

Source says.

> *Springer paywall, title-level, and the thesis uses it only for the existence of the category.*

Metadata. [verified] DBLP and Crossref both give pages **680–700**.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[50] `cai2026interop`

peer reviewed, level B

Reference. X. Cai et al. "Fortifying the Seams Between C/C++ and Rust: Characterizing Bugs in Interop Tools". In: *ACM Transactions on Software Engineering and Methodology* (2026). DOI: 10.1145/3795532. URL: https://doi.org/10.1145/3795532

Entry. DOI 10.1145/3795532.

Cited in. 3 places.

> **Full draft, Section 2.6.** The defect classes specific to Rust/C boundaries have been catalogued separately from those of either language **[49]**, and the binding generators themselves carry bugs **[50]**.
>
> **Full draft, Appendix B.** The seam carries its own defect classes and its own tooling bugs, lighter crossing models exist for embedded use, aliasing patterns expressible in C but not in safe Rust bound translation in principle, interoperation is the central open problem in embedded Rust, `unsafe` concentrates at exactly these boundaries, and the two largest documented adoptions into existing C and C++ report the resulting friction empirically **[49, 50, 99, 100, 101, 102, 80, 81]**.

> **Full draft, Appendix G.** And the generated part of a seam is not above suspicion, because binding generators have their own defect history [**50**].

Claim it carries. binding generators carry their own defect history

Source says.

> *Crossref-deposited abstract: "we **mined 320 bugs in the popular interop tools, i.e., Bindgen, Cbindgen, and CXX**. We observe that the main obstacle is not tool crashes or performance issues, but the **inability to generate correct and functional code** ... Memory layout mismatch is the most common cause with Bindgen."*

Metadata. [verified] Crossref matches authors, journal, year, DOI. Volume/issue not yet assigned, so the entry correctly omits them.

Verdict. OK, supports "review it as code rather than trusting it as tooling output" at 3558.

Record. Sources_evidence.md, entry record.

---

[51] `pearce2026rustmc` peer reviewed, level A

Reference. O. Pearce, J. Lange, and D. O'Keeffe. "RustMC: Automated Verification of Real-World Concurrent Rust". In: *Formal Techniques for Distributed Objects, Components, and Systems (FORTE)*. Lecture Notes in Computer Science. Springer, 2026, pp. 235–255. DOI: 10.1007/978-3-032-28187-6_14. URL: https://doi.org/10.1007/978-3-032-28187-6_14

Entry. DOI 10.1007/978-3-032-28187-6_14.

Cited in. 2 places.

> **Full draft, Section 2.6.** A stateless model checker for Rust now explores the state space of a program together with its foreign dependencies, which puts data races arising from the interaction of the two languages within reach of a checker even though they sit outside what the borrow checker sees [**51**].

> **Full draft, Chapter 8.** Beyond that, the framework can take captured anonymised traces in place of the synthetic workload, further control-plane subsystems or the data-heavy L2/PHY path, and a whole-program model checker covering foreign dependencies [**51**].

Source says.

> *"RustMC: Automated Verification of Real-World Concurrent Rust". Oliver Pearce, Julien Lange, Dan O'Keeffe (Royal Holloway, University of London).* Formal Techniques for Distributed Objects, Components, and Systems *(FORTE 2026), Urbino, 8–12 June 2026, LNCS, pages 235–255. DOI 10.1007/978-3-032-28187-6_14. (A February 2025 preprint exists as arXiv 2502.06293 under the earlier title "RustMC: Extending the GenMC stateless model checker to Rust". Cite the FORTE version.)*
>
> ***This is the best find of the revised sweep and it lands on two of the thesis's claims at once.** A stateless model checker for concurrent Rust built on GenMC, and, from the abstract, it verifies "Rust code and any foreign function dependencies", finding "concurrency bugs in unsafe Rust code, FFI calls to C/C++, and atomic operations". The authors state it is "the only available framework capable of exhaustively exploring the state space of Rust programs and their dependencies".*

Record. LITERATURE_NOTES.md, section The anchor: Pearce, Lange and O'Keeffe, FORTE 2026.

---

[52] `karlback2021ran` thesis, level A · Thesis_v2 [32], Plan [11]

Reference. R. Karlbäck and A. Orö. "Holistic View on Alternative Programming Languages for Radio Access Network Applications in Cloud and Embedded Deployments". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–2021/023–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2021. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-176307

Entry. source page. Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–2021/023–SE.

Cited in. 10 places.

Full draft, Section 2.7. One weighs runtime cost against development efficiency across C++, Rust, and Go [**52**].

Full draft, Section 2.7. Three of them vary the language together with the runtime and the concurrency model, so a measured difference cannot be attributed to the language rather than to the framework built on it [**52, 54, 55**].

Full draft, Section 7.2. They call the pattern surprising and attribute it tentatively to the initial stack size of the green threads [**52**].

Full draft, Section 7.5. They "aimed for providing the best solution in each language, instead of trying to make the code as identical as possible between the languages" [**52**].

Full draft, Section 7.5, table cell. Orö, 2021 [**52**]

Full draft, Section 7.5. None is unoccupied on its own, as the matched implementations [**71, 82, 83, 84, 85, 89, 56**], the percentile sweep stopping at a latency requirement [**55**] and the effort measurements [**52, 78**] all show.

Full draft, Section 7.6. Peak resident set size sits between the two groups, with the sign found here reported by two neighbouring theses and reversed by a third [**53, 77, 52**].

Thesis_v2, Chapter 2. No earlier study isolates the language this way for a stateful, concurrent subsystem [**32, 33**].

Thesis_v2, Chapter 7. One close study reports the opposite sign, with Rust lowest and the gap widening as concurrency rises [**32**].

Plan, Section 2. Karlbäck and Orö compare Rust and Go against C++ for RAN applications and conclude C++ remains the choice where performance is critical [**11**].

Source says.

, claim by claim (page numbers are PDF pages of `FULLTEXT01.pdf`*):*

- "conclude that C++ remains the choice where performance is critical" *(`Ha_Vien_Mastersthesis_2026.tex:3692`), supported, p. 79: "It is quite clear that* ***C++ is the superior language if low CPU usage is important****", alongside the mixed processing-time results on the same page.*
- Memory: "find Rust the lowest of the three languages and C++ the highest, with the gap widening as concurrency rises" *(`Ha_Vien_Mastersthesis_2026.tex:3437`), supported, p. 80: "****We see that C++ has the highest memory usage, especially when looking at larger amount of sequences, and Rust has the least memory usage.****"*
- "a pattern they flag as unexplained" *(`Ha_Vien_Mastersthesis_2026.tex:3438`).* ***overstated.*** *Same passage, p. 80: "****What is surprising with these measurements is how little memory Rust uses. The main theory for this is that the initial stack size for the Rust green threads could be the smallest****, or that they adapt dynamically fast[er]. . . ". They call it surprising and offer a theory, and their theory is the same green-thread mechanism the thesis then argues for, so saying so strengthens the thesis (fix #7).*
- ***The quotation at `Ha_Vien_Mastersthesis_2026.tex:3695-3697`, not found in the source.*** *The thesis prints: "an effort . . . to utilize the different strengths of each language, and stay away from using exactly the same implementation". The source, §6.2.5 Implementation, p. 84 of the PDF (****printed page 74****), says: "This way we hopefully utilized the languages built-in support for the different parts of the code and gave each language an as representative solution as possible and* ***stayed away from using the exact same solution in each language****. We aimed for providing the best solution in each language, instead of trying to make the code as identical as possible between the languages." And later on the same page: ". . . a real world scenario where a programmer most likely would, and should,* ***utilize the strengths of each language*** *when implementing the solution." The phrase "an effort was made" does occur on that page, but in a different sentence about a different subject: "****Although an effort was made to minimize this****, multiple implementation details could probably have been done better", where "this" is their own inexperience skewing results in C++'s favour.* ***Suggested replacement*** *(accurate,*

> *and makes the same point): > The authors record that they deliberately did not write the same program three times: they > "aimed for providing the best solution in each language, instead of trying to make the code as > identical as possible between the languages", and "stayed away from using the exact same > solution in each language" \cite{karlback2021ran}.*

Metadata. [verified] mostly. The DiVA record (`urn:nbn:se:liu:diva-176307` → `diva2:1563049`) is the right thesis: cover page reads "Linköping University, Department of Computer and Information Science, Master's thesis, 30 ECTS, Datateknik, **2021**, **LIU-IDA/LITH-EX-A–2021/023–SE**, *Holistic view on alternative programming languages for Radio Access Network applications in cloud and embedded deployments*". One nuance: the cover lists **Anton Orö before Rasmus Karlbäck**, the entry and the thesis text use "Karlbäck and Orö". DiVA's own citation order is alphabetical, so this is defensible, but be aware the title page reads the other way.

Verdict. FIX, the substance is right, the quotation marks are not (fix #1).

Record. Sources_evidence.md, entry record.

---

[53] `olsson2023ranmgmt`

thesis, level A · Thesis_v2 [46], Plan [12]

Reference. V. Olsson. "Rust Programming Language as an Alternative to C for RAN Management Applications". Carried out at Ericsson AB. Master's thesis. KTH Royal Institute of Technology, School of Electrical Engineering and Computer Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-325822

Entry. source page. Carried out at Ericsson AB.

Cited in. 8 places.

> **Full draft, Section 2.7.** One asks whether Rust could replace C in RAN node management [**53**].

> **Full draft, Section 2.7.** The remaining two hold the design closer to fixed but report execution time and peak memory rather than tail behaviour under rising load [**53, 56**], and the closest of them, Emberg and Wadsten, measures a stateless transformation rather than a stateful, concurrent subsystem.

> **Full draft, Section 7.2.** A second Ericsson-hosted study of RAN node management software found Rust's peak memory usage generally higher than C's [**53**], and a matched pair of user-space device drivers outside telecom put the Rust version's footprint about 29% above the C version's [**77**].

> **Full draft, Section 7.5.** Olsson's later management-software comparison agrees with this study on two of three findings, including that resource result, and does not report percentiles or drive the software past capacity [**53**].

> **Full draft, Section 7.5, table cell.** Olsson, 2023 [**53**]

> **Full draft, Section 7.6.** Peak resident set size sits between the two groups, with the sign found here reported by two neighbouring theses and reversed by a third [**53, 77, 52**].

> **Thesis_v2, Chapter 7.** Two further studies agree with the direction found here [**46, 47**].

> **Plan, Section 2.** Olsson reaches the same domain from the management side and reports Rust's execution speed on par with or better than C's, but its peak memory higher [**12**].

Claim it carries. Rust's peak memory usage generally higher than C's, and execution speed on par with or better than C's

Source says.

> *(FULLTEXT01.pdf, p. 4, abstract): "The results show that the execution speed of Rust applications are on par or better than their C counterpart, while the peak memory usage generally is higher in Rust. It was also shown that significant improvements in safety can be achieved, even in cases where the application is fairly straight forward."*

Metadata. [verified] DiVA `diva2:1751095`, full text `FULLTEXT01.pdf` retrieved.

Verdict. OK. Both halves of the claim are the source's own summary sentence, and this is the neighbouring study whose peak-memory sign agrees with this thesis.

Record. Sources_evidence.md, entry record.

---

[54] `olsson2023asyncpackets`

thesis, level A · Plan [13]

Reference. J. Olsson and I. Stenström. "Rust Async for Thread-per-Core Packet Processing". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–23/052–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-197029

Entry. source page. Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–23/052–SE.

Cited in. 5 places.

> **Full draft, Section 2.7.** One builds a thread-per-core packet processor on asynchronous Rust and measures it against the DPDK application it mirrors [**54**].

> **Full draft, Section 2.7.** Three of them vary the language together with the runtime and the concurrency model, so a measured difference cannot be attributed to the language rather than to the framework built on it [**52, 54, 55**].

> **Full draft, Section 7.5.** Olsson and Stenström's asynchronous Rust at a fifth to a half of DPDK's throughput is the sharpest illustration of why the model has to be held fixed: an effect of two to five times is not the scale at which code generation operates [**54**].

> **Full draft, Section 7.5, table cell.** Stenström, 2023 [**54**]

> **Plan, Section 2.** Olsson and Stenström build a thread-per-core packet processor on asynchronous Rust and measure between 20% and 50% of the throughput of the DPDK application it mirrors [**13**].

Claim it carries. asynchronous Rust at a fifth to a half of the throughput of the DPDK application it mirrors, and judged not currently viable

Source says.

> *(`FULLTEXT02.pdf`, p. 3, abstract): "The results show that the Rust application with the decentralized scheduler has a throughput that is between 20% and 50% of its DPDK counterpart, while its latency is far worse." And: "the result of this thesis indicates that using Rust async for packet processing systems is currently not a viable approach."*

Metadata. [verified] DiVA `diva2:1789208`, full text `FULLTEXT02.pdf` retrieved.

Verdict. OK, with the scope note above: the figure is the decentralized arm.

Record. Sources_evidence.md, entry record.

---

[55] `arvidsson2025async`

thesis, level A · Thesis_v2 [44], Plan [14]

Reference. A. Arvidsson and M. Kaller. "Evaluating Asynchronous Programming Frameworks for 5G Control Plane Procedures". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–25/055–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2025. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-214746

Entry. source page. Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–25/055–SE.

Cited in. 12 places.

> **Full draft, Section 2.7.** One measures median and 99th-percentile latency for control-plane procedures across five asynchronous programming frameworks [**55**].

Full draft, Section 2.7. Three of them vary the language together with the runtime and the concurrency model, so a measured difference cannot be attributed to the language rather than to the framework built on it [**52, 54, 55**].

Full draft, Section 6.4.2. The closest prior work reports a comparable degradation past a threshold request rate and attributes it to the asynchronous runtime [**55**].

Full draft, Section 7.2. They trace it to stackless `async/await`, which permits a context switch immediately before an I/O operation begins [**55**].

Full draft, Section 7.5. Across five asynchronous frameworks on 5G control-plane procedures, Boost.Fibers shows the lowest latency at every load, and while the Rust runtimes track it at the median, their 99th percentile rises more steeply beyond a certain request rate [**55**].

Full draft, Section 7.5, table cell. Kaller, 2025 [**55**]

Full draft, Section 7.5. None is unoccupied on its own, as the matched implementations [**71, 82, 83, 84, 85, 89, 56**], the percentile sweep stopping at a latency requirement [**55**] and the effort measurements [**52, 78**] all show.

Full draft, Section 7.6. No study located by the protocol of Appendix *appendix:search* reports a load-dependent reversal between the two languages, and the one reporting the same shape attributes it to a runtime this design excludes [**55**].

Full draft, Chapter 8. It can also compare Rust async runtimes against the thread-per-shard model, which would separate the tail divergence Arvidsson and Kaller attribute to `async/await` [**55**], measure in-process isolation of the two heaps at the seam [**47, 48**], and track defect density and review effort as Rust is adopted in a production RAN codebase.

Thesis_v2, Chapter 7. It finds the same shape and traces it to stackless `async/await` [**44**].

Thesis_v2, Chapter 8. Evaluating Rust async runtimes against the thread-per-shard model would separate the runtime from the language in the tail result [**44**].

Plan, Section 2. The Rust runtimes track the shape of its median curve, but their p99 rises more steeply past a threshold rate [**14**].

Source says.

, claim by claim (PDF pages):

- Frameworks compared and Boost.Fibers fastest*, p. 3, abstract: "**C++ with Boost.Fibers, Rust with both Tokio and Monoio, Go with Goroutines, and Erlang** … The research reveals that **C++ with Boost.Fibers achieves the highest performance, exhibiting the lowest latency and highest throughput, while Rust frameworks present a competitive alternative.**"*
- "Boost.Fibers shows the lowest latency at every load" *(`Ha_Vien_Masterthesis_2026.tex:3709`), p. 50: "**it is evident that Boost.Fibers consistently exhibits the lowest latency**" across the load sweep, same page: "as the load, sequence frequency, increases, latency also rises".*
- "the Rust runtimes track it at the median, their 99th percentile rises more steeply beyond a certain request rate"*, supported by §9.1.2 (p. 77) plus the load-sweep figures: "C++ with Boost.Fibers displays a **minimal difference between** [median and p99]… In contrast, **both Rust frameworks exhibit a much greater disparity between their median and 99th percentile latencies**", with the effect "especially under high-load conditions" (p. 78). The* shape *claim (rate threshold) is read off their figures rather than a sentence, accurate, but if challenged, point to the figures.*
- "the authors attribute the effect to stackless `async/await` permitting a context switch immediately before an I/O operation is initiated" *(`Ha_Vien_Masterthesis_2026.tex:3713-3715`). **exact**, §9.1.2, pp. 77–78: "**Rust utilizes stackless async/await** for user-level threading, whereas C++ with Boost.Fibers employs stackful fibers … **both Rust frameworks may perform a context switch at any await point … Scenario 2 can occur in Rust, where a context switch happens just before an I/O operation is initiated** … This behavior leads to increased latency, **particularly noticeable in the 99th percentile latency measurements.**"*

Metadata. [verified] The DiVA record (`urn:nbn:se:liu:diva-214746` → `diva2:1970356`) matches on every field: "Master's thesis, 30 ECTS, Computer Science and Engineering, **2025**, **LIU-IDA/LITH-EX-A–25/055–SE**, *Evaluating Asynchronous Programming Frameworks for 5G Control Plane Procedures*, **Albin Arvidsson, Martin Kaller**", external supervisor at Ericsson.

Verdict. OK, the strongest rival result in the thesis is represented fairly and attributed in the source's own terms.

Record. Sources_evidence.md, entry record.

---

[56] `emberg2023asn1`

thesis, level A · Thesis_v2 [33], Plan [15]

Reference. C. Emberg and A. Wadsten. "Attainable Safety and Long-Term Resource-Efficiency Using Rust: Evaluating the Viability of Rust in Terms of Development Effort and Performance for the Decompilation of ASN.1 Messages". Carried out at Ericsson, Linköping. Report LIU-IDA/LITH-EX-A–23/078–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-195552

Entry. source page. Carried out at Ericsson, Linköping. Report LIU-IDA/LITH-EX-A–23/078–SE.

Cited in. 8 places.

> **Full draft, Section 2.7.** The last builds matched C++ and Rust implementations of an ASN.1 decompiler and compares them statistically [**56**].

> **Full draft, Section 2.7.** The remaining two hold the design closer to fixed but report execution time and peak memory rather than tail behaviour under rising load [**53, 56**], and the closest of them, Emberg and Wadsten, measures a stateless transformation rather than a stateful, concurrent subsystem.

> **Full draft, Section 7.5.** Emberg and Wadsten are the closest of the five in method: a matched pair, an FFI seam, a statistical comparison and an effort measurement, two years earlier [**56**].

> **Full draft, Section 7.5, table cell.** Wadsten, 2023 [**56**]

> **Full draft, Section 7.5.** None is unoccupied on its own, as the matched implementations [**71, 82, 83, 84, 85, 89, 56**], the percentile sweep stopping at a latency requirement [**55**] and the effort measurements [**52, 78**] all show.

> **Full draft, Section 7.6.** That the threat is real and not merely conceivable is documented in the same industry, by the two benchmarking sessions of Emberg and Wadsten that reversed their own verdict [**56**].

> **Thesis_v2, Chapter 2.** No earlier study isolates the language this way for a stateful, concurrent subsystem [**32, 33**].

> **Plan, Section 2.** Emberg and Wadsten come closest in method, building matched C++ and Rust ASN.1 decompilers, reaching third-party C++ from Rust through a C wrapper, and comparing the two statistically [**15**].

Claim it carries. matched C++ and Rust ASN.1 decompilers reaching third-party C++ through a C wrapper, compared statistically, whose two benchmarking sessions reversed their own verdict

Source says.

> *(`FULLTEXT01.pdf`, p. 3, abstract): "Two ASN.1 decompiler implementations were made, one in C++ and one in Rust. Both decompilers invoked third-party C++ functions: in Rust, this was done by first wrapping them in C. The performance of both decompilers was measured during two benchmarking sessions, between which additional optimization was performed on both. The results were then compared statistically using the Mann-Whitney rank sum test method. The first benchmarking session showed similar performance for both decompilers … In the second benchmarking session, the Mann-Whitney test showed that the Rust decompiler performed worse, with a mean difference of 85.19-91.36%." The C-wrapper route is drawn in Figure 5.2, p. 42.*

Metadata. [verified] DiVA `diva2:1772502`, full text `FULLTEXT01.pdf` retrieved.

Verdict. OK. Every clause the thesis attributes to this source is in its own abstract, and the 85 to 91 per cent range recorded in `LITERATURE_NOTES.md` is exact at 85.19 to 91.36.

Record. Sources_evidence.md, entry record.

---

[57] `gjomemo2016mesc`

peer reviewed, level B · Plan [20]

Reference. R. Gjomemo et al. "Leveraging Static Analysis Tools for Improving Usability of Memory Error Sanitization Compilers". In: *Proceedings of the 2016 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. Two of the six authors are affiliated with Nokia. 2016, pp. 323–334. DOI: 10.1109/QRS.2016.44. URL: https://doi.org/10.1109/QRS.2016.44

Entry. DOI 10.1109/QRS.2016.44. Two of the six authors are affiliated with Nokia.

Cited in. 2 places.

> **Full draft, Section 2.7.** The published record on the other side is emptier: there is no study of Rust among the indexed publications of the vendor hosting this work, and the nearest peer-reviewed position from that side argues for making memory-error sanitisation practical rather than for replacing it with a compile-time guarantee [**57**], which is the strategy this thesis tests against.

> **Plan, Section 2.** The nearest peer-reviewed position from that side, co-authored by two Nokia researchers, argues for making memory-error sanitisation compilers practical on the premise that certifying compilation is not [**20**], which is the after-the-fact detection strategy this thesis sets a compile-time guarantee against.

Claim it carries. argues for making memory-error sanitisation compilers practical on the premise that certifying compilation is not

Source says.

> *(publisher-deposited abstract via OpenAlex, no open copy exists): "There has been considerable interest in having a compiler to ensure the safety of compiled code either through static verification or through instrumented runtime checks.* ***While certifying compilation has shown much promise, it has not been practical, leaving code instrumentation as the next best strategy*** *for compilation. We term such compilers Memory Error Sanitization Compilers (MESCs) … The benchmarks demonstrate an improvement in runtime performance that* ***makes incorporation of runtime checks a viable option for defenses.****"*

Metadata. [verified] Crossref: six authors as in the entry, pages 323-334, August 2016. **Two Nokia affiliations confirmed** through OpenAlex institution data: Edmund W. Ballou and Kedar S. Namjoshi are both Nokia (United States). The other four are University of Illinois Chicago and University of Dayton. That settles the plan's "co-authored by two Nokia researchers".

Verdict. OK, and an exact match on both halves. The premise clause and the practicality clause are both the source's own sentences. Level B only because the full text is paywalled and the abstract already settles it.

Record. Sources_evidence.md, entry record.

---

[58] `michaelides2026ranlatency`

peer reviewed, level A

Reference. S. Michaelides et al. "Evaluation of Security-Induced Latency on 5G RAN Interfaces and User Plane Communication". In: *Proceedings of the 19th ACM Conference on Security and Privacy in Wireless and Mobile Networks (WiSec '26)*. 2026, pp. 176–181. DOI: 10.1145/3765613.3797453. URL: https://doi.org/10.1145/3765613.3797453

Entry. DOI 10.1145/3765613.3797453.

Cited in. 1 place.

> **Full draft, Section 3.3.** A measurement study of a disaggregated 5G RAN times the CU-CP-to-DU UE Context Setup procedure over F1-C, and the CU-CP-to-CU-UP bearer context setup over E1, under different interface-protection schemes, and finds the choice worth hundreds of microseconds: on F1-C, IPsec adds on the order of 120 μs on average to UE context setup, against roughly 430 μs for DTLS [**58**].

Claim it carries. “IPsec adds roughly 120 μs on average to UE context setup, against roughly 430 μs for DTLS”

Source says.

> *(arXiv 2602.12059, the authors’ copy): p. 4. “Over the F1-C, we monitor the **UE Context Setup procedure** … To ensure statistical confidence, we **repeat the experiment 1,000 times**”, p. 5. “On the CP, specifically the F1-C interface, **all IPsec configurations add on average ∼120 μs to UE context setup, compared to ∼430 μs for DTLS**. The fastest IPsec configuration (AES256-HMAC) adds only ∼40 μs … Over the E1 interface, IPsec configurations introduce on average ∼80 μs.”*

Metadata. [verified] Crossref matches authors, venue, pages, year and DOI.

Verdict. OK, both numbers and the procedure are exactly as the thesis states.

Record. Sources_evidence.md, entry record.

---

[59] `natella2016fault`

peer reviewed, level B · Thesis_v2 [36], Plan [29]

Reference. R. Natella, D. Cotroneo, and H. S. Madeira. “Assessing Dependability with Software Fault Injection: A Survey”. In: *ACM Computing Surveys* 48.3 (2016), 44:1–44:55. DOI: 10.1145/2841425. URL: https://doi.org/10.1145/2841425

Entry. DOI 10.1145/2841425.

Cited in. 4 places.

> **Full draft, Section 4.2.2.** The method is therefore adversarial, following the established practice of software fault injection, where faults are introduced deliberately to see how a system behaves in their presence [**59**].

> **Full draft, Appendix B.** The fourth strand is the domain and the method: the tail-latency and microsecond-scale literature [**6, 7, 30, 104**], thread-per-core structuring moving the tail in its own right [**45**], O-RAN disaggregation [**3**], control-plane latency as an active target in the core network too [**105, 106**], adoption evidence pointing at the learning curve rather than run-time cost [**78, 79, 107**], and standard guidance on repeat-based evaluation and fault injection [**32, 59**].

> **Thesis_v2, Chapter 4.** RQ2 is answered by controlled fault injection [**36**].

> **Plan, Section 4.** Evaluation adds a controlled fault-injection study [**29**] and an FFI prototype.

Source says.

> *Crossref-deposited abstract: “**Software Fault Injection is a method to anticipate worst-case scenarios caused by faulty software through the deliberate injection of software faults.** This survey provides a comprehensive overview of the state of the art… ”*

Metadata. [verified] Crossref matches, `pages = 44:1-44:55` is the article-number convention.

Verdict. OK, matches the thesis’s definition at 1426 closely.

Record. Sources_evidence.md, entry record.

---

[60] `efron1993bootstrap`

book, level B · Thesis_v2 [39], Plan [25]

Reference. B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993. ISBN: 978-0-412-04231-7

Cited in. 4 places.

> **Full draft, Section 4.4.** Results are summarised by the median with a nonparametric bootstrap 95% confidence interval [**60**] rather than by the mean with a standard deviation, because latency distributions are heavy-tailed and skewed and a distribution-free interval on the median makes no normality assumption.

> **Thesis_v2, Chapter 4.** Medians come with nonparametric bootstrap confidence intervals **[39]**.
>
> **Thesis_v2, Chapter 6.** Every point is a median with a bootstrap 95% confidence interval **[38, 39]**.
>
> **Plan, Section 4.** The workload is swept from nominal load to overload on pinned cores, with interleaved repeats and interval estimates rather than single figures **[24]**: medians with nonparametric bootstrap confidence intervals **[25]**, and exact paired permutation tests corrected across the family by the Holm procedure **[26]**.

Claim it carries. nonparametric bootstrap 95 % confidence intervals

Source says.

> *Standard reference, the percentile and BCa interval construction the thesis uses is Chapters 12–14. Book not opened for this audit.*

Metadata. [nit] **superseded by the 24 August 2026 pass.** Crossref holds the record (Efron and Tibshirani, 1993, DOI 10.1007/978-1-4899-4541-9) and the ISBN matches, which is why this entry read OK on 15 August. The DOI itself **does not resolve**: `doi.org` returns 404 and so does the Springer book page. A field comparison against Crossref cannot see that, only a fetch can. → `doi` and `url` removed, ISBN kept, withdrawn handle recorded in a BibTeX comment above the entry so that it does not print.

Verdict. Claim fine, link dead, fixed.

Record. Sources_evidence.md, entry record.

---

[61] `rfc3393` standard, level A

Reference. C. Demichelis and P. Chimento. *IP Packet Delay Variation Metric for IP Performance Metrics (IPPM)*. Request for Comments (RFC) 3393. Standards Track. Defines packet delay variation as the difference between consecutive one-way delays. Internet Engineering Task Force (IETF), Nov. 2002. DOI: 10.17487/RFC3393. URL: https://www.rfc-editor.org/rfc/rfc3393

Entry. DOI 10.17487/RFC3393. Standards Track. Defines packet delay variation as the difference between consecutive one-way delays.

Cited in. 2 places.

> **Full draft, Section 4.4.** That is the packet-delay-variation quantity of RFC 3393 **[61]**, and it matches the sense in which this thesis uses the term: variation from one operation to the next, not spread of the distribution as a whole.
>
> **Full draft, Section 6.4.1.** They do not describe how far consecutive operations differ from each other, which RFC 3393 names inter-operation delay variation (Section *sec:stats*) **[61]**.

Claim it carries. jitter defined as the absolute difference between consecutive latencies, “the packet-delay-variation quantity of RFC 3393”

Source says.

> *(`rfc3393.txt`, abstract and §1): “The metric is based on the difference in the One-Way-Delay of selected packets. This difference in delay is called ’IP Packet Delay Variation (ipdv)’.”*

Metadata. [verified] Crossref: Demichelis and Chimento, IETF, November 2002, DOI 10.17487/RFC3393. (Crossref does not carry the RFC number in a comparable field, the entry’s `number = 3393` is correct.)

Verdict. OK. The thesis’s usage, variation from one operation to the next, not spread of the distribution, is the RFC’s sense.

Record. Sources_evidence.md, entry record.

---

[62] `holm1979simple` peer reviewed, level B · Thesis_v2 [40], Plan [26]

Reference. S. Holm. "A Simple Sequentially Rejective Multiple Test Procedure". In: *Scandinavian Journal of Statistics* 6 (1979), pp. 65–70. URL: https://www.jstor.org/stable/4615733

Entry. source page.

Cited in. 3 places.

> **Full draft, Section 4.4.** Because the same comparison is made for several metrics and both allocators, the family of tests is corrected with the Holm step-down procedure **[62]**, which controls the family-wise error rate without assuming independence between the tests.
>
> **Thesis_v2, Chapter 4.** Exact paired permutation tests are corrected for multiple comparisons by the Holm procedure **[40]**.
>
> **Plan, Section 4.** The workload is swept from nominal load to overload on pinned cores, with interleaved repeats and interval estimates rather than single figures **[24]**: medians with nonparametric bootstrap confidence intervals **[25]**, and exact paired permutation tests corrected across the family by the Holm procedure **[26]**.

Claim it carries. the Holm step-down procedure controlling family-wise error without assuming independence

Source says.

> *Metadata verified, the paper itself is behind JSTOR. The property the thesis attributes to it (FWER control without independence assumptions) is the paper's own result.*

Metadata. [verified] OpenAlex: *Scandinavian Journal of Statistics*, volume 6, pages **65–70**, 1979, Sture Holm. The JSTOR stable ID in the entry's URL (4615733) matches. **Do not add a DOI here.** OpenAlex advertises `10.2307/4615733`, but that handle is unregistered: `doi.org` returns 404 and the handle API reports "not found". It was added during this audit and then reverted. The entry should stay DOI-less, with the JSTOR URL as the locator.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[63] `tene_hdrhistogram` tool record, level A · Thesis_v2 [29], Plan [28]

Reference. G. Tene. *HdrHistogram: A High Dynamic Range Histogram.* Project documentation and reference implementation. Section "Corrected vs. raw value recording calls" documents the `recordValueWithExpectedInterval` correction applied when a response time exceeds the expected interval between requests. Accessed August 2026. URL: https://github.com/HdrHistogram/HdrHistogram

Entry. source page. Section "Corrected vs. raw value recording calls" documents the `recordValueWithExpectedInterval` correction applied when a response time exceeds the expected interval between requests. Accessed August 2026.

Cited in. 6 places.

> **Full draft, Section 5.3.** Latency passes through the same high-dynamic-range (HDR) histogram algorithm **[63]** on both sides, configured the same way (range [1 ns, 3.6 s], three significant figures).
>
> **Full draft, Section 6.2.** The send schedule is fixed and the histogram is corrected, so a stall introduces no coordinated-omission bias **[31, 63]**.
>
> **Thesis_v2, Chapter 2.** They must also be recorded without coordinated omission, that is, without letting a load generator that has fallen behind hide its own slowest requests **[28, 29]**.
>
> **Thesis_v2, Chapter 4.** Latency is recorded in identically configured HDR histograms **[29]**.
>
> **Thesis_v2, Chapter 5.** Both record latency through one HDR histogram algorithm configured identically **[29]**.

> **Plan, Section 4.** Latency is captured in high-dynamic-range histograms against an intended send schedule, so coordinated omission does not hide the tail **[27, 28]**.

Source says.

> *(`hdrhistogram.github.io`): "A Histogram that supports **recording and analyzing sampled data value counts across a configurable integer value range with configurable value precision** within the range. Value precision is expressed as the number of significant digits…", this fully supports the use at `Ha_Vien_Mastersthesis_2026.tex:1810` (identical HDR configuration, three significant figures).*

Metadata. [verified] Gil Tene is credited on the project page (Authors, Contributors and License section). Accessed August 2026 as stated.

Issue. The page does **not** discuss coordinated omission, which is what the citation is being asked to carry at 2402, 3922, `Thesis_v2.tex:175` and `Thesis_Plan_v2.tex:141`. In all four places `schroeder2006openclosed` is cited alongside and does carry it (fix #8).

Verdict. OK for the histogram, re-point or drop for coordinated omission.

Record. Sources_evidence.md, entry record.

---

[64] `lizard`

tool record, level A · Thesis_v2 [41]

Reference. T. Yin. *Lizard: An extensible cyclomatic complexity analyzer for many programming languages*. Version 1.23.0. Accessed August 2026. 2026. URL: https://github.com/terryyin/lizard

Entry. source page. Version 1.23.0. Accessed August 2026.

Cited in. 3 places.

> **Full draft, Section 5.4.4.** Table *tab:complexity* therefore adds cyclomatic complexity, measured by `lizard` **[64]** over both cores.

> **Full draft, Section 5.4.4, caption.** Size and cyclomatic-complexity proxies for the two cores, measured with `lizard` **[64]** applied identically to both languages.

> **Thesis_v2, Chapter 5.** The net result is a Rust core 18% longer in code lines whose total cyclomatic complexity is 15% lower **[41]**.

Claim it carries. cyclomatic complexity measured with one tool applied identically to both languages

Source says.

> *Registry record as above.*

Metadata. [verified] PyPI: author **Terry Yin**, project home `lizard.ws`, GitHub `terryyin/lizard`, version **1.23.0** exists and is the current release (uploaded 2 June 2026), consistent with "Version 1.23.0. Accessed August 2026" and `year = 2026`.

Verdict. OK. Pinning the version in the entry is exactly right for reproducibility.

Record. Sources_evidence.md, entry record.

---

[65] `zhou2025oltp`

peer reviewed, level A · Thesis_v2 [42]

Reference. X. Zhou et al. "OLTP Through the Looking Glass 16 Years Later: Communication Is the New Bottleneck". In: *15th Annual Conference on Innovative Data Systems Research (CIDR 2025)*. 2025. URL: https://www.vldb.org/cidrdb/papers/2025/p17-zhou.pdf

Entry. source page.

Cited in. 3 places.

> **Full draft, Section 6.4.1.** On a shared-nothing database partitioned one per worker thread like this service, Zhou et al. report that once the per-operation work is optimised what dominates the rest is communication across isolation boundaries and not the operation [**65**].
>
> **Full draft, Section 7.6.** Three findings have published company: median equivalence below saturation, with matched pairs in three settings putting safe Rust within single-digit percentages of its C or C++ counterpart [**71, 82, 85**], a cross-thread handoff dominating the end-to-end cost [**65**], and most of RQ2's boundary classes, which the literature already documents.
>
> **Thesis_v2, Chapter 6.** The rest is a cross-thread handoff charged to the operating system [**42**].

Claim it carries. once per-operation work is optimised, cost is dominated by communication across isolation boundaries

Source says.

> *(p. 1, abstract): "we revisit these assumptions and demonstrate that **communication overhead is now the dominant factor affecting OLTP performance** ... revealing that **messaging costs play a critical role in end-to-end performance** ... **increased isolation can lead to higher communication costs**."*

Metadata. [verified] DBLP confirms CIDR 2025 and assigns no pages, the entry correctly omits them. PDF retrieved from vldb.org.

Verdict. OK, this is the strongest external corroboration in the bibliography for the thesis's handoff-dominance result, and it is used exactly once, in the right place.

Record. Sources_evidence.md, entry record.

---

[66] `clangtidyuseaftermove` tool record, level A

Reference. The LLVM Project. *clang-tidy check `bugprone-use-after-move`*. Extra Clang Tools documentation. Accessed August 2026. 2026. URL: https://clang.llvm.org/extra/clang-tidy/checks/bugprone/use-after-move.html

Entry. source page. Extra Clang Tools documentation. Accessed August 2026.

Cited in. 1 place.

> **Full draft, Section 6.4.5.** A static analyser configured for it would plausibly flag the pattern, and `clang-tidy` ships such a check, whose documentation records that it follows control flow but not paths and considers only local variables and function parameters [**66**].

Source says.

> *https://clang.llvm.org/extra/clang-tidy/checks/bugprone/use-after-move.html, read 24 August 2026.*
>
> *`use_after_move` is the one in-guarantee class no C++ sanitiser reports, and `AGENT.md` requires the clang-tidy caveat wherever that is quoted. The caveat can now be cited rather than asserted, and in the tool's own words: the check "currently only considers calls of `std::move` on local variables or function parameters. It does not check moves of member variables or global variables", it is flow-sensitive but not path-sensitive, and it flags a use-after-move even on types that define no move operations, deliberately. Cite as project documentation exactly as `tsandeadlock` is cited, and keep the sentence's claim to what the tool documents about itself.*

Record. LITERATURE_NOTES.md, section C.9 clang-tidy 'bugprone-use-after-move' documentation.

---

[67] `hughes2025gcrust` peer reviewed, level B

Reference. J. Hughes and L. Tratt. "Garbage Collection for Rust: The Finalizer Frontier". In: *Proceedings of the ACM on Programming Languages* 9.OOPSLA2 (2025), pp. 3588–3614. DOI: 10.1145/3763179. URL: https://doi.org/10.1145/3763179

Entry. DOI 10.1145/3763179.

Cited in. 1 place.

> **Full draft, Section 6.4.5.** The safe-Rust cycle leaks in silence, because leaking is safe in Rust by explicit design [**67, 68**].

Source says.

> *Hughes & Tratt, PACMPL 9(OOPSLA2):3588–3614, 2025. "Garbage Collection for Rust: The Finalizer Frontier". Opens with the fact the RQ2 boundary needs: Rust's lack of GC makes shared-ownership data structures "awkward, inefficient, or both". Peer-reviewed support for the reference-cycle leak class, one of the two boundary classes that came back with the C++ tooling ahead.*

Record. LITERATURE_NOTES.md, peer-reviewed additions of the third sweep.

---

[68] `cui2024rcanary` peer reviewed, level A

Reference. M. Cui et al. "rCanary: Detecting Memory Leaks Across Semi-Automated Memory Management Boundary in Rust". In: *IEEE Transactions on Software Engineering* 50.9 (2024), pp. 2472–2484. DOI: 10.1109/TSE.2024.3443624. URL: https://doi.org/10.1109/TSE.2024.3443624

Entry. DOI 10.1109/TSE.2024.3443624.

Cited in. 1 place.

> **Full draft, Section 6.4.5.** The safe-Rust cycle leaks in silence, because leaking is safe in Rust by explicit design [**67, 68**].

Source says.

> *"rCanary: Detecting Memory Leaks Across Semi-Automated Memory Management Boundary in Rust". IEEE Transactions on Software Engineering 50(9), pages 2472-2484, 2024. DOI 10.1109/TSE.2024.3443624. Crossref and OpenAlex records read, abstract read in full.*
>
> *The abstract states the thesis's own `leak_cycle` conclusion in the authors' words. Rust's ownership model "is anticipated to eliminate memory leaks. However, we observed that user intervention drives it into semi-automated memory management and makes it error-prone to cause leaks. In contrast to violating memory-safety guarantees restricted by the `unsafe` keyword, **the boundary of leaking memory is implicit, and the compiler would not emit any warnings for developers**." Applied to more than 1,200 real crates, it found 19 with leaks.*
>
> ***Why this is the single best find in the sweep.** The reference-cycle leak is the boundary class an examiner is most likely to call artificial, and this is a 2024 journal paper whose entire premise is that the class is real, silent and common enough to be worth a static checker. It also supplies the exact framing the thesis wants: the leak boundary is implicit, where the `unsafe` boundary is marked. Cite it in §`sec:faultcat`, in the RQ2 discussion where LeakSanitizer catches what Rust misses, and in the future-work sentence about tooling, since rCanary is the Rust-side answer the thesis notes is missing. It pairs with `hughes2025gcrust` exactly as sweep 5 §E.1 predicted.*

Record. LITERATURE_NOTES.md, section C.3 Cui, Xu, Tian and Zhou, TSE 2024.

---

[69] `dietz2015overflow` peer reviewed, level A

Reference. W. Dietz et al. "Understanding Integer Overflow in C/C++". In: *ACM Transactions on Software Engineering and Methodology* 25.1 (2015), 2:1–2:29. DOI: 10.1145/2743019. URL: https://doi.org/10.1145/2743019

Entry. DOI 10.1145/2743019.

Cited in. 1 place.

Full draft, Section 6.4.5. Defined-but-wrong is easier to debug than undefined-and-wrong and no less incorrect [**69**].

Source says.

"*Understanding Integer Overflow in C/C++*". ACM Transactions on Software Engineering and Methodology *25(1), 2015. DOI 10.1145/2743019.* ***One field to settle before use: Crossref's deposit gives pages 1-29 while the journal numbers it as an article. Check the venue's expected form.***

From the abstract: "intentional uses of wraparound behaviors are more common than is widely believed", with "over 200 distinct locations in the SPEC CINT2000 benchmarks where overflow occurs", and, orthogonally, "overflows are found in both well-defined and undefined flavors" with applications having "been broken by improvements in compiler optimizations".

Why the thesis needs it. *The* `integer_overflow` *row is one of the two boundary classes where the C++ tooling came back ahead, and the thesis's own framing is that Rust* defines *release-mode wrapping while C++ leaves signed overflow undefined, which is why UndefinedBehaviorSanitizer has something to report. Dietz et al. is the peer-reviewed source for both halves of that: wrapping is often deliberate, so a checker cannot treat every overflow as a bug, and undefined overflow has broken real programs when optimisers improved. It also justifies why the class belongs in a defect catalogue at all rather than being dismissed as a contrived case.*

Record. LITERATURE_NOTES.md, section C.2 Dietz, Li, Regehr and Adve, TOSEM 2015.

---

[70] `bornholt2021shardstore`

peer reviewed, level A · Thesis_v2 [43]

Reference. J. Bornholt et al. "Using Lightweight Formal Methods to Validate a Key-Value Storage Node in Amazon S3". In: *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*. 2021, pp. 836–850. DOI: 10.1145/3477132.3483540. URL: https://doi.org/10.1145/3477132.3483540

Entry. DOI 10.1145/3477132.3483540.

Cited in. 2 places.

Full draft, Section 6.4.5. Two Rust model checkers answer it, in the division of labour a production Rust system has been validated with: Loom explores interleavings soundly under bounded partial-order reduction, and Shuttle samples them probabilistically to reach cases beyond Loom [**70**].

Thesis_v2, Chapter 6. Two Rust model checkers do report both original boundary defects, Loom exploring interleavings soundly and Shuttle sampling the ones Loom cannot reach [**43**], but only after each was re-imported against the checker's own primitives and shrunk to model size.

Source says.

"*Using Lightweight Formal Methods to Validate a Key-Value Storage Node in Amazon S3*". *James Bornholt, Rajeev Joshi, Vytautas Astrauskas, Brendan Cully, Bernhard Kragl, Seth Markle, Kyle Sauri, Drew Schleit, Grant Slatton, Serdar Tasiran, Jacob Van Geffen, Andrew Warfield. SOSP '21, pages 836–850. DOI 10.1145/3477132.3483540.*

This is the find of the sweep. *It is a peer-reviewed account of a production Rust system validated with* the same two model checkers this thesis uses, for the same reason*, by the team that wrote one of them. Read the paper's own description against* `Ha_Vien_Mastersthesis_2026.tex`*:2932–3047 and the draft is doing the same thing on a smaller object.*

Three passages are directly usable, quoted from the paper's section on checking concurrent executions:

- *Loom "implements the CDSChecker algorithm for sound model checking in the release/acquire memory model, and uses bounded partial-order reduction to reduce state explosion."*
- *"we developed and open-sourced the stateless model checker Shuttle, which implements randomized algorithms such as probabilistic concurrency testing."*

- *"We use Loom to soundly check all interleavings of small, correctness-critical code such as custom concurrency primitives, and Shuttle to randomly check interleavings of larger test harnesses to which Loom does not scale."*

That third sentence is the division of labour `faults/run_model_checking.sh` *implements, stated independently in a top venue.*

Record. LITERATURE_NOTES.md, section B.1 Bornholt et al., SOSP 2021.

---

[71] `emmerich2019drivers` peer reviewed, level A · Plan [6]

Reference. P. Emmerich et al. "The Case for Writing Network Drivers in High-Level Programming Languages". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–13. DOI: 10.1109/ANCS.2019.8901892. URL: https://doi.org/10.1109/ANCS.2019.8901892

Entry. DOI 10.1109/ANCS.2019.8901892.

Cited in. 7 places.

> **Full draft, Section 7.2.** The median result is consistent with the wider evidence that Rust's safety checks are largely absorbed by modern out-of-order CPUs [**12, 71**].
>
> **Full draft, Section 7.5.** Its safety checks are absorbed by out-of-order execution, not literally free [**71**].
>
> **Full draft, Section 7.5.** None is unoccupied on its own, as the matched implementations [**71, 82, 83, 84, 85, 89, 56**], the percentile sweep stopping at a latency requirement [**55**] and the effort measurements [**52, 78**] all show.
>
> **Full draft, Section 7.6.** Three findings have published company: median equivalence below saturation, with matched pairs in three settings putting safe Rust within single-digit percentages of its C or C++ counterpart [**71, 82, 85**], a cross-thread handoff dominating the end-to-end cost [**65**], and most of RQ2's boundary classes, which the literature already documents.
>
> **Full draft, Appendix B.** Compiled systems languages sit close on time and energy [**44, 91**], Rust's safety is cheap at run time [**12, 92**], and that holds where timing is tightest, in embedded hypervisors, production cloud infrastructure, and packet processing within a few percent of a C reference [**93, 22, 71, 94**].
>
> **Plan, Section 1.** Closer studies find real costs, both in networking code [**6**] and in specific mechanisms such as bounds and overflow checking [**7**].
>
> **Plan, Section 2.** Matched implementations built from one design exist in systems software [**6, 17, 18, 19**] and, in telecom, in Emberg and Wadsten.

Claim it carries. most sharply at : "running 4 % slower than its C reference while executing 63 % more instructions per packet"

Source says.

> *(arXiv 1909.06344, p. 1, abstract): "Out-of-order CPUs mitigate the cost of safety checks:* ***Our Rust driver executes 63% more instructions per packet but is only 4% slower than a reference C implementation.****" Table 9 (p. 11) gives the normalised results (Rust 98 % of C).*

Metadata. [verified] Author list follows the paper (IEEE's record drops "García" from Esaú García Sánchez-Torija, the entry is right).

Verdict. OK, both numbers are the source's own, in the source's own order.

Record. Sources_evidence.md, entry record.

---

[72] zhang2022runtime

peer reviewed, level A · Plan [7]

Reference. Y. Zhang et al. “Towards Understanding the Runtime Performance of Rust”. In: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2022, 140:1–140:6. DOI: 10.1145/3551349.3559494. URL: https://doi.org/10.1145/3551349.3559494

Entry. DOI 10.1145/3551349.3559494.

Cited in. 5 places.

> **Full draft, Section 7.2.** The prediction also named where to look first if that happened: bounds and overflow checking [**72, 73**].

> **Full draft, Section 7.2.** The mechanisms the literature has localised [**72, 73**] are candidates to test, not conclusions to import.

> **Full draft, Section 7.6.** The mechanisms most likely responsible have been isolated elsewhere [**72, 73**].

> **Full draft, Appendix B.** Where a cost does appear it has been pinned down: to bounds, overflow and division checks, to that same check set against hardware-enforced capabilities, and to recurring anti-patterns [**72, 73, 95**].

> **Plan, Section 1.** Closer studies find real costs, both in networking code [**6**] and in specific mechanisms such as bounds and overflow checking [**7**].

Claim it carries. the literature localises Rust’s cost to bounds, overflow and division checks, which is where the thesis’s registered prediction said to look first

Source says.

> *(author copy, §1 and §4 with Figure 2): “the major source of performance overhead is the run-time checks inserted by the compiler to ensure memory safety. For instance,* ***out-of-bounds checks account for 52.7% of all the overhead****.” And: “With all run-time checks enabled, Rust incurs an 1.96x overhead, which drops to 1.77x, 1.27x, and 1.23x after we in turn disable* ***integer-overflow checks, out-of-bound checks, and division-by-zero checks****.”*

Metadata. [verified] Crossref. Author copy at softsec.cs.utah.edu/papers/RustPerf.pdf, and ACM publishes a full-HTML version.

Verdict. OK, and precise. The source names exactly the three check kinds the thesis attributes to it, in that order, which is the strongest available support for §7.2’s account of what the registered prediction got wrong.

Record. Sources_evidence.md, entry record.

---

[73] harris2023morello

peer reviewed, level A

Reference. S. Harris et al. “Rust for Morello: Always-On Memory Safety, Even in Unsafe Code (Experience Paper)”. In: *Proceedings of the 37th European Conference on Object-Oriented Programming (ECOOP)*. vol. 263. Leibniz International Proceedings in Informatics (LIPIcs). 2023, 39:1–39:27. DOI: 10.4230/LIPIcs.ECOOP.2023.39. URL: https://doi.org/10.4230/LIPIcs.ECOOP.2023.39

Entry. DOI 10.4230/LIPIcs.ECOOP.2023.39.

Cited in. 4 places.

> **Full draft, Section 7.2.** The prediction also named where to look first if that happened: bounds and overflow checking [**72, 73**].

> **Full draft, Section 7.2.** The mechanisms the literature has localised [**72, 73**] are candidates to test, not conclusions to import.

> **Full draft, Section 7.6.** The mechanisms most likely responsible have been isolated elsewhere [**72, 73**].

> **Full draft, Appendix B.** Where a cost does appear it has been pinned down: to bounds, overflow and division checks, to that same check set against hardware-enforced capabilities, and to recurring anti-patterns **[72, 73, 95]**.

Claim it carries. the bounds check specifically, set against hardware-enforced capabilities

Source says.

> *(`LIPIcs.ECOOP.2023.39.pdf`, p. 3, contributions list): "A performance analysis of Rust on Morello (§4, §5): ... **Analysis of the benefits of bounds checking elision on capability hardware (§5.1).**" Abstract (p. 1): "a comparison of the performance of Rust's runtime safety checks and the hardware-supported checks of Morello".*

Metadata. [verified] The LIPIcs PDF's own running heads confirm **39:1–39:27** and volume 263. DataCite gives the container as "LIPIcs, Volume 263, ECOOP 2023", the entry's booktitle is the conference name, which is conventional.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[74] `zhang2026deadlock`

peer reviewed, level A · Thesis_v2 [45], Plan [22]

Reference. Y. Zhang et al. "Two Birds One Stone: Effective Static Detection of Resource and Communication Deadlocks in Rust Programs". In: *Automated Software Engineering* 33.2, 65 (2026). Extends the preprint arXiv:2401.01114. DOI: 10.1007/s10515-026-00614-z. URL: https://doi.org/10.1007/s10515-026-00614-z

Entry. DOI 10.1007/s10515-026-00614-z. Extends the preprint arXiv:2401.01114.

Cited in. 3 places.

> **Full draft, Section 7.2.** Static deadlock detection for Rust is an active research problem in its own right **[74]**, and model checkers for Rust kernel code are motivated explicitly by the observation that ownership and borrowing leave data races, deadlocks, and livelocks reachable **[75]**.

> **Thesis_v2, Chapter 7.** A `Mutex` guard whose lifetime the borrow checker tracks perfectly says nothing about whether two threads take their locks in a compatible order **[45]**.

> **Plan, Section 3.** Memory-safety defects in real Rust code invariably trace back to `unsafe` **[21]**, which one boundary class now reproduces directly, while deadlock remains an open problem for the language **[22]**.

Claim it carries. static deadlock detection for Rust is an active research problem

Source says.

> *(the preprint the entry's `note` points to, arXiv 2401.01114, p. 1): "Although Rust claims to enable fearless concurrency, **concurrency errors remain prevalent in practical applications, with deadlocks particularly prominent and harmful**. Current static deadlock detection techniques either do not cater to Rust programs ... or solely concentrate on resource deadlocks, disregarding communication deadlocks."*

Metadata. [verified] Crossref resolves the DOI and matches authors, journal, volume, issue and year.

Verdict. OK. Good that the journal version is cited rather than the preprint.

Record. Sources_evidence.md, entry record.

---

[75] `tang2025converos`

peer reviewed, level A

Reference. R. Tang et al. "Converos: Practical Model Checking for Verifying Rust OS Kernel Concurrency". In: *Proceedings of the 2025 USENIX Annual Technical Conference (USENIX ATC)*. USENIX Association, 2025, pp. 1143–1159. URL: https://www.usenix.org/conference/atc25/presentation/tang

Entry. source page.

Cited in. 1 place.

> **Full draft, Section 7.2.** Static deadlock detection for Rust is an active research problem in its own right **[74]**, and model checkers for Rust kernel code are motivated explicitly by the observation that ownership and borrowing leave data races, deadlocks, and livelocks reachable **[75]**.

Claim it carries. model checkers for Rust kernel code are motivated by ownership and borrowing leaving data races, deadlocks and livelocks reachable

Source says.

> *(`atc25-tang.pdf`, p. 2): "**Although Rust greatly enhances memory safety and prevents data races in safe code, OS kernel development inevitably requires unsafe, which may still introduce concurrency-related undefined behaviors such as data races. Logical errors and flawed designs can lead to deadlocks, livelocks, and functional violations.**" And, in the abstract: "uncovering 20 bugs that led to issues such as data races, deadlocks, livelocks, and kernel panics."*

Metadata. [verified] DBLP confirms pages **1143–1159** and the six-author list.

Verdict. OK, an unusually exact match to the sentence it supports.

Record. Sources_evidence.md, entry record.

---

[76] `lu2008mistakes` peer reviewed, level A

Reference. S. Lu et al. "Learning from Mistakes: A Comprehensive Study on Real World Concurrency Bug Characteristics". In: *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2008, pp. 329–339. DOI: 10.1145/1346281.1346323. URL: https://doi.org/10.1145/1346281.1346323

Entry. DOI 10.1145/1346281.1346323.

Cited in. 1 place.

> **Full draft, Section 7.2.** In a field study of 105 real concurrency bugs, about a third of the non-deadlock cases violated the order the programmers intended, not their locking, and the authors record that such intentions are not readily expressible through locks at all **[76]**.

Source says.

> *"Learning from Mistakes: A Comprehensive Study on Real World Concurrency Bug Characteristics". ASPLOS 2008, pages 329-339. DOI 10.1145/1346281.1346323. Crossref record read, abstract read in full.*
>
> *105 concurrency bugs from MySQL, Apache, Mozilla and OpenOffice. The finding that matters here, quoted from the abstract: "around one third of the examined non-deadlock concurrency bugs are caused by violation to programmers' order intentions, which may not be easily expressed via synchronization primitives like locks and transactional memories". A second one is directly about detection: "around 34% of the examined non-deadlock concurrency bugs involve multiple variables, which are not well addressed by existing bug detection tools".*
>
> ***Why the thesis needs it.*** *The `lost_update` boundary class is currently defended by the thesis's own reasoning: every access is correctly synchronised, so there is no unsynchronised access for ThreadSanitizer to find and nothing for the borrow checker to reject. This is a canonical, heavily cited paper saying that a third of real concurrency bugs are of exactly that shape, and that they are not expressible as a locking discipline. It converts the thesis's most easily doubted RQ2 row into an instance of a documented category. Cite it in §`sec:faultcat` beside `natella2016fault` and in the RQ2 discussion where the lost update is explained.*
>
> ***Caveat to state in the same sentence.*** *The sample is 2008-era C and C++ desktop and server software, so it establishes the category and not its present-day frequency.*

Record. LITERATURE_NOTES.md, section C.1 Lu, Park, Seo and Zhou, ASPLOS 2008.

---

[77] `sundvisson2026drivers`

thesis, level A · Thesis_v2 [47]

Reference. A. Sundvisson and S. Hofer. "User-Space Device Drivers on Embedded Linux: A Comparative Case Study of C and Rust". Bachelor's thesis. Blekinge Institute of Technology, Department of Software Engineering, 2026. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:bth-29775

Entry. source page.

Cited in. 4 places.

> **Full draft, Section 7.2.** A second Ericsson-hosted study of RAN node management software found Rust's peak memory usage generally higher than C's [**53**], and a matched pair of user-space device drivers outside telecom put the Rust version's footprint about 29% above the C version's [**77**].
>
> **Full draft, Section 7.5.** Two 2026 bachelor's theses from outside the vendor set apply parts of the method to smaller objects: matched C and Rust user-space drivers, with a fixed memory overhead and no measurable run-time cost [**77**], and a CWE staging study across C++, Rust and Python [**87**].
>
> **Full draft, Section 7.6.** Peak resident set size sits between the two groups, with the sign found here reported by two neighbouring theses and reversed by a third [**53, 77, 52**].
>
> **Thesis_v2, Chapter 7.** Two further studies agree with the direction found here [**46, 47**].

Claim it carries. matched C and Rust user-space drivers with the Rust footprint about 29 per cent above C, a fixed overhead and no measurable run-time cost

Source says.

> *Located in two places.*
>
> - *p. 3, abstract: "The C driver was approximately 0.178 % faster, a difference detectable at 99 % confidence but well within a ±1 % equivalence zone. The Rust driver consumed approximately 29 % more memory and contained five small unsafe blocks wrapping unavoidable system calls."*
> - *p. 35: "Rust maintains a constant footprint of 1800 kB, whereas C exhibits 1400 kB, corresponding to an approximately 29 % increase."*

Metadata. [verified] DiVA `diva2:2071063`, full text `FULLTEXT01.pdf` retrieved.

Verdict. OK, and stronger than the thesis needs. "Constant footprint" is the source's own word for the fixed overhead, and the ±1 per cent equivalence zone is what licenses "no measurable run-time cost".

Record. Sources_evidence.md, entry record.

---

[78] `fulton2021adopting`

peer reviewed, level A · Thesis_v2 [48], Plan [31]

Reference. K. R. Fulton et al. "Benefits and Drawbacks of Adopting a Secure Programming Language: Rust as a Case Study". In: *Proceedings of the 17th Symposium on Usable Privacy and Security (SOUPS)*. USENIX Association, 2021, pp. 597–616. URL: https://www.usenix.org/conference/soups2021/presentation/fulton

Entry. source page.

Cited in. 6 places.

> **Full draft, Section 7.2.** The dominant cost is therefore human and not machine: designing a C-compatible surface and, as the adoption studies report [**78, 79**], climbing Rust's learning curve.
>
> **Full draft, Section 7.5.** None is unoccupied on its own, as the matched implementations [**71, 82, 83, 84, 85, 89, 56**], the percentile sweep stopping at a latency requirement [**55**] and the effort measurements [**52, 78**] all show.
>
> **Full draft, Appendix B.** The fourth strand is the domain and the method: the tail-latency and microsecond-scale literature [**6, 7, 30, 104**], thread-per-core structuring moving the tail in its own right [**45**], O-RAN disaggregation [**3**], control-plane latency as an active target in the core network too [**105, 106**], adoption evidence pointing at the learning curve rather than run-time cost [**78, 79, 107**], and standard guidance on repeat-based evaluation and fault injection [**32, 59**].

Full draft, Appendix G. The dominant cost identified here is human, which matches what adoption studies report [**78**].

Thesis_v2, Chapter 7. On migration the dominant cost is human [**48, 49**].

Plan, Section 5. item Evidence-based guidelines for incremental Rust adoption, where the dominant cost is the learning curve rather than run-time [**31**].

Claim it carries. the dominant cost is human, not run-time

Source says.

> *(soups2021-fulton.pdf, p. 2, abstract): "a range of positive features … **as well as drawbacks including a steep learning curve, limited library support, and concerns about the ability to hire additional Rust developers**", p. 3: "**most seriously a steep learning curve** to adjust to the paradigms that enforce security guarantees" and "much of the **cost of adoption occurs up front**, while benefits tend to accrue later".*

Metadata. [verified] DBLP confirms pages **597–616** and the five-author list.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[79] `crichton2024learning`

peer reviewed, level A · Thesis_v2 [49]

Reference. W. Crichton and S. Krishnamurthi. "Profiling Programming Language Learning". In: *Proceedings of the ACM on Programming Languages* 8.OOPSLA1 (2024), pp. 29–54. DOI: 10.1145/3649812. URL: https://doi.org/10.1145/3649812

Entry. DOI 10.1145/3649812.

Cited in. 3 places.

Full draft, Section 7.2. The dominant cost is therefore human and not machine: designing a C-compatible surface and, as the adoption studies report [**78, 79**], climbing Rust's learning curve.

Full draft, Appendix B. The fourth strand is the domain and the method: the tail-latency and microsecond-scale literature [**6, 7, 30, 104**], thread-per-core structuring moving the tail in its own right [**45**], O-RAN disaggregation [**3**], control-plane latency as an active target in the core network too [**105, 106**], adoption evidence pointing at the learning curve rather than run-time cost [**78, 79, 107**], and standard guidance on repeat-based evaluation and fault injection [**32, 59**].

Thesis_v2, Chapter 7. On migration the dominant cost is human [**48, 49**].

Claim it carries. the learning curve as the dominant adoption cost

Source says.

> *(arXiv 2401.01257, p. 1, abstract, restated p. 2): "Over 13 months, **62,526 readers answered questions 1,140,202 times** … we find that **many readers drop-out of the book early when faced with difficult language concepts like Rust's ownership types**."*

Metadata. [verified] Crossref matches every field.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[80] `li2026rustlinux`

peer reviewed, level B

Reference. H. Li et al. "Rust Meets Linux: Lessons from an Evolving Experiment". In: *ACM Transactions on Computer Systems* 44.2 (2026). Extended version of the authors' USENIX ATC '24 study of Rust-for-Linux, 9:1–9:33. DOI: 10.1145/3789260. URL: https://doi.org/10.1145/3789260

Entry. DOI 10.1145/3789260. Extended version of the authors' USENIX ATC '24 study of Rust-for-Linux.

Cited in. 3 places.

> **Full draft, Section 7.2.** That makes migration risk an organisational concern and not a performance one, and the two largest documented adoptions into existing C and C++ codebases report the same ordering of costs [**80, 81**].
>
> **Full draft, Section 7.5.** It finds that Rust's safety features introduce runtime overhead and development complexity when they are not carefully managed [**80**].
>
> **Full draft, Appendix B.** The seam carries its own defect classes and its own tooling bugs, lighter crossing models exist for embedded use, aliasing patterns expressible in C but not in safe Rust bound translation in principle, interoperation is the central open problem in embedded Rust, `unsafe` concentrates at exactly these boundaries, and the two largest documented adoptions into existing C and C++ report the resulting friction empirically [**49, 50, 99, 100, 101, 102, 80, 81**].

Source says.

> *Li, Guo, Yang, Wang & Xu, ACM TOCS 44(2), 2026. "Rust Meets Linux: Lessons from an Evolving Experiment". Same author list as `li2024rustforlinux` (USENIX ATC '24): this is the journal version, and it sharpens the conclusion. "If not properly managed, Rust's safety features even introduce notable runtime overhead and increased development complexity." Citation **`upgrade`**, and a contrary result the thesis should carry rather than soften.* Article number not confirmed. Crossref reports pages 1–33, check before citing with a page range.

Record. LITERATURE_NOTES.md, peer-reviewed additions of the third sweep.

---

[81] `fan2026android` peer reviewed, level B

Reference. Y. Fan et al. "An Empirical Analysis of Rust Integration in Android Open Source Project". In: *Companion Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering (FSE Companion)*. 2026, pp. 545–556. DOI: 10.1145/3803437.3805227. URL: https://doi.org/10.1145/3803437.3805227

Entry. DOI 10.1145/3803437.3805227.

Cited in. 2 places.

> **Full draft, Section 7.2.** That makes migration risk an organisational concern and not a performance one, and the two largest documented adoptions into existing C and C++ codebases report the same ordering of costs [**80, 81**].
>
> **Full draft, Appendix B.** The seam carries its own defect classes and its own tooling bugs, lighter crossing models exist for embedded use, aliasing patterns expressible in C but not in safe Rust bound translation in principle, interoperation is the central open problem in embedded Rust, `unsafe` concentrates at exactly these boundaries, and the two largest documented adoptions into existing C and C++ report the resulting friction empirically [**49, 50, 99, 100, 101, 102, 80, 81**].

Claim it carries. one of the two largest documented adoptions into an existing C and C++ codebase

Source says.

> *(deposited abstract): "the first longitudinal and systematic empirical study of **Rust in AOSP**. By analyzing 16 quarterly snapshots from January 2021 to October 2024, we track adoption trends and map the distribution of Rust code across 67 components … we characterize platform-level integration mechanisms and examine unsafe governance."*

Verdict. OK for the adoption and integration claim. The "same ordering of costs" that §7.2 draws from pairing this with `li2026rustlinux` is the thesis's own synthesis of the two, which is how it is written.

Record. Sources_evidence.md, entry record.

[82] li2021ipc

peer reviewed, level A · Plan [17]

Reference. Z. Li et al. “Understanding the Overheads of Hardware and Language-Based IPC Mechanisms”. In: *Proceedings of the 11th Workshop on Programming Languages and Operating Systems (PLOS)*. 2021, pp. 53–61. DOI: 10.1145/3477113.3487275. URL: https://doi.org/10.1145/3477113.3487275

Entry. DOI 10.1145/3477113.3487275.

Cited in. 4 places.

> **Full draft, Section 7.5.** Four systems studies come closest in method, though not in domain, because they too build the same component twice and attribute the difference: IPC mechanisms for one network function **[82]**, RedLeaf against purpose-built C baselines **[83]**, a concurrent STAMP translation that is the closest published match to this design **[84]**, and a kernel UDP tunnelling driver **[85]**.

> **Full draft, Section 7.5.** None is unoccupied on its own, as the matched implementations **[71, 82, 83, 84, 85, 89, 56]**, the percentile sweep stopping at a latency requirement **[55]** and the effort measurements **[52, 78]** all show.

> **Full draft, Section 7.6.** Three findings have published company: median equivalence below saturation, with matched pairs in three settings putting safe Rust within single-digit percentages of its C or C++ counterpart **[71, 82, 85]**, a cross-thread handoff dominating the end-to-end cost **[65]**, and most of RQ2’s boundary classes, which the literature already documents.

> **Plan, Section 2.** Matched implementations built from one design exist in systems software **[6, 17, 18, 19]** and, in telecom, in Emberg and Wadsten.

Claim it carries. a matched C++ and Rust pair in a network-function setting putting safe Rust within single-digit percentages of unsafe C++

Source says.

> *(author copy, abstract and §4 Conclusions): “we develop two implementations (one in C++ and one in Rust) of a network function processing framework similar to Netbricks … We find that while high-level Rust abstractions can introduce significant overhead, a carefully planned (with respect to performance) Rust implementation remains fast and **executes within 4-8% of unsafe C++**.” Restated in the conclusions: “on realistic workloads Rust incurs only minor overhead of 4-8% compared to unsafe C++.”*

Metadata. [verified] Crossref, pages 53-61. Author copies at mars-research.github.io and users.cs.utah.edu/~aburtsev.

Verdict. OK. 4 to 8 per cent is single-digit percentages, and the matched-pair design is the source’s own. Two scope notes above: the figure is conditional on a carefully planned implementation, and the paper builds several network functions rather than one.

Record. Sources_evidence.md, entry record.

[83] narayanan2020redleaf

peer reviewed, level A

Reference. V. Narayanan et al. “RedLeaf: Isolation and Communication in a Safe Operating System”. In: *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2020, pp. 21–39. URL: https://www.usenix.org/conference/osdi20/presentation/narayanan-vikram

Entry. source page.

Cited in. 2 places.

> **Full draft, Section 7.5.** Four systems studies come closest in method, though not in domain, because they too build the same component twice and attribute the difference: IPC mechanisms for one network function **[82]**, RedLeaf against purpose-built C baselines **[83]**, a concurrent STAMP translation that is the closest published match to this design **[84]**, and a kernel UDP tunnelling driver **[85]**.

> **Full draft, Section 7.5.** None is unoccupied on its own, as the matched implementations **[71, 82, 83, 84, 85, 89, 56]**, the percentile sweep stopping at a latency requirement **[55]** and the effort measurements **[52, 78]** all show.

Claim it carries. evaluated against C baselines constructed for the purpose

Source says.

> *(`osdi20-narayanan_vikram.pdf`): p. 2. "we develop efficient versions of a 10Gbps Intel ixgbe network and NVMe solid-state disk device drivers that **match the performance of the fastest DPDK and SPDK equivalents**", p. 12, Figure 5. "C vs Rust performance comparison", and the ixgbe driver comparison against Linux and DPDK.*

Metadata. [verified] DBLP confirms pages **21–39** and the seven-author list as entered.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[84] `suchert2023stamp`

peer reviewed, level B · Plan [18]

Reference. F. Suchert and J. Castrillon. "STAMP-Rust: Language and Performance Comparison to C on Transactional Benchmarks". In: *Benchmarking, Measuring, and Optimizing (Bench 2022)*. Vol. 13852. Lecture Notes in Computer Science. Springer, 2023, pp. 160–175. DOI: 10.1007/978-3-031-31180-2_10. URL: https://doi.org/10.1007/978-3-031-31180-2_10

Entry. DOI 10.1007/978-3-031-31180-2_10.

Cited in. 4 places.

> **Full draft, Section 7.5.** Four systems studies come closest in method, though not in domain, because they too build the same component twice and attribute the difference: IPC mechanisms for one network function **[82]**, RedLeaf against purpose-built C baselines **[83]**, a concurrent STAMP translation that is the closest published match to this design **[84]**, and a kernel UDP tunnelling driver **[85]**.

> **Full draft, Section 7.5.** Idiomatic safe Rust implementations of the STAMP transactional benchmarks run slower than both the C originals and machine-transpiled Rust **[84]**, and a general study of Rust against C reports it significantly slower in some cases **[88]**.

> **Full draft, Section 7.5.** None is unoccupied on its own, as the matched implementations **[71, 82, 83, 84, 85, 89, 56]**, the percentile sweep stopping at a latency requirement **[55]** and the effort measurements **[52, 78]** all show.

> **Plan, Section 2.** Matched implementations built from one design exist in systems software **[6, 17, 18, 19]** and, in telecom, in Emberg and Wadsten.

Source says.

> *Suchert & Castrillon, Bench 2022 (LNCS 13852:160–175, publ. 2023). Manually translated the **STAMP** transactional benchmark suite to Rust, plus a `c2rust`-transpiled version. Finding: "idiomatic implementations using safe Rust are generally slower than both C and transpiled code, they guarantee memory safety and improve code quality." A matched pair on a **concurrent** workload, contrary result, peer-reviewed. Better neighbour for `tab:related` than any of the general-purpose benchmark rows.*

Record. LITERATURE_NOTES.md, peer-reviewed additions of the third sweep.

---

[85] `gonzalez2023rustudp`

peer reviewed, level B · Plan [19]

Reference. A. Gonzalez, D. Mvondo, and Y.-D. Bromberg. "Takeaways of Implementing a Native Rust UDP Tunneling Network Driver in the Linux Kernel". In: *Proceedings of the 12th Workshop on Programming*

Languages and Operating Systems (PLOS). 2023, pp. 18–25. DOI: 10.1145/3623759.3624547. URL: https://doi.org/10.1145/3623759.3624547

Entry. DOI 10.1145/3623759.3624547.

Cited in. 4 places.

> **Full draft, Section 7.5.** Four systems studies come closest in method, though not in domain, because they too build the same component twice and attribute the difference: IPC mechanisms for one network function [**82**], RedLeaf against purpose-built C baselines [**83**], a concurrent STAMP translation that is the closest published match to this design [**84**], and a kernel UDP tunnelling driver [**85**].
>
> **Full draft, Section 7.5.** None is unoccupied on its own, as the matched implementations [**71, 82, 83, 84, 85, 89, 56**], the percentile sweep stopping at a latency requirement [**55**] and the effort measurements [**52, 78**] all show.
>
> **Full draft, Section 7.6.** Three findings have published company: median equivalence below saturation, with matched pairs in three settings putting safe Rust within single-digit percentages of its C or C++ counterpart [**71, 82, 85**], a cross-thread handoff dominating the end-to-end cost [**65**], and most of RQ2's boundary classes, which the literature already documents.
>
> **Plan, Section 2.** Matched implementations built from one design exist in systems software [**6, 17, 18, 19**] and, in telecom, in Emberg and Wadsten.

Source says.

> *Gonzalez, Mvondo & Bromberg, PLOS@SOSP 2023 (pp. 18–25). Rust UDP tunnelling driver in the Linux kernel against an equivalent C driver: latency* ***+0.1906%*** *(p = 1.464e-15), throughput* ***−0.00090%*** *(p = 6.004e-5). A matched networking pair whose difference is statistically overwhelming and practically nil, the cleanest published instance of the distinction the thesis's pre-committed interval-vs-*p *rule (§5.5) exists to enforce.*

Record. LITERATURE_NOTES.md, peer-reviewed additions of the third sweep.

---

[86] `zhuang2026flowprioritize` peer reviewed, level B

Reference. K. Zhuang. "FlowPrioritize: A Model-Driven Priority Scheduling Library for Asynchronous Rust Runtimes". In: *SoftwareX* 35 (2026), p. 102771. DOI: 10.1016/j.softx.2026.102771. URL: https://doi.org/10.1016/j.softx.2026.102771

Entry. DOI 10.1016/j.softx.2026.102771.

Cited in. 1 place.

> **Full draft, Section 7.5.** Priority scheduling is something an asynchronous Rust runtime has to be given by a library [**86**].

Source says.

> *Zhuang, SoftwareX 35:102771, 2026. FlowPrioritize, a priority-scheduling library for async Rust runtimes. Supporting evidence, alongside* ***skoudlil2025asyncrust****, that priority and determinism are things async Rust runtimes must be* given*, not properties they have.*

Record. LITERATURE_NOTES.md, peer-reviewed additions of the third sweep.

---

[87] `andersson2026stdmemsafety` thesis, level A

Reference. O. Andersson. "Programming Language Standards and Their Impact on Memory Safety: C++, Rust and Python". Report LIU-IDA/LITH-EX-G–26/057–SE. Bachelor's thesis. Linköping University, Department of Computer and Information Science, 2026. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-226479

Entry. source page. Report LIU-IDA/LITH-EX-G–26/057–SE.

Cited in. 1 place.

> **Full draft, Section 7.5.** Two 2026 bachelor's theses from outside the vendor set apply parts of the method to smaller objects: matched C and Rust user-space drivers, with a fixed memory overhead and no measurable run-time cost **[77]**, and a CWE staging study across C++, Rust and Python **[87]**.

Claim it carries. a CWE staging study across C++, Rust and Python

Source says.

> *(`FULLTEXT01.pdf`, p. 4): "the manual memory management of C++, the ownership-based compile-time model of Rust, and the automatic garbage collection of Python. These languages are evaluated by reproducing a suite of Common Weakness Enumeration (CWE) vulnerabilities, including buffer overflows, use-after-free, and null pointer dereference conditions to determine the stage at which faults are detected: compile-time, runtime, or silent execution."*

Metadata. [verified] DiVA `diva2:2091281`, full text `FULLTEXT01.pdf` retrieved.

Verdict. OK. "Stage at which faults are detected: compile-time, runtime, or silent execution" is the same three-outcome partition this thesis uses for RQ2, in the source's own words.

Record. Sources_evidence.md, entry record.

---

[88] `okawa2024ruststudy` peer reviewed, level B

Reference. S. Okawa and S. Yamaguchi. "A Performance Study on Rust and C Programs". In: *Proceedings of the 12th International Symposium on Computing and Networking Workshops (CANDARW)*. 2024, pp. 355–359. DOI: 10.1109/CANDARW64572.2024.00066. URL: https://doi.org/10.1109/CANDARW64572.2024.00066

Entry. DOI 10.1109/CANDARW64572.2024.00066.

Cited in. 1 place.

> **Full draft, Section 7.5.** Idiomatic safe Rust implementations of the STAMP transactional benchmarks run slower than both the C originals and machine-transpiled Rust **[84]**, and a general study of Rust against C reports it significantly slower in some cases **[88]**.

Source says.

> *Okawa & Yamaguchi, CANDARW 2024 (pp. 355–359), finds Rust "significantly lower" in some cases and proposes mitigation. Minor, small venue, use only if a second general contrary result is wanted next to `emmerich2019drivers`.*

Record. LITERATURE_NOTES.md, peer-reviewed additions of the third sweep.

---

[89] `pirelli2023safelowlevel` peer reviewed, level B

Reference. S. Pirelli and G. Candea. "Safe Low-Level Code Without Overhead is Practical". In: *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*. 2023, pp. 2173–2184. DOI: 10.1109/ICSE48619.2023.00183. URL: https://doi.org/10.1109/ICSE48619.2023.00183

Entry. DOI 10.1109/ICSE48619.2023.00183.

Cited in. 2 places.

> **Full draft, Section 7.5.** Safe low-level code without run-time overhead has been demonstrated for network device drivers, the same class of program as Emmerich's, when the safety obligations are discharged through type invariants rather than checks **[89]**.

> **Full draft, Section 7.5.** None is unoccupied on its own, as the matched implementations **[71, 82, 83, 84, 85, 89, 56]**, the percentile sweep stopping at a latency requirement **[55]** and the effort measurements **[52, 78]** all show.

Source says.

> *Pirelli & Candea, ICSE 2023 (pp. 2173–2184). Argues safety without run-time overhead is achievable through type invariants, and shows* ***safe network device drivers with no run-time overhead*** *in extended Rust and C#. The peer-reviewed counterweight to* `emmerich2019drivers`*: the thesis currently presents the "not literally free" side of this question only.*

Record. LITERATURE_NOTES.md, peer-reviewed additions of the third sweep.

---

[90] `qin2024safety` peer reviewed, level B

Reference. B. Qin et al. "Understanding and Detecting Real-World Safety Issues in Rust". In: *IEEE Transactions on Software Engineering* 50.6 (2024), pp. 1306–1324. DOI: 10.1109/TSE.2024.3380393. URL: https://doi.org/10.1109/TSE.2024.3380393

Entry. DOI 10.1109/TSE.2024.3380393.

Cited in. 2 places.

> **Full draft, Section 7.5.** Empirical studies of real-world Rust find that memory- and thread-safety bugs still occur in practice, with blocking bugs and misused synchronisation among them **[27, 90]**.

> **Full draft, Appendix B.** The safety strand is settled the same partial way, with machine-checked foundations for the safe fragment and the aliasing discipline `unsafe` code has to respect **[11, 39, 40]**, confirmation that `unsafe` is used sparingly and that memory-safety CVEs always involve it **[27, 41, 42]**, auditing that finds defects at scale even so and unsafety spreading through dependency chains **[97, 98, 90, 43]**, and the reports motivating the shift set against a long record of memory corruption resisting runtime mitigation **[8, 9, 17, 19, 20, 33]**.

Claim it carries. memory- and thread-safety bugs still occur in practice, with blocking bugs and misused synchronisation among them

Source says.

> *(deposited abstract): "a comprehensive, empirical study of Rust safety issues by close, manual inspection of* ***70 memory bugs, 100 concurrency bugs, and 110 programming errors leading to unexpected execution panics*** *from five open-source Rust projects, five widely-used Rust libraries, and two online security databases."*

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[91] `pereira2017energy` peer reviewed, level A · Plan [5]

Reference. R. Pereira et al. "Energy Efficiency Across Programming Languages: How Do Energy, Time, and Memory Relate?" In: *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*. 2017, pp. 256–267. DOI: 10.1145/3136014.3136031. URL: https://doi.org/10.1145/3136014.3136031

Entry. DOI 10.1145/3136014.3136031.

Cited in. 2 places.

> **Full draft, Appendix B.** Compiled systems languages sit close on time and energy **[44, 91]**, Rust's safety is cheap at run time **[12, 92]**, and that holds where timing is tightest, in embedded hypervisors, production cloud infrastructure, and packet processing within a few percent of a C reference **[93, 22, 71, 94]**.

> **Plan, Section 1.** The "Rust performs like C++" consensus rests on general-purpose benchmarks **[5]** that report average-case behaviour.

Claim it carries. compiled systems languages sit close together on time and energy, from general-purpose benchmarks reporting average-case behaviour

Source says.

> *(Table 4, "Normalized global results for Energy, Time, and Memory", p. 263): C is the 1.00 baseline on both axes,* ***Rust is 1.03 energy and 1.04 time****, Go 1.05, C++ 1.34 energy and 1.56 time. The study covers "twenty seven well-known software languages" over "ten different programming problems", which is the general-purpose average-case design the thesis attributes to it.*

Metadata. [verified] Crossref. Green open access, and Table 4 is reproduced in several independent places.

Verdict. OK. See the scope note above: the table supports the thesis's point more strongly than the sentence claims, and the thesis is right not to quote the figures.

Record. Sources_evidence.md, entry record.

---

[92] `levy2017kernel` peer reviewed, level B

Reference. A. Levy et al. "The Case for Writing a Kernel in Rust". In: *Proceedings of the 8th Asia-Pacific Workshop on Systems (APSys)*. 2017, pp. 1–7. DOI: 10.1145/3124680.3124717. URL: https://doi.org/10.1145/3124680.3124717

Entry. DOI 10.1145/3124680.3124717.

Cited in. 1 place.

> **Full draft, Appendix B.** Compiled systems languages sit close on time and energy [**44, 91**], Rust's safety is cheap at run time [**12, 92**], and that holds where timing is tightest, in embedded hypervisors, production cloud infrastructure, and packet processing within a few percent of a C reference [**93, 22, 71, 94**].

Claim it carries. inside a list, that Rust's safety is cheap at run time in kernel settings

Source says.

> *(deposited abstract): "Previous efforts towards this end concluded that writing a kernel requires changing Rust. This paper reaches a different conclusion, that* ***no changes to Rust are needed and a kernel can be implemented with a very small amount of unsafe code****."*

Verdict. OK for feasibility with little `unsafe`, which is what the list uses it for. The run-time cost half of that list sentence is carried by `balasubramanian2017rust` beside it, and this entry should not be quoted alone for a cost claim.

Record. Sources_evidence.md, entry record.

---

[93] `mo2023rustshyper` peer reviewed, level C

Reference. C. Mo et al. "Rust-Shyper: A Reliable Embedded Hypervisor Supporting VM Migration and Hypervisor Live-Update". In: *Journal of Systems Architecture* 142 (2023), p. 102948. DOI: 10.1016/j.sysarc.2023.102948. URL: https://doi.org/10.1016/j.sysarc.2023.102948

Entry. DOI 10.1016/j.sysarc.2023.102948.

Cited in. 1 place.

> **Full draft, Appendix B.** Compiled systems languages sit close on time and energy [**44, 91**], Rust's safety is cheap at run time [**12, 92**], and that holds where timing is tightest, in embedded hypervisors, production cloud infrastructure, and packet processing within a few percent of a C reference [**93, 22, 71, 94**].

Source says.

> *Elsevier paywall, title-level support for "embedded hypervisor in Rust".*

Metadata. [verified] Crossref matches, including the article number as page.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[94] `xiao2024rpkt` peer reviewed, level C

Reference. Y. Xiao et al. "rpkt: A Generic, Safe, and Efficient Userspace Packet Processing Library in Rust". In: *Network and Parallel Computing (NPC 2024)*. Lecture Notes in Computer Science. 2025, pp. 454–465. DOI: 10.1007/978-981-96-2864-3_36. URL: https://doi.org/10.1007/978-981-96-2864-3_36

Entry. DOI 10.1007/978-981-96-2864-3_36.

Cited in. 1 place.

> **Full draft, Appendix B.** Compiled systems languages sit close on time and energy [**44, 91**], Rust's safety is cheap at run time [**12, 92**], and that holds where timing is tightest, in embedded hypervisors, production cloud infrastructure, and packet processing within a few percent of a C reference [**93, 22, 71, 94**].

Claim it carries. packet processing within a few percent of a C reference

Source says.

> *Springer paywall, title-level support ("Generic, Safe, and Efficient Userspace Packet Processing Library in Rust"). The "few percent" half of the sentence is carried by* `emmerich2019drivers`*, which is verified above and cited alongside.*

Metadata. [verified] Pages **454–465** confirmed by DBLP and Crossref. Note the split year: DBLP lists the conference as **NPC 2024**, Springer published the LNCS proceedings in **2025**, which is what the entry's `year` field says. Optional addition: `series = {Lecture Notes in Computer Science}`.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[95] `cui2025perfbugs` peer reviewed, level B

Reference. C. Cui and H. Xu. "Unleashing the Efficiency of Rust: An Empirical Study of Performance Bugs in Rust Projects". In: *Proceedings of the 2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*. 2025, pp. 371–381. DOI: 10.1109/ISSRE66568.2025.00045. URL: https://doi.org/10.1109/ISSRE66568.2025.00045

Entry. DOI 10.1109/ISSRE66568.2025.00045.

Cited in. 1 place.

> **Full draft, Appendix B.** Where a cost does appear it has been pinned down: to bounds, overflow and division checks, to that same check set against hardware-enforced capabilities, and to recurring anti-patterns [**72, 73, 95**].

Claim it carries. recurring performance anti-patterns

Source says.

> *(deposited abstract): "we have mined the repository of 100 well-known projects on GitHub and collected 114 performance bugs. We manually audit each case and find* ***8 optimization patterns unique to Rust****, including three types of checking, two types of cloning, and three types of data collection."*

Verdict. OK. "Recurring anti-patterns" is the abstract's eight patterns.

Record. Sources_evidence.md, entry record.

---

[96] naylor2026cherisimt

peer reviewed, level A

Reference. M. Naylor et al. “CHERI-SIMT: Implementing Capability Memory Protection in GPUs”. In: *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2026, pp. 50–66. DOI: 10.1145/3760250.3762234. URL: https://doi.org/10.1145/3760250.3762234

Entry. DOI 10.1145/3760250.3762234.

Cited in. 1 place.

> **Full draft, Appendix B.** One like-for-like port puts the bounds-check share at 34% of a 46% geometric-mean overhead, and cautions that code credited with cheap checks has sometimes declared its hot functions `unsafe` [**96**].

Source says.

> *“CHERI-SIMT: Implementing Capability Memory Protection in GPUs”. Matthew Naylor, Alexandre Joannou, A. Theodore Markettos, Paul Metzger, Simon W. Moore, Timothy M. Jones. ASPLOS ’26, pages 50–66. DOI 10.1145/3760250.3762234.* ***PARTIAL on the year: Crossref’s deposit records 2025 while the event field records ASPLOS ’26. Confirm the citable year before use.***
>
> *The GPU subject looks off-topic and the relevant part is not the subject. The authors build a* ***like-for-like C++-to-Rust port*** *of a benchmark suite and report “a geometric mean overhead of 46% from using Rust”, of which “bounds checking alone accounts for a 34% overhead”.*
>
> ***The reason to add it is the paragraph after that number.*** *It engages `harris2023morello`, which this thesis already cites, and disputes how it should be read: prior work “reports that the Rust compiler is very good at eliminating bounds checks, resulting in low overhead … However, care is needed when interpreting these results as it is unclear how many bounds checks are avoided due to the use of unsafe code blocks. For example, in the matrixmultiply benchmark they use, the key functions are declared as unsafe and operate directly on raw pointers.”*

Record. LITERATURE_NOTES.md, section F.0b Naylor et al., ASPLOS 2026.

---

[97] bae2021rudra

peer reviewed, level B

Reference. Y. Bae et al. “Rudra: Finding Memory Safety Bugs in Rust at the Ecosystem Scale”. In: *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*. 2021, pp. 84–99. DOI: 10.1145/3477132.3483570. URL: https://doi.org/10.1145/3477132.3483570

Entry. DOI 10.1145/3477132.3483570.

Cited in. 1 place.

> **Full draft, Appendix B.** The safety strand is settled the same partial way, with machine-checked foundations for the safe fragment and the aliasing discipline `unsafe` code has to respect [**11, 39, 40**], confirmation that `unsafe` is used sparingly and that memory-safety CVEs always involve it [**27, 41, 42**], auditing that finds defects at scale even so and unsafety spreading through dependency chains [**97, 98, 90, 43**], and the reports motivating the shift set against a long record of memory corruption resisting runtime mitigation [**8, 9, 17, 19, 20, 33**].

Claim it carries. ecosystem-scale auditing finds defects nonetheless

Source says.

> *(deposited abstract): “Rust’s safety guarantee relies on the soundness of* ***all unsafe code in the program as well as the standard and external libraries****, making it hard to reason about their correctness. In other words,* ***a single bug in any unsafe code breaks the whole program’s safety guarantee.”***

Verdict. OK for the premise. The ecosystem-scale audit result itself, the package-set scan and its defect count, sits in the body rather than the deposited abstract, so this is the one entry in this group whose claim

is only partly carried by what was read. It is used inside a list beside `zheng2024security`, `qin2024safety` and `evans2020rust`, all of which carry it. A gold open-access PDF exists at ACM for anyone wanting to close it properly.

Record. Sources_evidence.md, entry record.

---

[98] `zheng2024security` peer reviewed, level A

Reference. X. Zheng et al. "A Closer Look at the Security Risks in the Rust Ecosystem". In: *ACM Transactions on Software Engineering and Methodology* 33.2 (2024), 34:1–34:30. DOI: 10.1145/3624738. URL: https://doi.org/10.1145/3624738

Entry. DOI 10.1145/3624738.

Cited in. 1 place.

> **Full draft, Appendix B.** The safety strand is settled the same partial way, with machine-checked foundations for the safe fragment and the aliasing discipline `unsafe` code has to respect **[11, 39, 40]**, confirmation that `unsafe` is used sparingly and that memory-safety CVEs always involve it **[27, 41, 42]**, auditing that finds defects at scale even so and unsafety spreading through dependency chains **[97, 98, 90, 43]**, and the reports motivating the shift set against a long record of memory corruption resisting runtime mitigation **[8, 9, 17, 19, 20, 33]**.

Source says.

> *(arXiv 2308.15046, p. 1): "we first compile a dataset of **433 vulnerabilities, 300 vulnerable code repositories, and 218 vulnerability fix commits** in the Rust ecosystem, spanning over 7 years."*

Metadata. [verified] TOSEM 33(2), issue dated February 2024 → `year = 2024` right (Crossref online-first 2023). Pages `34:1-34:30` correct.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[99] `schuermann2023encapsulated` peer reviewed, level B

Reference. L. Schuermann, A. Thomas, and A. Levy. "Encapsulated Functions: Fortifying Rust's FFI in Embedded Systems". In: *Proceedings of the 1st Workshop on Kernel Isolation, Safety and Verification (KISV)*. 2023, pp. 41–48. DOI: 10.1145/3625275.3625397. URL: https://doi.org/10.1145/3625275.3625397

Entry. DOI 10.1145/3625275.3625397.

Cited in. 1 place.

> **Full draft, Appendix B.** The seam carries its own defect classes and its own tooling bugs, lighter crossing models exist for embedded use, aliasing patterns expressible in C but not in safe Rust bound translation in principle, interoperation is the central open problem in embedded Rust, `unsafe` concentrates at exactly these boundaries, and the two largest documented adoptions into existing C and C++ report the resulting friction empirically **[49, 50, 99, 100, 101, 102, 80, 81]**.

Claim it carries. lighter crossing models proposed for embedded use

Source says.

> *(deposited abstract): "practical systems must interact with code written in memory-unsafe languages. This is especially true in **security and safety-critical embedded systems**, where subsystems such as cryptographic implementations are subject to industrial and governmental certification requirements … **Any single bug in either the foreign code or the cross-language interactions may arbitrarily violate the memory safety of the wrapping language.**"*

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[100] `emre2023aliasing` peer reviewed, level B

Reference. M. Emre et al. “Aliasing Limits on Translating C to Safe Rust”. In: *Proceedings of the ACM on Programming Languages* 7.OOPSLA1 (2023), pp. 551–579. DOI: 10.1145/3586046. URL: https://doi.org/10.1145/3586046

Entry. DOI 10.1145/3586046.

Cited in. 1 place.

> **Full draft, Appendix B.** The seam carries its own defect classes and its own tooling bugs, lighter crossing models exist for embedded use, aliasing patterns expressible in C but not in safe Rust bound translation in principle, interoperation is the central open problem in embedded Rust, `unsafe` concentrates at exactly these boundaries, and the two largest documented adoptions into existing C and C++ report the resulting friction empirically [**49, 50, 99, 100, 101, 102, 80, 81**].

Claim it carries. aliasing patterns expressible in C but not in safe Rust bound translation in principle

Source says.

> *Crossref-deposited abstract: “existing translation methods are **severely limited by a lack of precision in the Rust compiler’s safety checker, causing many safe pointer manipulations to be labeled as potentially unsafe** … increasing the number of pointers that can be translated to safe Rust references by 75% over the baseline (from 12% to 21% of all pointers).”*

Metadata. [verified] Crossref matches every field.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[101] `sharma2024embedded` peer reviewed, level A

Reference. A. Sharma et al. “Rust for Embedded Systems: Current State and Open Problems”. In: *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2024, pp. 2296–2310. DOI: 10.1145/3658644.3690275. URL: https://doi.org/10.1145/3658644.3690275

Entry. DOI 10.1145/3658644.3690275.

Cited in. 1 place.

> **Full draft, Appendix B.** The seam carries its own defect classes and its own tooling bugs, lighter crossing models exist for embedded use, aliasing patterns expressible in C but not in safe Rust bound translation in principle, interoperation is the central open problem in embedded Rust, `unsafe` concentrates at exactly these boundaries, and the two largest documented adoptions into existing C and C++ report the resulting friction empirically [**49, 50, 99, 100, 101, 102, 80, 81**].

Claim it carries. interoperation is the central open problem in embedded Rust

Source says.

> *(arXiv 2311.05063, p. 1, abstract): “**Rust interoperability enables developing Rust applications on top of existing C codebases** … we found that existing Rust software support is inadequate, SAST tools cannot handle certain features … and **the prevalence of advanced types in existing Rust software makes it challenging to engineer interoperable code**.”*

Metadata. [verified] Crossref matches (booktitle wording differs cosmetically).

Verdict. OK.

Record. Sources_evidence.md, entry record.

[102] `mccormack2026unsafe` peer reviewed, level A

Reference. I. McCormack et al. "A Mixed Methods Study on the Implications of Unsafe Rust for Interoperation, Encapsulation, and Tooling". In: *ACM Transactions on Software Engineering and Methodology* (2026). DOI: 10.1145/3798277. URL: https://doi.org/10.1145/3798277

Entry. DOI 10.1145/3798277.

Cited in. 1 place.

> **Full draft, Appendix B.** The seam carries its own defect classes and its own tooling bugs, lighter crossing models exist for embedded use, aliasing patterns expressible in C but not in safe Rust bound translation in principle, interoperation is the central open problem in embedded Rust, `unsafe` concentrates at exactly these boundaries, and the two largest documented adoptions into existing C and C++ report the resulting friction empirically [**49, 50, 99, 100, 101, 102, 80, 81**].

Claim it carries. `unsafe` concentrates at exactly these boundaries

Source says.

> *(arXiv 2404.02230, p. 1, abstract): "we study **how developers reason about foreign function calls**, the limitations of the tools that they currently use ... **limited tooling support for foreign function calls made participants uncertain about their design choices, and certain foreign aliasing and concurrency patterns were difficult to encapsulate**."*

Metadata. [verified] Crossref matches, no volume/issue yet, correctly omitted.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[103] `hong2025ctorust` peer reviewed, level B

Reference. J. Hong and S. Ryu. "Automatically Translating C to Rust". In: *Communications of the ACM* 68.11 (2025), pp. 58–65. DOI: 10.1145/3737696. URL: https://doi.org/10.1145/3737696

Entry. DOI 10.1145/3737696.

Cited in. 1 place.

> **Full draft, Appendix B.** Automating the rewrite is not a solved problem either, since current translators still emit code whose safety the compiler cannot validate [**103**].

Source says.

> *Hong & Ryu, CACM 68(11):58–65, 2025. "Automatically Translating C to Rust". Peer-reviewed home for the RQ4 translation story, in place of leaning on preprints (§6.2).*

Record. LITERATURE_NOTES.md, peer-reviewed additions of the third sweep.

---

[104] `huang2017predictability` peer reviewed, level B

Reference. J. Huang et al. "A Top-Down Approach to Achieving Performance Predictability in Database Systems". In: *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD)*. 2017, pp. 745–758. DOI: 10.1145/3035918.3064016. URL: https://doi.org/10.1145/3035918.3064016

Entry. DOI 10.1145/3035918.3064016.

Cited in. 1 place.

> **Full draft, Appendix B.** The fourth strand is the domain and the method: the tail-latency and microsecond-scale literature [**6, 7, 30, 104**], thread-per-core structuring moving the tail in its own right [**45**], O-RAN disaggregation [**3**], control-plane latency as an active target in the core network too [**105, 106**], adoption evidence pointing at the learning curve rather than run-time cost [**78, 79, 107**], and standard guidance on repeat-based evaluation and fault injection [**32, 59**].

Claim it carries. part of the microsecond-scale predictability literature

Source says.

> *(deposited abstract): "While much of the research on transaction processing has focused on improving overall performance in terms of throughput and mean latency, surprisingly less attention has been given to **performance predictability: how often individual transactions exhibit execution latency far from the mean**."*

Verdict. OK. Latency far from the mean rather than the mean itself is exactly the distinction the thesis cites it for.

Record. Sources_evidence.md, entry record.

---

[105] `liu2023l25gc` peer reviewed, level B

Reference. Y.-S. Liu et al. "L^{2}5GC+: An Improved, 3GPP-Compliant 5G Core for Low-Latency Control Plane Operations". In: *Proceedings of the 2023 IEEE 12th International Conference on Cloud Networking (CloudNet)*. 2023, pp. 203–211. DOI: 10.1109/CloudNet59005.2023.10490024. URL: https://doi.org/10.1109/CloudNet59005.2023.10490024

Entry. DOI 10.1109/CloudNet59005.2023.10490024.

Cited in. 1 place.

> **Full draft, Appendix B.** The fourth strand is the domain and the method: the tail-latency and microsecond-scale literature **[6, 7, 30, 104]**, thread-per-core structuring moving the tail in its own right **[45]**, O-RAN disaggregation **[3]**, control-plane latency as an active target in the core network too **[105, 106]**, adoption evidence pointing at the learning curve rather than run-time cost **[78, 79, 107]**, and standard guidance on repeat-based evaluation and fault injection **[32, 59]**.

Claim it carries. control-plane latency is an active target in the core network

Source says.

> *(deposited abstract): "the **control plane in 5G core (5GC) still presents challenges** due to inefficiencies in handling control plane operations (including session establishment, handovers and idle-to-active state-transitions) … introduces substantial overheads that **impact latency**."*

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[106] `park2025ctrlbench` peer reviewed, level B

Reference. C. Park and J. Nam. "CtrlBench: Towards Accurate API-Level Benchmarking in 5G Core Control Plane". In: *Proceedings of the 2025 16th International Conference on Information and Communication Technology Convergence (ICTC)*. 2025, pp. 1832–1837. DOI: 10.1109/ICTC66702.2025.11388531. URL: https://doi.org/10.1109/ICTC66702.2025.11388531

Entry. DOI 10.1109/ICTC66702.2025.11388531.

Cited in. 1 place.

> **Full draft, Appendix B.** The fourth strand is the domain and the method: the tail-latency and microsecond-scale literature **[6, 7, 30, 104]**, thread-per-core structuring moving the tail in its own right **[45]**, O-RAN disaggregation **[3]**, control-plane latency as an active target in the core network too **[105, 106]**, adoption evidence pointing at the learning curve rather than run-time cost **[78, 79, 107]**, and standard guidance on repeat-based evaluation and fault injection **[32, 59]**.

Claim it carries. control-plane latency is an active target, measured procedure by procedure

Source says.

> *(deposited abstract): "CtrlBench, a domain-specific **microbenchmarking tool for the 5G Core Control Plane** ... It can generate configurable load and chain-dependent APIs, giving granular, fine-grained, and reproducible latency measurements ... **Existing end-to-end metrics, such as Registration Time, compress entire procedures into a single number. This hides which Network Function or API actually causes delays.**"*

Verdict. OK, and the closing sentence is an independent statement of this thesis's own decomposition argument, that an end-to-end figure hides where the cost sits.

Record. Sources_evidence.md, entry record.

---

[107] `ardito2021verbosity` peer reviewed, level B

Reference. L. Ardito et al. "Evaluation of Rust Code Verbosity, Understandability and Complexity". In: *PeerJ Computer Science* 7 (2021), e406. DOI: 10.7717/peerj-cs.406. URL: https://doi.org/10.7717/peerj-cs.406

Entry. DOI 10.7717/peerj-cs.406.

Cited in. 1 place.

> **Full draft, Appendix B.** The fourth strand is the domain and the method: the tail-latency and microsecond-scale literature **[6, 7, 30, 104]**, thread-per-core structuring moving the tail in its own right **[45]**, O-RAN disaggregation **[3]**, control-plane latency as an active target in the core network too **[105, 106]**, adoption evidence pointing at the learning curve rather than run-time cost **[78, 79, 107]**, and standard guidance on repeat-based evaluation and fault injection **[32, 59]**.

Claim it carries. adoption evidence points at the learning curve rather than run-time cost

Source says.

> *Crossref-deposited abstract: "The Rust code had an **average verbosity** in terms of the raw size of the code ... **Rust code had a better Cyclomatic Complexity, Halstead Metrics, and Maintainability Indexes than C and C++** but performed worse than the other considered object-oriented languages."*

Metadata. [verified] Crossref matches every field.

Verdict. OK.

Record. Sources_evidence.md, entry record.

---

[108] `qazi2017pepc` peer reviewed, level A

Reference. Z. A. Qazi et al. "A High Performance Packet Core for Next Generation Cellular Networks". In: *Proceedings of the Conference of the ACM Special Interest Group on Data Communication (SIGCOMM)*. 2017, pp. 348–361. DOI: 10.1145/3098822.3098848. URL: https://doi.org/10.1145/3098822.3098848

Entry. DOI 10.1145/3098822.3098848.

Cited in. 1 place.

> **Full draft, Appendix B.** Rust has mobile-network precedent in the datapath **[108]** and, on the control-plane side, an open-source gNB-CU with a separated CU-CP and CU-UP over E1.

Claim it carries. Rust has mobile-network precedent in the datapath

Source says.

> *(author copy, §4.1 Implementation): "NetBricks uses the Rust programming language [38], and **PEPC is also written in rust**, and runs inside NetBricks execution environment."*

Metadata. [verified] Crossref, pages 348-361. Author copy at `web.lums.edu.pk/~zafar/sigcomm17.pdf`.

Verdict. OK. Worth recording how close this came to being a defect: the Crossref-deposited abstract is truncated and never mentions Rust, so on the abstract alone this citation looks unsupported. The implementation section settles it outright. **An abstract that omits the implementation language is not evidence that the language is absent.**

Record. Sources_evidence.md, entry record.

---

[109] `alsoran` tool record, level A

Reference. N. Larkin. *Alsoran: A 5G gNodeB Centralized Unit Written in Rust.* Open-source software repository. Proof of concept, development discontinued. Accessed August 2026. URL: https://github.com/nplrkn/alsoran

Entry. source page. Proof of concept, development discontinued. Accessed August 2026.

Cited in. 1 place.

> **Full draft, Appendix B.** That last one is an existence proof and not a comparison, reporting no latency and carrying no C++ counterpart built to the same specification **[109]**.

Claim it carries. an open-source proof-of-concept gNB-CU with separated CU-CP and CU-UP over E1, reporting no latency and with no C++ counterpart

Source says.

> *(repository README): "Alsoran is a **Rust implementation of the gNodeB Centralized Unit (gNB-CU)** of the 5G Radio Access Network". "This project is currently a **proof of concept** and not yet a fully functional gNB-CU", under Current support: "**Separate CU-CP and CU-UP communicating using E1**", "Scale-out of GNB-CU-CP workers using multiple TNLAs", "ASN.1 libraries for NGAP, E1AP, F1AP and RRC", and the banner "This project is no longer active".*

Metadata. [verified] Repository description: "5G gNodeB Centralized Unit written in Rust". MIT licence, last pushed June 2025. The owner's GitHub account (`nplrkn`) shows the given name "Nic" only, the surname "Larkin" in the entry could not be confirmed from the repository itself, check the QCore project page if you want it airtight. Consider adding an access date, as the entry has no `year`.

Verdict. OK, the thesis's characterisation is accurate, including the absence of latency numbers.

Record. Sources_evidence.md, entry record.

Part 3. Findings and open items

Citations that were attached to a claim the source does not make

All six are closed as of 31 August 2026, and they are kept here because the pattern recurs. In every case the work itself was sound and cited correctly elsewhere, so each was closed by moving or dropping the citation rather than by replacing the source. No source was added that the bibliography did not already hold.

Four of the six existed only in `Thesis_v2` and the full draft had them right, which locates the cause: compressing a chapter into five paragraphs detaches a citation from the clause it was attached to. The two that reached the full draft are both in Chapter 1, which `Thesis_v2` carries as a copy. So the rule for the next compression pass is to re-read what each surviving citation now sits next to.

1. **Chapter 1, on Rust promising performance competitive with C++.**

   `klabnik2023rust`, `Thesis_v2` [10], draft [10]. `jung2018rustbelt`, `Thesis_v2` [11], draft [11].

   RustBelt is a machine-checked safety proof and makes no performance claim. The type-system half of the sentence was carried by the book. Nothing in the pair supported the performance half. The same sentence was in the full draft, because this chapter was copied from it, and in the plan in a different wording.

   Closed. The pair now carries only the compile-time guarantee, phrased as enforced "on machine-checked foundations", and the performance clause carries `balasubramanian2017rust`, which is the source the full draft already uses for that framing. The plan's sentence drops the performance claim, which its next paragraph states and cites on its own.

2. **Chapter 1, on thousands of procedures per second at a budget of a few milliseconds.**

   `3gpp38913`, `Thesis_v2` [5], draft [5].

   TR 38.913 clause 7.4 sets a control-plane latency target. It states no procedure rate, so the citation covered the second half of a compound sentence only.

   Closed. The sentence now reads that each procedure has to complete inside the millisecond-scale budget the 5G system sets for control-plane signalling, which is what clause 7.4 gives. The rate stands as this study's own workload assumption, in the wording already used later in the same chapter.

3. **Chapter 6, second use, on the seven boundary classes being chosen because safe Rust is expected not to prevent them.**

   `natella2016fault`, `Thesis_v2` [36], draft [59].

   Natella's survey of software fault injection is the right source for the method sentence in Chapter 4. It says nothing about the boundary of Rust's guarantee, and Chapter 3 already states the catalogue is this study's own. The full draft never had this one.

   Closed. The citation is dropped. The method sentence in Chapter 4 keeps it.

4. **Chapter 6, on the boundary defects being re-imported against the checker's own primitives and shrunk to model size.**

   `bornholt2021shardstore`, `Thesis_v2` [43], draft [70].

   ShardStore is the peer-reviewed account of Loom and Shuttle and supports the division of labour between them, which is how the full draft uses it. The rewrite and the shrink are this study's own measured cost.

   Closed. The citation moves onto the division of labour, naming Loom and Shuttle as the full draft does, and the rewrite cost is left as the study's own finding.

5. **Chapter 7, on confining unsafe to a seam crate making the reviewable surface a countable thing.**

   `evans2020rust`, `Thesis_v2` [52], draft [43].

   Evans et al. report that fewer than 30 per cent of crates use unsafe directly and that more than half cannot be fully checked because of unsafe in the call chain. All five sites in the full draft cite that for dependency propagation. The countability of the surface is this thesis's own guideline.

   Closed. The citation moves onto the dependency-propagation clause, matching the full draft, and the countable surface follows as the guideline it is.

6. **Chapter 7, on the parity suite bounding behaviour and not performance.**

   `emberg2023asn1`, `Thesis_v2` [33], draft [56].

   That limitation is this study's own, and both places the full draft states it carry no citation. Emberg and Wadsten compared matched C++ and Rust ASN.1 decompilers on performance.

   Closed. The citation is dropped. The entry's other use, on no earlier study isolating the language for a stateful concurrent subsystem, was sound and stands.

Entries whose supporting passage was never located

None. This list is closed.

The list was six entries until 31 August 2026, all of them student theses or a paywalled paper, and three of the defects above sat on three of those keys. Where nobody had written down what the source says, the prose drifted. Closing it took one afternoon, because five of the six were openly deposited with full PDFs. The correlation is the reason to keep this list at zero.

Level C, metadata confirmed and passage not opened

These 6 entries could not be closed without university access, and each is used for a title-level claim or inside a list. Three have no abstract deposited anywhere, so there is nothing short of the full text to read. Three have no DOI because the claim is a specification clause or a book chapter rather than a finding, and in each of those the citation sits beside a verified one that carries the same point.

- `3gpp38331` [4], v2 [4]
- `klabnik2023rust` [10], v2 [10]
- `3gpp22261` [28], v2 [34]
- `li2022crosslang` [49], v2 [37]
- `mo2023rustshyper` [93]
- `xiao2024rpkt` [94]

The one preprint still load-bearing

`bugden2022rust` is the last arXiv-only entry carrying weight. It is cited twice and both times beside a peer-reviewed source. Replacing it with a peer-reviewed equivalent is optional.

Differences that look like defects and are not

Recorded so no later pass "corrects" them.

- **Crossref year against issue year.** Crossref dates the POPL 2018 and POPL 2020 issues a year early because the deposit precedes the issue. The entries follow what ACM prints.
- **Article-number page ranges.** `66:1-66:34` against Crossref's `1-34`, and eight more like it. ACM prints the article number and the entries follow ACM.
- **Author names that differ from the publisher record.** D'Oro, O'Keeffe, Tao Wei, García Sánchez-Torija and di Luzio all differ between the publisher record and the paper's own title page. The entries follow the papers.
- **Semicolons in 3GPP titles.** `NG-RAN; Architecture description` and four like it keep their semicolons, because the punctuation is part of the official title. Changing them would misquote the standard.
- **HTTP 403 and 202 on publisher links.** Bot blocking rather than dead links. Every one of those DOIs resolves through the Crossref API.

Part 4. Coverage

Source kind	n	share
peer reviewed	72	66 %
standard	11	10 %
grey	8	7 %
thesis	7	6 %
tool record	5	5 %
committee draft	3	3 %
book	2	2 %
regulation	1	1 %
total	**109**	

Evidence	n
Level A, passage located	71
Level B, deposited record carries it	32
Level C, metadata only	6
No located passage	0
total	**109**

Every entry in the bibliography is cited somewhere, so there is no orphan and no citation without an entry. Across the three documents there are 256 citation commands naming 370 works between them, since one command often cites several. 72 entries (66 %) are in peer-reviewed venues. The full draft cites all 109, `Thesis_v2` cites 52 and the plan cites 31.